Cheatsheet · 50 chapters on one page

Artificial Intelligence cheatsheet

Every formula, key idea, table, memory aid and common mistake from the chapters, gathered in one place for quick revision. Nothing here is new. Each card links back to the lesson that teaches it, so open the lesson when a line does not make sense yet.

245 key ideas | 346 formulas | 168 tables | 112 tips and traps

STEP 1 Core AI · Intelligence, agents and problems

1.1 What AI is and intelligent agents

The four approaches to AI, rational agents, PEAS, environment types, the social machine and the five kinds of agent program.

KEY IDEAS

- **Definition** AI builds agents that perceive and act. This course uses the acting rationally view: choose the action with the best expected performance.
- **PEAS** Describe any task by Performance measure, Environment, Actuators, Sensors.
- **Environment types** Fully or partially observable, deterministic or stochastic, episodic or sequential, static or dynamic, discrete or continuous, single or multi agent.
- **Agent programs** Simple reflex, model based, goal based, utility based, learning. Table driven needs $\sum |P|^t$ rows, so it is impossible.
- **Rational, not perfect** Rationality depends on the performance measure, prior knowledge, available actions and percepts so far. Bad luck is not irrational.
- **Social machine** Needs etiquette, intuitive psychology, common sense, intuitive physics and layers from instinctive to self conscious (Minsky, Society of Mind).

FORMULAS

Agent function Exam favourite

$$f : P^* \rightarrow A$$

Maps every percept sequence (everything the agent has sensed so far, P*) to an action in the set A. It is the mathematical description of behaviour.

Lesson: Agents, environments and PEAS

Agent program

$$\text{agent} = \text{architecture} + \text{program}$$

The program runs on the physical machine (the architecture) and computes the agent function one percept at a time.

Lesson: Agents, environments and PEAS

Rational agent Exam favourite

$$\text{choose } a^* = \text{argmax}_a \, E[\text{performance} \mid \text{percepts so far}, a]$$

For every percept sequence, pick the action that is expected to give the best performance measure, given what the agent knows.

Lesson: Agents, environments and PEAS

Performance over a lifetime

$$U = \sum_{t=1}^{T} r_t$$

Add up the score r_t earned at each time step t. In the vacuum world r_t = number of clean squares (minus 1 per move if moving costs energy).

Lesson: The maths behind a rational agent

Expected performance of an action Exam favourite

$$E[U \mid a] = \sum_{s'} P(s' \mid a)\, U(s')$$

Weight each possible result s' by its probability. A rational agent picks the action with the largest value.

Lesson: The maths behind a rational agent

Size of a table driven agent Exam favourite

$$\text{rows} = \sum_{t=1}^{T} |P|^t = |P|(|P|^T - 1) / (|P| - 1)$$

|P| is the number of different percepts, T the lifetime. A geometric series, so it grows exponentially in T.

Lesson: The maths behind a rational agent

TABLES

Table: · like a human · rationally (the right thing)

	Like a human	Rationally (the right thing)
Thinking	Thinking humanly: model how people think (cognitive science, brain studies).	Thinking rationally: follow correct rules of reasoning (logic, the "laws of thought").
Acting	Acting humanly: behave so that people cannot tell it is a machine (the Turing test).	Acting rationally: choose the action expected to do best (the **rational agent** approach, used in this course).

Lesson: What intelligence and AI mean

Table: skill · what it means in plain words · example in the scene

Skill	What it means in plain words	Example in the scene
Social etiquette	Knowing polite and expected behaviour	Waiting for the person to finish speaking
Intuitive psychology	Sensing that someone is in trouble or worried, and wanting to help	Noticing the person is struggling and offering a hand
Awareness of its own limits (proprioception)	Knowing what its own body can and cannot do	Not trying to lift something too heavy
Common sense	Everyday facts everyone knows	Glass breaks if dropped
Domain knowledge	Expert facts about the task	How a machine is repaired
Intuitive physics	A feel for how objects move and fall	A ball rolls down a slope

Lesson: A social machine: the man machine symbiosis scene

Table: agent · performance · environment · actuators

Agent	Performance	Environment	Actuators	Sensors
Self driving taxi	Safe, fast, legal, comfortable, profit	Roads, traffic, people, weather	Steering, brake, accelerator, horn, display	Cameras, radar, GPS, speedometer
Medical diagnosis	Healthy patient, low cost	Patient, hospital, staff	Questions, tests, treatments	Symptoms, test results, answers
Spam filter	Spam caught, real mail kept	Email stream, users	Move to spam, deliver	Words, sender, links
Vacuum robot	Clean squares over time, little energy	Two squares A and B with dirt	Left, Right, Suck, NoOp	Location sensor, dirt sensor

Lesson: Agents, environments and PEAS

Table: question · easy answer · hard answer · example of the hard case

Question	Easy answer	Hard answer	Example of the hard case
Can the agent see everything that matters?	Fully observable	Partially observable	Poker: you cannot see the other cards
Is the next state fixed by the action?	Deterministic	Stochastic (random)	A dice game, a slippery floor
Does each decision stand alone?	Episodic	Sequential	Chess: a move now matters later
Does the world change while the agent thinks?	Static	Dynamic	Taxi driving
Are states and times countable steps?	Discrete	Continuous	Steering angle of a car
How many agents?	Single agent	Multi agent (competitive or cooperative)	Chess (competitive), football team (cooperative)

Lesson: Kinds of environment and kinds of agent

Table: task · observable · deterministic · episodic

Task	Observable	Deterministic	Episodic	Static	Discrete	Agents
Crossword puzzle	Fully	Yes	Sequential	Static	Discrete	Single
Chess with a clock	Fully	Yes	Sequential	Semi	Discrete	Multi
Taxi driving	Partially	Stochastic	Sequential	Dynamic	Continuous	Multi
Image classification	Fully	Yes	Episodic	Static	Continuous	Single

Lesson: Kinds of environment and kinds of agent

Table: approach · strength · weakness · used today for

Approach	Strength	Weakness	Used today for
Thinking humanly	Explains human minds	Hard to test, we do not fully know how people think	Cognitive science, user models
Thinking rationally	Exact, provable logic	Not everything is certain, and logic alone can be slow	Theorem provers, verification
Acting humanly	Clear test (Turing test)	Copies human mistakes too, says little about how	Chatbots, games
Acting rationally	Measurable, general, works with uncertainty	Needs a good performance measure	Almost all modern AI systems

Lesson: How the rational agent view compares

Table: agent · memory · looks ahead? · good when

Agent	Memory	Looks ahead?	Good when
Table driven	Huge table	No	Never in practice, only as a thought experiment
Simple reflex	None	No	Fully observable, simple rules
Model based	Internal state	No	Partially observable worlds
Goal based	State + goal	Yes, by search or planning	Many steps are needed to reach a goal
Utility based	State + utility	Yes, weighs trade offs	Several goals, risk, uncertainty
Learning	Changes itself	Depends on the base agent	The world is unknown or changing

Lesson: How the rational agent view compares

TIPS AND TRAPS

REMEMBER In class the four boxes were explained like this: thinking humanly is how we think, thinking rationally is making correct decisions in our head, acting humanly is doing what we think, and acting rationally is working it out so that it leads to a real, measurable result.

Lesson: What intelligence and AI mean

COMMON MISTAKE A table driven agent stores one action for every possible percept sequence. With 4 percepts and a lifetime of T steps it needs $4 + 4^2 + \ldots + 4^T$ rows. For a taxi camera this number is larger than the atoms in the universe. Agent programs exist to produce good behaviour from a small program instead of a huge table.

Lesson: Kinds of environment and kinds of agent

IN THE EXAM A rational agent is not one that is always right. It is one that chooses the best action given what it knows. Bad luck does not make a choice irrational.

Lesson: From intuition to research

1.2 Problems as search

States, actions, start and goal, nodes and the frontier, tree against graph search, and the four properties of search.

KEY IDEAS

- **Search problem** State space, actions with costs $C(x, a, y) \geq \varepsilon > 0$, initial state, goal test (a condition). Heuristics are optional.
- **Node** STATE, PARENT, ACTION, PATH-COST g(n). Follow PARENT links to rebuild the path.
- **Tree against graph search** Tree search can loop on cycles. Graph search keeps an explored set and skips seen states.
- **Four properties** Completeness, optimality, time and space, measured with b, d and m. Full tree: $(b^{d+1} - 1) / (b - 1)$ nodes.
- **Water jugs** 14 of 20 states reachable from (0, 0). (1, 2) unreachable. Getting 2 gallons in the 4 gallon jug takes 6 actions.
- **Formulation** Search cannot choose the states for you. A good abstraction can shrink the space enormously (8 queens: 1.8×10^{14} to 2,057).

FORMULAS

Successor function Exam favourite

$$S(x) = \{\langle y, a\rangle : \text{action } a \text{ takes } x \text{ to } y\}$$

All states one action away from x, each tagged with the action used. Two different actions may reach the same state.

Lesson: Inputs, outputs and nodes of a search algorithm

Action cost

$$C(x, a, y) \geq \varepsilon > 0$$

The cost of going from x to y with action a. C = ∞ if a does not lead from x to y. Every cost is at least a small positive ε.

Lesson: Inputs, outputs and nodes of a search algorithm

Path cost Exam favourite

$$g(n) = \sum_i C(s_i, a_i, s_{i+1})$$

Add the costs along the path from the start to node n.

Lesson: Inputs, outputs and nodes of a search algorithm

Branching factor and depth

b = most children of any node, d = depth of the shallowest goal, m = largest depth

These three numbers measure every algorithm.

Lesson: The selection rule and the four properties

Nodes in a full tree

Exam favourite

$$1 + b + b^2 + \ldots + b^d = (b^{d+1} - 1)/(b - 1) = O(b^d)$$

The number of nodes up to depth d. It grows exponentially with d.

Lesson: The selection rule and the four properties

Size of a state space

$|S|$ = product of the number of values of each part of the state

Jugs: 4 values for gal3 (0 to 3) × 5 for gal4 (0 to 4) = 20 states, of which 14 are reachable from (0, 0).

Lesson: The maths behind search problems

8 puzzle reachable states

Exam favourite

$$9!/2 = 181{,}440$$

Nine tiles (with the blank) in 9! orders, but only half can be reached from a given start, because a move never changes the parity of the arrangement.

Lesson: The maths behind search problems

Paths are much bigger than states

Exam favourite

tree nodes up to depth d ≈ b^d while states may be few

In a grid with 4 moves, a tree search to depth d makes 4^d paths but only about $2d^2$ distinct cells are within d steps. This is why graph search remembers explored states.

Lesson: The maths behind search problems

TABLES

Table: why search · limits of search

Why search	Limits of search
Game playing programs based on search beat world champions.	Search does not tell you how to **formulate** the problem. Choosing the states and actions is up to you.
Many problems have no special algorithm. Casting them as search is often the easiest way.	Search only solves a problem once it is correctly formulated.
Local search gives good approximations for huge optimisation problems.	Without good knowledge (heuristics) it can be very slow.
Problem specific heuristics let search use extra knowledge, and planning can be cast as search.	

Lesson: Why search, and what a search problem is

Table: part · meaning · romania trip · water jugs

Part	Meaning	Romania trip	Water jugs
State space	All configurations the problem can be in, kept as computer data	The city you are in	Pairs (gal3, gal4), gallons in each jug
Actions (transitions)	Moves from one state to another	Drive to a neighbouring city	Fill, empty, or pour one jug into the other
Initial state and goal	Where you start and the condition you want	In Arad, goal: be in Bucharest	Start (0, 0), goal e.g. (0, 2) or (*, 3)
Heuristics	Hints that guide the search	Straight line distance to Bucharest	How far each jug is from the target amount

Lesson: The formalism: states, actions, start, goal

Table: property · question it answers

Property	Question it answers
Completeness	If a solution exists, will the algorithm always find one?
Optimality	Will it find the cheapest solution (when actions have costs)?
Time complexity	How many nodes does it generate before it finds a solution?
Space complexity	What is the largest number of nodes it must keep in memory?

Lesson: The selection rule and the four properties

Table: · tree search · graph search

	Tree search	Graph search
Remembers explored states?	No	Yes (explored set)
Can loop for ever on cycles?	Yes	No, on a finite state space
Memory	Only the frontier	Frontier plus every explored state
Best when	The space is a true tree, or memory is very tight	Many paths lead to the same state (maps, puzzles)

Lesson: How problem formulation choices compare

Table: formulation · states · comment

Formulation	States	Comment
Add one queen at a time anywhere	$64 \times 63 \times \ldots \times 57 \approx 1.8 \times 10^{14}$ sequences	Very wasteful
Add one queen per column, in a row not attacked	2,057 states	Same problem, tiny space
Complete state: 8 queens, one per column, move a queen in its column	$8^8 \approx 16.8$ million states	Used by local search (chapter 4.1, 5.3)

Lesson: How problem formulation choices compare

TIPS AND TRAPS

REMEMBER — The goal can be a **condition**, not one state. "(*, 3)" means any state where the 4 gallon jug holds 3, whatever the 3 gallon jug holds. The goal test is a function that returns true or false.

Lesson: The formalism: states, actions, start, goal

COMMON MISTAKE — Cycles can cause non termination. Arad → Sibiu → Arad → Sibiu ... is a valid path in tree search, so a search may never stop. Graph search fixes this by remembering explored states.

Lesson: Inputs, outputs and nodes of a search algorithm

IN THE EXAM — Exam questions often ask you to formulate a problem. Always list all four parts: states, actions (with costs), initial state, goal test. Say how big the state space is.

Lesson: From intuition to research

STEP 2 Core AI · Uninformed search

2.1 Breadth first and uniform cost search

BFS with a FIFO queue, UCS with a priority queue on g(n), when each is optimal, their cost, and why UCS tests the goal late.

KEY IDEAS

- **BFS** FIFO queue. Complete (finite b). Fewest actions, optimal only for equal costs. Time and space $O(b^d)$.
- **UCS** Priority queue on g(n). Complete and optimal if every cost $\geq \varepsilon > 0$. $O(b^{1 + \lfloor C^*/\varepsilon \rfloor})$.
- **Goal test timing** BFS: test at generation. UCS: test at expansion (Sibiu: 310 is replaced by 278).
- **Romania** BFS 450 via Fagaras. UCS 418 via Rimnicu Vilcea and Pitesti, after taking out 13 cities.
- **Optimality proof** Lemma 1: costs taken out never decrease. Lemma 2: cheaper paths come first. Lemma 3: first path to a state is cheapest.
- **Memory** b = 10, d = 10 needs about 10 TB. Memory runs out before time.

FORMULAS

Time of BFS

Exam favourite

$$1 + b + b^2 + \ldots + b^d = O(b^d)$$

With the goal test when a node is generated. If you test only when a node is taken out, the last level is generated too and the time is $O(b^{d+1})$.

Lesson: Is BFS complete and optimal, and what does it cost

Space of BFS

Exam favourite

$$O(b^{d-1}) \text{ explored} + O(b^d) \text{ frontier} = O(b^d)$$

Every node stays in memory. Memory, not time, is the real limit of BFS.

Lesson: Is BFS complete and optimal, and what does it cost

Complete and optimal

$$\text{if every action cost } C(x, a, y) \geq \varepsilon > 0$$

With zero cost actions UCS can loop for ever on a zero cost cycle.

Lesson: Uniform cost search

Time and space of UCS

Exam favourite

$$O(b^{1 + \lfloor C^*/\varepsilon \rfloor})$$

C* is the cost of the optimal solution and ε the smallest action cost. The search may go about C^*/ε levels deep. When all costs are equal this is b^{d+1}.

Lesson: Uniform cost search

Geometric series

Exam favourite

$$\sum_{i=0}^{d} b^i = (b^{d+1} - 1)/(b - 1) \leq b^d \cdot b/(b - 1)$$

The last level holds most of the nodes. For b = 10 it is about 90 percent of them.

Lesson: The maths: geometric series and the cost bound

Fraction of nodes in the last level

$$b^d/((b^{d+1} - 1)/(b - 1)) \to (b - 1)/b$$

For b = 2 half the nodes are in the last level. For b = 10, 90 percent.

Lesson: The maths: geometric series and the cost bound

UCS depth bound

$$\text{depth} \leq \lfloor C^*/\varepsilon \rfloor$$

A path of more than C^*/ε actions costs more than C*, so UCS never needs to expand it before the goal.

Lesson: The maths: geometric series and the cost bound

TABLES

Table: step · open

Step	OPEN
0	⟨(0,0)⟩
1	⟨(0,0),(3,0)⟩, ⟨(0,0),(0,4)⟩
2	⟨(0,0),(0,4)⟩, ⟨(0,0),(3,0),(0,0)⟩, ⟨(0,0),(3,0),(3,4)⟩, ⟨(0,0),(3,0),(0,3)⟩
3	⟨(0,0),(3,0),(0,0)⟩, ⟨(0,0),(3,0),(3,4)⟩, ⟨(0,0),(3,0),(0,3)⟩, ⟨(0,0),(0,4),(0,0)⟩, ⟨(0,0),(0,4),(3,4)⟩, ⟨(0,0),(0,4),(3,1)⟩

Lesson: Uninformed search and breadth first search

Table: depth d · nodes (b = 10) · time at 1 million nodes a second · memory at 1 kb a node

Depth d	Nodes (b = 10)	Time at 1 million nodes a second	Memory at 1 KB a node
2	110	0.11 ms	107 KB
4	11,110	11 ms	10.6 MB
6	10^6	1.1 s	1 GB
8	10^8	2 minutes	103 GB
10	10^{10}	3 hours	10 TB
12	10^{12}	13 days	1 PB
14	10^{14}	3.5 years	99 PB

Lesson: Is BFS complete and optimal, and what does it cost

Table: · breadth first · uniform cost

	Breadth first	Uniform cost
Frontier	FIFO queue	Priority queue on g(n)
Finds	Fewest actions	Lowest total cost
Goal test	At generation (saves a level)	At expansion (needed for optimality)
Complete	Yes, if b is finite	Yes, if every cost ≥ ε > 0
Time and space	$O(b^d)$	$O(b^{1 + \lfloor C^*/\varepsilon \rfloor})$
Romania answer	450 (3 roads)	418 (4 roads)

Lesson: BFS against UCS against Dijkstra

Table: · ucs · dijkstra

	UCS	Dijkstra
Stops	When the goal is taken out	After the cheapest path to every node is known
Graph given	Implicitly, by a successor function	Usually explicitly, all nodes known
Frontier starts with	Only the start	Often every node at ∞

Lesson: BFS against UCS against Dijkstra

TIPS AND TRAPS

COMMON MISTAKE Two lessons from the table. Memory runs out long before time does. And no amount of faster hardware saves you: each extra level multiplies the cost by b.

Lesson: Is BFS complete and optimal, and what does it cost

IN THE EXAM This is why UCS tests the goal when a path is taken OUT, not when it is generated. In the Sibiu example, Bucharest was generated with cost 310 before the 278 path existed. Testing at generation would have returned the wrong answer.

Lesson: Why uniform cost search is optimal: the three lemmas

2.2 Depth first, iterative deepening and bidirectional search

DFS, DLS, IDS and bidirectional search, their memory and time, path checking, and the full comparison table.

KEY IDEAS

- **DFS** Stack. O(b m) space, $O(b^m)$ time. Not complete on infinite or cyclic spaces, not optimal.
- **DLS** DFS with depth limit D. Returns solution, failure, or cutoff. Complete only if d ≤ D.
- **IDS** DLS with D = 0, 1, 2, ... Complete, optimal for equal costs, $O(b^d)$ time, O(b d) space.

- **IDS overhead** $\sum (d + 1 - i)\, b^i$, about $b/(b-1)$ times BFS. b = 10, d = 5: 123,456 against 111,111.
- **Path check** Drop a child already on the current path. Removes cycles, keeps O(b m) memory.
- **Bidirectional** Search from both ends, $O(b^{d/2})$. Needs reversible actions and one explicit goal.

FORMULAS

Nodes generated by IDS

Exam favourite

$$N(IDS) = (d + 1)\cdot 1 + d\cdot b + (d - 1)\cdot b^2 + \ldots + 1\cdot b^d = \sum_{i=0}^{d} (d + 1 - i)\, b^i$$

Level i is generated in every iteration from i to d, that is d + 1 − i times.

Lesson: Is iterative deepening wasteful

Overhead of IDS over BFS

Exam favourite

$$N(IDS)/N(BFS) \approx b/(b - 1)$$

For b = 10 that is 1.11, only 11 percent more work. For b = 2 it is 2 times.

Lesson: Is iterative deepening wasteful

Cost of IDS

$$\text{time } O(b^d), \text{ space } O(b\ d)$$

Complete when b is finite, and optimal when all actions cost the same.

Lesson: Is iterative deepening wasteful

Cost of bidirectional search

Exam favourite

$$2 \cdot b^{d/2} = O(b^{d/2}) \text{ instead of } O(b^d)$$

For b = 10 and d = 6 that is 2,000 nodes against 1,000,000.

Lesson: Bidirectional search

TABLES

Table: property · dfs · why

Property	DFS	Why
Complete	No on infinite or cyclic spaces (tree search). Yes on finite spaces with graph search or a path check.	It can follow an infinite branch for ever.
Optimal	No	It returns the first goal it meets, which may be deep and costly.
Time	$O(b^m)$	m is the largest depth. It may explore the whole tree, and m can be much bigger than d.
Space	O(b m)	Only one path plus the untried children of each node on it.

Lesson: Depth first search

Table: check · what it does · memory · what it misses

Check	What it does	Memory	What it misses
Parent check	Do not go straight back to the state you came from	None	Longer cycles like A → B → C → A
Path check	Do not add a child whose state is already on the current path	O(m), the path is already stored	Two different paths to the same state (redundant paths)
Full cycle check (graph search)	Do not add a child whose state was ever explored	O(number of states)	Nothing, but memory grows like BFS

Lesson: Path checking and cycle checking

Table: dls result · meaning

DLS result	Meaning
A solution	A goal was found at depth ≤ D
Failure	No solution exists at all (nothing was cut off)
Cutoff	No solution within the limit, but deeper nodes were cut off, so one may exist deeper

Lesson: Depth limited and iterative deepening search

Table: criterion · bfs · ucs · dfs

Criterion	BFS	UCS	DFS	DLS	IDS	Bidirectional
Complete	Yes (finite b)	Yes (costs ≥ ε)	No	No (only if d ≤ l)	Yes (finite b)	Yes (with BFS both ways)
Optimal	Equal costs only	Yes	No	No	Equal costs only	Equal costs only
Time	$O(b^d)$	$O(b^{1 + \lfloor C^*/\varepsilon \rfloor})$	$O(b^m)$	$O(b^l)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^d)$	$O(b^{1 + \lfloor C^*/\varepsilon \rfloor})$	O(b m)	O(b l)	O(b d)	$O(b^{d/2})$

Lesson: All five uninformed methods side by side

TIPS AND TRAPS

REMEMBER Backtracking search, a version of DFS that generates one child at a time and undoes changes when it goes back, needs only O(m) memory. It is the basis of CSP solvers (chapter 5.2) and Prolog.

Lesson: Depth first search

IN THE EXAM A path check makes DFS complete on any finite state space, while keeping memory at O(b m). It can still visit the same state many times along different paths, so the time may be much more than the number of states.

Lesson: Path checking and cycle checking

REMEMBER IDS is the preferred uninformed method when the search space is large and the depth of the solution is not known (slide 49).

Lesson: Depth limited and iterative deepening search

COMMON MISTAKE When the frontiers first meet, the joined path is not always the shortest. With uniform costs, finish the current level before stopping, then pick the best meeting point.

Lesson: Bidirectional search

STEP 3 Core AI · Informed search

3.1 Greedy best first and A* search

Greedy best first, A*, admissible and consistent heuristics, why A* is optimal, and weighted A*.

KEY IDEAS

- **Greedy best first** Order by h. Not optimal (Romania 450, slide 58 trap 110). Can loop in tree search.
- **A*** Order by f = g + h. Romania: Arad 366, Sibiu 393, RV 413, Fagaras 415, Pitesti 417, Bucharest 418.
- **Admissible** h(n) ≤ h*(n). Makes A* tree search optimal.
- **Consistent** h(n1) ≤ C(n1, a, n2) + h(n2). f never decreases, graph search optimal, implies admissible.
- **Optimally efficient** A* expands every node with f < C* and none with f > C*. No optimal method with the same h does better.
- **Weighted A*** f = g + w h, cost ≤ w C*.

FORMULAS

Evaluation function of A*

Exam favourite

$$f(n) = g(n) + h(n)$$

g(n) is the cost of the path of node n, h(n) the estimate from n to a goal. With h = 0 A* is UCS. With g ignored it is greedy.

Lesson: A* search: f(n) = g(n) + h(n)

Admissible

Exam favourite

$$h(n) \leq h^*(n) \text{ for every } n$$

h*(n) is the true cost of an optimal path from n to a goal (∞ if there is none). An admissible h never overestimates: it is optimistic. Also h(goal) = 0.

Lesson: Admissible and consistent heuristics

Consistent (monotone)

Exam favourite

$$h(n1) \leq C(n1, a, n2) + h(n2) \text{ for every action a from } n1 \text{ to } n2$$

A triangle inequality: the estimate cannot drop by more than the cost of a step. It must hold for every action between n1 and n2.

Lesson: Admissible and consistent heuristics

Consistency implies admissibility

$$(\forall\ n1, n2, a)\ h(n1) \leq C(n1, a, n2) + h(n2) \Rightarrow (\forall\ n)\ h(n) \leq h^*(n)$$

Proof by induction on the number of steps to the goal (below). The other direction is false.

Lesson: Admissible and consistent heuristics

What A* expands

$$\text{surely every n with } f(n) < C^*, \text{ maybe some with } f(n) = C^*, \text{ never any with } f(n) > C^*$$

With a consistent h.

Lesson: Is A* optimal, and how fast is it

Optimally efficient

Exam favourite

$$\text{no other optimal algorithm using the same h expands fewer nodes (up to ties)}$$

Any algorithm that skips a node with f(n) < C* risks missing a cheaper solution.

Lesson: Is A* optimal, and how fast is it

Effective branching factor

$$N + 1 = 1 + b^* + (b^*)^2 + \ldots + (b^*)^d$$

If A* generates N nodes for a solution at depth d, b* measures how well h focuses the search. b* close to 1 is ideal.

Lesson: Is A* optimal, and how fast is it

Weighted A*

$$f(n) = g(n) + w \cdot h(n),\ w \geq 1$$

w = 1 is A*. A larger w trusts h more, so the search is faster and greedier.

Lesson: The maths: weighted A* and the cost of a bad h

Bounded suboptimality

Exam favourite

$$\text{cost found} \leq w \cdot C^*$$

With an admissible h, weighted A* is at most w times worse than the optimum. w = 1.5 means at most 50 percent more cost.

Lesson: The maths: weighted A* and the cost of a bad h

Error and work

$$\text{if } |h^* - h| \leq c \text{ for a constant c, the work is about } b^c, \text{ not } b^d$$

Constant absolute error still costs exponential in c. Relative error $\varepsilon = (h^* - h)/h^*$ gives about $b^{\varepsilon d}$.

Lesson: The maths: weighted A* and the cost of a bad h

TABLES

Table: property · greedy best first

Property	Greedy best first
Complete	No in tree search (Iasi, Neamt loop). Yes in graph search on a finite space.
Optimal	No (450 against 418, 110 against 30)
Time and space	$O(b^m)$ in the worst case, but often far less with a good h

Lesson: Greedy best first search

Table: · ucs · greedy best first · a*

	UCS	Greedy best first	A*
Order by	g(n)	h(n)	g(n) + h(n)
Looks at	The past only	The future only	Both
Optimal	Yes	No	Yes, with admissible h (tree) or consistent h (graph)
Complete	Yes (costs ≥ ε)	Only in graph search on finite spaces	Yes (costs ≥ ε, finite b)
Romania, Arad to Bucharest	418, 13 cities taken out	450, 4 cities	418, 6 cities
Slide 58 trap	30	110	30

Lesson: Greedy against UCS against A*

TIPS AND TRAPS

IN THE EXAM Food for thought IV asks for necessary and sufficient conditions. For optimal A*: admissibility is sufficient for tree search, consistency is sufficient for graph search. Neither is necessary: a slightly overestimating h can still return the optimal path by luck, but then nothing is guaranteed.

Lesson: Heuristics: knowing roughly where the goal is

COMMON MISTAKE The time of A* is still exponential in general, and the memory is its real weak point: it keeps every generated node. That is why chapter 3.3 builds IDA*, RBFS and SMA*.

Lesson: Is A* optimal, and how fast is it

3.2 Designing and comparing heuristics

8 puzzle heuristics, relaxed problems, dominance, max, pattern databases and the effective branching factor.

KEY IDEAS

- **8 puzzle heuristics** h1 misplaced tiles, h2 Manhattan distance. Textbook start: h1 = 8, h2 = 18, true cost 26.
- **Relaxed problem** Remove rules. The exact relaxed cost is admissible and consistent.
- **Dominance** h2 ≥ h1 and both admissible: A* with h2 expands no more nodes.
- **Combining** max of admissible heuristics is admissible and dominates each one.
- **Effective branching factor** $N + 1 = 1 + b^* + \ldots + (b^*)^d$. Close to 1 is good.
- **Pattern database** Exact costs of a sub problem, looked up during search. Disjoint PDBs can be added.

FORMULAS

h1: misplaced tiles

$$h1(n) = \text{number of tiles not in their goal square}$$

Every misplaced tile needs at least one move.

Lesson: Two heuristics for the 8 puzzle

h2: Manhattan distance Exam favourite

$$h2(n) = \sum_{\text{tiles}} (|\text{row} - \text{goal row}| + |\text{column} - \text{goal column}|)$$

Every tile needs at least as many moves as its distance in rows plus columns.

Lesson: Two heuristics for the 8 puzzle

Dominance Exam favourite

h2 dominates h1 if $h2(n) \geq h1(n)$ for every n, and both are admissible

Then A* with h2 expands no more nodes than A* with h1 (apart from ties).

Lesson: Dominance and combining heuristics

Why

A* surely expands every n with $f(n) < C^*$, that is $h(n) < C^* - g(n)$

A node with $h2(n) < C^* - g(n)$ also has $h1(n) < C^* - g(n)$. So every node expanded with h2 is also expanded with h1.

Lesson: Dominance and combining heuristics

Combining Exam favourite

$h(n) = \max(h1(n), ..., hk(n))$

The max of admissible heuristics is admissible, and it dominates each of them. The max of consistent heuristics is consistent.

Lesson: Dominance and combining heuristics

Disjoint pattern databases Exam favourite

h = PDB(tiles 1 to 4) + PDB(tiles 5 to 8)

Adding two PDBs is admissible only if each counts only the moves of its own tiles. Then no move is counted twice.

Lesson: Pattern databases and learned heuristics

Size of a PDB

for k pattern tiles plus the blank on 9 squares: $9! / (9 - k - 1)!$ entries

For k = 4: $9 \times 8 \times 7 \times 6 \times 5 = 15{,}120$ entries.

Lesson: Pattern databases and learned heuristics

Effective branching factor b* Exam favourite

$N + 1 = 1 + b^* + (b^*)^2 + ... + (b^*)^d$

N nodes generated for a solution at depth d. b* is the branching factor a uniform tree of depth d would need to hold N + 1 nodes. A good heuristic has b* close to 1.

Lesson: The maths: effective branching factor

TABLES

Table: real rule: a tile can move from square a to square b if · remove this part · heuristic you get

Real rule: a tile can move from square A to square B if	Remove this part	Heuristic you get
A is next to B and B is blank	"and B is blank"	h2: Manhattan distance
A is next to B and B is blank	"A is next to B"	Gaschnig's heuristic: number of swaps with the blank
A is next to B and B is blank	both conditions	h1: misplaced tiles (each tile jumps in one move)

Lesson: Relaxed problems: where heuristics come from

Table: problem · relaxation · heuristic

Problem	Relaxation	Heuristic
Romania road trip	You may fly in a straight line	Straight line distance h_{SLD}
Robot on a grid with walls	No walls	Manhattan or octile distance
Water jugs, get 2 litres	Pour any amount you like	0 if a jug holds 2, else 1
Travelling salesperson	The tour need not be a cycle	Cost of a minimum spanning tree

Lesson: Relaxed problems: where heuristics come from

Table: depth d · bfs nodes · a*(h1) nodes · a*(h2) nodes

Depth d	BFS nodes	A*(h1) nodes	A*(h2) nodes	b* BFS	b* h1	b* h2
2	7	6	6	2.15	2.00	2.00
6	81	17	15	1.83	1.31	1.26
10	602	72	34	1.74	1.35	1.21
12	1,580	164	57	1.72	1.38	1.23
16	10,198	953	207	1.68	1.42	1.27
20	47,880	5,508	725	1.63	1.45	1.29

Lesson: Dominance and combining heuristics

Table: heuristic · admissible · value at the start state · cost to compute

Heuristic	Admissible	Value at the start state	Cost to compute	Nodes at d = 20
h = 0 (uniform cost)	Yes	0	None	about 48,000 (BFS)
h1 misplaced tiles	Yes	8	9 checks	5,508
h2 Manhattan	Yes	18	9 small sums	725
max(h1, h2)	Yes	18 (= h2, since h2 dominates)	Both	725
Pattern database	Yes	At least as large as one group of tiles needs	One table lookup, after building the table	Much fewer
h* (perfect)	Yes	26	As hard as the problem	26 (straight to the goal)

Lesson: Comparing heuristics

TIPS AND TRAPS

REMEMBER The relaxed problem must be easy to solve, or computing h will cost more than the search saves. Manhattan distance is a sum of independent tile problems, which takes 9 small steps.

Lesson: Relaxed problems: where heuristics come from

COMMON MISTAKE Larger is better only while h stays admissible. 2 × h2 is larger, but it overestimates, so A* may then return a longer solution.

Lesson: Comparing heuristics

3.3 Memory bounded heuristic search: IDA*, RBFS and SMA*

IDA* with f cutoffs, RBFS with f-limits and backed up values, SMA*, and the memory against time trade.

KEY IDEAS

- **Why** A* stores every node and runs out of memory first. Memory bounded methods trade memory for repeated work.
- **IDA*** DFS with an f cutoff, next cutoff = smallest f over the old one. Romania: 366, 393, 413, 415, 417, 418.
- **RBFS** f-limit = min(limit, best alternative). Unwind when best f > limit and back up that f. Linear memory.
- **SMA*** A* until memory is full, then drop the highest f leaf and back up its f. Optimal if the solution fits.
- **Costs** IDA* and RBFS: O(b d) memory. IDA* worst case $O(N^2)$ time with many distinct f values.

FORMULAS

Iterations of IDA*

Exam favourite

$$\text{one per distinct f value up to } C^*$$

With integer costs and a good h that is few. With many different real valued costs, each iteration may add only one new node, and IDA* can take about N^2 time for N nodes A* would expand.

Lesson: IDA*: iterative deepening A*

Memory

Exam favourite

$$A^*: O(b^d)\ \text{IDA}^*, \text{RBFS: } O(b\ d)\ \text{SMA}^*: \text{M, the memory you give it}$$

b branching factor, d solution depth.

Lesson: The maths: memory and repeated work

IDA* number of iterations

$$k = \text{number of distinct f values in } (h(\text{start}), C^*]$$

Romania: 366, 393, 413, 415, 417, 418 gives 6 iterations.

Lesson: The maths: memory and repeated work

Worst case of IDA*

$$1 + 2 + \ldots + N = N(N+1)/2 = O(N^2)$$

If each iteration adds just one new node to the N nodes A* expands.

Lesson: The maths: memory and repeated work

TABLES

Table: method · idea · memory

Method	Idea	Memory
IDA*	Iterative deepening, but the cutoff is on f = g + h instead of depth	O(b d), linear
RBFS	Best first search written as a recursion that remembers only the f of the best alternative	O(b d), linear
SMA*	A* that uses all the memory it has, and drops the worst leaf when memory is full	Whatever you give it

Lesson: Why A* runs out of memory

Table: property · sma*

Property	SMA*
Complete	Yes, if the depth of the shallowest goal is less than the memory size (the path must fit)
Optimal	Yes, if an optimal solution fits in memory. Otherwise it returns the best solution that fits.
Weak point	When memory is very tight, it can thrash: switch back and forth between paths and regenerate them many times

Lesson: SMA*: use all the memory you have

Table: · bfs · ids · a*

	BFS	IDS	A*	IDA*	RBFS	SMA*
Uses h	No	No	Yes	Yes	Yes	Yes
Optimal	Equal costs	Equal costs	Yes	Yes (admissible h)	Yes (admissible h)	If the solution fits
Memory	$O(b^d)$	O(b d)	$O(b^d)$	O(b d)	O(b d)	M
Repeated work	None	Top levels per iteration	None	One iteration per f value	Each time it switches back	Only for dropped nodes
Romania, Arad to Bucharest	450	450	418	418, 6 iterations	418, 2 switches	418

Lesson: BFS against IDS against IDA* against RBFS

TIPS AND TRAPS

IN THE EXAM

The minor exam of this course was announced as a coding task, such as writing RBFS. The code in lesson 3 is short enough to learn by heart, and the notebook lets you practise it.

Lesson: Why A* runs out of memory

REMEMBER All three methods are a trade between time and memory. In practice, SMA* style methods are the most robust when memory is limited but not tiny.

Lesson: SMA*: use all the memory you have

3.4 AND OR search and AO*

AND OR trees, conditional and cyclic plans, AND OR search on the erratic vacuum world, and AO* on the slide example.

KEY IDEAS

- **AND and OR nodes** OR: the agent chooses, one child must work. AND: nature chooses, every child must work.
- **Contingency plan** Erratic vacuum from state 1: [Suck, if State = 5 then [Right, Suck] else []].
- **AND OR search** DFS that fails a branch on a state already on its path. Complete on finite spaces.
- **Cyclic plan** Slippery world: [Suck, L1: Right, if State = 5 then L1 else Suck]. Needs fairness.
- **AO*** Expand the best partial graph, AND = sum, OR = min, revise upward. Slide example: 6, then 9 against 8, ends at 5. Optimal only with admissible h.

FORMULAS

A cyclic plan Exam favourite

[Suck, L1: Right, if State = 5 then L1 else Suck]

L1 is a label. "if State = 5 then L1" means go back to the label and try Right again. In code it is a while loop: while State = 5: Right.

Lesson: Slippery worlds and cyclic plans

When a cyclic plan is a solution

every leaf is a goal, and a goal can be reached from every point of the plan

Each try must succeed with some chance. If the floor is so slippery that Right never works, no plan helps.

Lesson: Slippery worlds and cyclic plans

Cost at an OR node

$$\text{cost}(n) = \min_{\text{connectors } k} \text{cost}(n, k)$$

The agent picks the cheapest way.

Lesson: The maths: cost of a solution graph and complexity

Cost of an AND connector Exam favourite

$$\text{cost}(n, k) = \sum_{\text{children } c \text{ of } k} (C(n, c) + \text{cost}(c))$$

All parts must be done, so their costs add.

Lesson: The maths: cost of a solution graph and complexity

Admissibility for AO*

$h(n) \leq$ the true cost of the best solution graph below n

With such an h, AO* returns an optimal solution graph (on graphs without cycles).

Lesson: The maths: cost of a solution graph and complexity

TABLES

Table: · or node · and node

	OR node	AND node
Who chooses	The agent	Nature (the environment)
Solved when	At least one child is solved	Every child is solved
Cost	The cheapest child (plus the edge)	The sum over all children (plus the edges)
Drawn as	Separate branches	Branches joined by an arc

Lesson: When actions do not always do the same thing

Table: · a* or bfs · and or search (dfs) · ao*

	A* or BFS	AND OR search (DFS)	AO*
Actions	Deterministic	Non deterministic, or problems split into parts	Same as AND OR, with costs
Solution	A path (a list of actions)	A tree: a conditional plan	The cheapest solution graph
Uses h	A*: yes	No	Yes
Handles loops	Explored set	Fails on a repeated state, cyclic plans need labels	Classic AO* assumes no cycles (LAO* handles them)
Example	Romania route	Erratic vacuum plan	Slide graph, cost 5

Lesson: AND OR search against ordinary search

Table: · and or (contingency) · markov decision process (chapter 10.1)

	AND OR (contingency)	Markov decision process (chapter 10.1)
Outcomes	A set of possible states	States with probabilities
Goal	Reach a goal in every outcome	Maximise expected reward
Result	Conditional plan	Policy for every state

Lesson: AND OR search against ordinary search

TIPS AND TRAPS

REMEMBER Failing on a state already on the path does not miss solutions: if a plan exists from a repeated state, it also exists from its first occurrence. That keeps AND OR search complete on finite state spaces.

Lesson: The erratic vacuum world and AND OR search

COMMON MISTAKE A cyclic plan only works under a **fairness** assumption: an action that can succeed will succeed if you try it often enough. If the failure has a hidden cause (the wheel is broken), trying again is useless and the agent must learn the cause instead.

Lesson: Slippery worlds and cyclic plans

COMMON MISTAKE Look closely at the slide numbers: h(D) = 4 but D really costs only 1 (D → J). So this h overestimates and is not admissible. Here AO* still ends at the best answer 5, but try the second tip (h(H) = 6): AO* stops at 9 while the true best is 7. Like A*, AO* is only guaranteed optimal when h never overestimates.

Lesson: AO*: best first search on AND OR graphs

STEP 4 Core AI · Local and evolutionary search

4.1 Local search: hill climbing and simulated annealing

Hill climbing and its traps, sideways moves, restarts, beam search, and simulated annealing with cooling schedules.

KEY IDEAS

- **Local search** One current state, an objective, neighbours. No path, constant memory, works on huge spaces.
- **Hill climbing** Best neighbour until none is better. Stuck at local maxima, ridges and plateaux. 8 queens: about 14 percent solved.
- **Fixes** Sideways moves (about 94 percent), random restarts ($1/p$ runs), stochastic and first choice, local beam search.
- **Simulated annealing** Accept worse with $e^{\Delta E/T}$. High T explores, low T exploits.

- **Cooling schedules** Geometric $T \leftarrow \alpha T$ ($0.8 \le \alpha \le 1$), linear $T0 - k\Delta T$, logarithmic $C / \ln(k + 2)$, constant, exponential $T0\ \alpha^k$.

FORMULAS

Random restart

Exam favourite

$$\text{expected restarts} = 1 / p$$

p is the chance one run succeeds. For 8 queens p ≈ 0.148, so about 7 runs: 6 failures (about 3 steps each) and 1 success (about 4 steps), roughly 22 steps in all.

Lesson: Ways out: sideways moves, randomness and restarts

Acceptance probability

Exam favourite

$$P(\text{accept}) = e^{\Delta E / T} \text{ for } \Delta E < 0, \text{ and } 1 \text{ for } \Delta E \ge 0$$

High T: P near 1, almost any move is accepted. Low T: P near 0, only uphill moves.

Lesson: Simulated annealing

Why the slide table

$$\text{as T grows, } \Delta E / T \to 0 \text{ and } e^{\Delta E/T} \to e^0 = 1$$

The slide lists e^x for x = 1, ½, ¼, ..., $1/1024$: 2.718, 1.649, 1.284, ..., 1.001. As the exponent shrinks, the value approaches 1.

Lesson: Simulated annealing

Gradient ascent for continuous states

Exam favourite

$$x \leftarrow x + \eta \nabla f(x)$$

In continuous spaces, move a small step η along the gradient. Too big η overshoots, too small is slow.

Lesson: The maths: acceptance, restarts and gradients

Newton Raphson step

$$x \leftarrow x - H_f(x)^{-1} \nabla f(x)$$

Uses the second derivatives (Hessian H) to choose the step. Very fast near an optimum.

Lesson: The maths: acceptance, restarts and gradients

Probability of success with r restarts

$$1 - (1 - p)^r$$

p = 0.148, r = 10: $1 - 0.852^{10} \approx 0.80$.

Lesson: The maths: acceptance, restarts and gradients

TABLES

Table: feature of the landscape · what it is · why it is a problem

Feature of the landscape	What it is	Why it is a problem
Local maximum	A peak higher than all its neighbours but lower than the global maximum	Every move goes down, so a greedy climber stops
Ridge	A narrow line of peaks with slopes on both sides	No single move goes up along the ridge, so the climber zig zags slowly
Plateau: flat local maximum	A flat area with no uphill exit	No neighbour is better
Plateau: shoulder	A flat area from which you can still go up later	The climber cannot see that, and stops unless it may move sideways

Lesson: Local search: forget the path, improve the state

Table: method on random 8 queens boards (our run, 2,000 boards) · solved · average steps when it works · average steps when stuck

Method on random 8 queens boards (our run, 2,000 boards)	Solved	Average steps when it works	Average steps when stuck
Steepest ascent hill climbing	14.8 percent	4.0	3.1
With up to 100 sideways moves	93.8 percent	19.3	60.9

Lesson: Hill climbing

Table: variant · idea · helps with

Variant	Idea	Helps with
Sideways moves	Allow moves to an equal valued neighbour, up to a limit (to avoid looping on a flat top)	Shoulders
Stochastic hill climbing	Pick a random uphill move, with more chance for steeper ones	Some ridges, slower but sometimes better
First choice hill climbing	Generate random neighbours until one is better, take it	States with thousands of neighbours
Random restart hill climbing	Run hill climbing again from new random starts, keep the best result	Local maxima. Complete with probability approaching 1
Local beam search	Keep k states. Generate all their successors, keep the best k overall	Shares information between the k searches
Stochastic beam search	Keep k successors chosen at random, better ones more likely	Stops all k states crowding onto one hill

Lesson: Ways out: sideways moves, randomness and restarts

Table: cooling schedule (slide 83) · formula · comment

Cooling schedule (slide 83)	Formula	Comment
Geometric	$T_{k+1} = \alpha\, T_k$, $0.8 \le \alpha \le 1$	The most common. α near 1 cools slowly and explores longer.
Linear	$T_k = T_0 - k\,\Delta T$	Drops by the same amount each step.
Logarithmic	$T_k = C / \ln(k + 2)$	C = the highest energy barrier to cross. Very slow, but with it annealing finds the global optimum with probability approaching 1.
Constant	$T_k = C$	This is the Metropolis algorithm, no cooling.
Exponential	$T_k = T_0\, \alpha^k$	The closed form of geometric cooling.

Lesson: Simulated annealing

Table: · hill climbing · simulated annealing · genetic algorithm

	Hill climbing	Simulated annealing	Genetic algorithm
Breadth of the search	One state, narrow	One state, wanders widely while hot	A population, broad
Tests worse solutions	Never	Yes, with probability $e^{\Delta E/T}$	Yes, weak individuals may still breed
Avoids local optima	No (only with restarts)	Yes, if cooled slowly	Often, through crossover and mutation
Memory	One state	One state	k states
Speed	Fastest per run	Medium, depends on the schedule	Slowest per generation
Prefer it when	The landscape is smooth or you can restart cheaply	There are many local optima and one good answer is enough	Good solutions are made of good parts that can be combined

Lesson: Comparing local search methods

TIPS AND TRAPS

REMEMBER

Local beam search is not k independent restarts. If one state finds a great region, its children can fill all k places, so the search moves its effort to where the progress is.

Lesson: Ways out: sideways moves, randomness and restarts

REMEMBER

Choose T0 so that most worse moves are accepted at the start, often from the average cost difference between neighbours. Too fast cooling turns annealing into plain hill climbing.

Lesson: Simulated annealing

4.2 Genetic algorithms

Population, fitness, selection, crossover and mutation, the slide x² example, and when a GA beats other local search.

KEY IDEAS

- **GA words** Gene, chromosome, population, fitness, selection, crossover, mutation.
- **Roulette wheel** $P(i) = f_i / \sum f$. Expected copies $f_i / \bar{f}$.
- **Slide example** Fitness 169, 576, 64, 361, total 1170, average 292.5. Children 01100, 11001, 11011 (729), 10000.
- **Other selections** Tournament, rank, elitism. Rank and tournament ignore the scale of fitness.
- **When GAs help** When good solutions are built from blocks crossover can combine. Watch diversity to avoid premature convergence.

FORMULAS

Fitness for 8 queens

$$\text{fitness} = 28 - (\text{number of attacking pairs})$$

28 is the number of pairs among 8 queens, $8 \times 7 / 2$.

Lesson: A GA for 8 queens

Search space for the digit string

$$8^8 \approx 16.8 \text{ million}$$

The same as for hill climbing on 8 queens.

Lesson: A GA for 8 queens

Roulette wheel probability Exam favourite

$$P(i) = f_i / \sum_j f_j$$

Expected copies of i in a population of n: $n\,P(i) = f_i / \bar{f}$.

Lesson: The maths: selection pressure and schemata

Schema theorem (Holland)

$$E[m(H, t+1)] \geq m(H, t) \cdot (f(H) / \bar{f}) \cdot [1 - p_c\, \delta(H) / (l - 1) - o(H)\, p_m]$$

m(H, t) individuals match schema H (like 1***1). f(H) is their average fitness, δ(H) the defining length, o(H) the order (fixed positions), l the string length, p_c and p_m the crossover and mutation rates.

Lesson: The maths: selection pressure and schemata

Chance a string escapes mutation

$$(1 - p_m)^l$$

With $p_m = 0.01$ and $l = 5$: $0.99^5 \approx 0.951$.

Lesson: The maths: selection pressure and schemata

TABLES

Table: word · meaning · in the x^2 example

Word	Meaning	In the x^2 example
Gene	One encoded decision value	One bit
Chromosome (individual)	A full candidate solution, a string of genes	01101
Population	The set of individuals in one generation	4 strings
Fitness function	How good an individual is (higher is better)	$f(x) = x^2$, with x the number the bits encode
Selection	Choosing parents, fitter ones more often	Roulette wheel
Crossover	Swap the tails of two parents after a cut point	0110|1 and 1100|0 give 01100 and 11001
Mutation	A small random change	Flip one bit

Lesson: Search by evolution

Table: string · x · $f(x) = x^2$ · $p(pick) = f / \sum f$

String	x	$f(x) = x^2$	$P(pick) = f / \sum f$	Expected copies = f / average	Actual copies
01101	13	169	0.14	0.58	1
11000	24	576	0.49	1.97	2
01000	8	64	0.05 (the slide rounds to 0.06)	0.22	0
10011	19	361	0.31	1.23	1
Total		1170	1.00	4.00	4
Average		292.5 (the slide rounds to 293)			
Maximum		576			

Lesson: The slide example: maximise x^2 with 5 bits

Table: selection method · how it works · strength · weakness

Selection method	How it works	Strength	Weakness
Roulette wheel (fitness proportionate)	$P(i) = f_i / \sum f$. Spin a wheel with slices sized by fitness	Simple, matches the theory	One very fit individual can take over too early. Needs positive fitness
Tournament	Pick k individuals at random, the fittest of them wins	Easy to tune with k, works with any fitness scale	Large k is very greedy
Rank	Sort by fitness, pick by rank, not by raw value	Not fooled by huge fitness gaps	Ignores how much better the best is
Elitism	Copy the best few unchanged into the next generation	Never loses the best found so far	Can reduce diversity

Lesson: Selection, crossover and mutation in detail

Table: operator (slide 84) · example · note

Operator (slide 84)	Example	Note
One point crossover	AB|CD and MN|OP give ABOP and MNCD	The cut point is chosen at random
Two point and uniform crossover	Swap a middle segment, or each gene with chance 0.5	Mix more freely
Flip mutation	01101 → 01001	For bit strings
Switch (swap) mutation	ABCD → ADCB	Keeps a permutation valid, for example a tour
Scramble mutation	ABCD → ACBD (shuffle a segment)	Bigger jump

Lesson: Selection, crossover and mutation in detail

Table: feature · hill climbing · simulated annealing · genetic algorithm

Feature	Hill climbing	Simulated annealing	Genetic algorithm
Inspiration	Climbing a hill	Annealing in metallurgy	Natural selection and evolution
Search space	Local	Broad (with controlled randomness)	Very broad (population based)
Moves to worse solutions	No	Yes (probabilistic)	Yes (via mutation and crossover)
Avoids local optima	No	Yes	Yes
Memory usage	Low	Medium	High
Parameter tuning needed	Minimal	High (temperature, cooling)	High (population, mutation rate)
Speed	Fast	Moderate	Slower (population evolution)
Complexity	Simple	Moderate	High
Best use case	Small or simple problems	Problems with many local optima	Complex optimisation problems

Lesson: Comparing hill climbing, annealing and genetic algorithms

TIPS AND TRAPS

COMMON MISTAKE The slide table lists x = 10 for the string 10000. In binary 10000 is 16, and $16^2 = 256$ matches the f column, so x should read 16.

Lesson: The slide example: maximise x^2 with 5 bits

COMMON MISTAKE For problems like the travelling salesperson, plain one point crossover makes invalid tours (a city twice). Use order crossover or swap mutation, which always give valid permutations.

Lesson: Selection, crossover and mutation in detail

REMEMBER Crossover helps when nearby genes form meaningful blocks, like the left half of the board. If the encoding scatters related decisions, crossover mostly breaks good solutions, and a GA is no better than random restarts.

Lesson: A GA for 8 queens

REMEMBER Short, low order schemata with above average fitness grow exponentially. These are the "building blocks" that crossover combines.

Lesson: The maths: selection pressure and schemata

STEP 5 Core AI · Constraint satisfaction

5.1 Constraint satisfaction problems and arc consistency

CSP formulation, constraint types, node and arc consistency, AC-3 and its three outcomes.

KEY IDEAS

- **CSP** Variables X, domains D, constraints C. Solution: complete and consistent assignment.
- **Examples** Australia: 3^7 = 2,187 assignments, 18 solutions. N queens with $X_{ij} \in \{0, 1\}$. Car assembly with precedence $T1 + d1 \leq T2$ and disjunctive constraints.
- **Arc consistency** $\forall x \in dom(X) \; \exists y \in dom(Y)$: r(x, y). AC-3 costs $O(c\, d^3)$.
- **Three outcomes** Empty domain: no solution. All singletons: unique solution. Otherwise search.
- **Forward checking against AC-3** FC checks arcs from the newly assigned variable only. AC-3 checks every arc and finds failures earlier.

FORMULAS

Arc consistency

Exam favourite

$$\forall\, x \in \mathrm{dom}(X)\ \exists\, y \in \mathrm{dom}(Y)\colon r(x, y)$$

Every value of X has a supporting value of Y. A network is arc consistent when all its arcs are.

Lesson: Node and arc consistency

The three outcomes (slide 6)

a domain is empty ⇒ no solution. Every domain has one value ⇒ the unique solution. Some domains have several values ⇒ search is still needed

Arc consistency alone does not always decide the problem.

Lesson: Node and arc consistency

Cost of AC-3

Exam favourite

$$O(c\, d^3)$$

c binary constraints, domains of size at most d. Each arc can be added to the queue at most d times (once per value deleted from its head), and REVISE costs d^2 checks.

Lesson: The AC-3 algorithm

Number of complete assignments

Exam favourite

$$d^n$$

n variables, domain size d. Australia: $3^7 = 2{,}187$.

Lesson: The maths: counting assignments and solutions

Naive search tree

Exam favourite

$$n! \cdot d^n \text{ leaves}$$

If any unassigned variable can be chosen at each level: n d choices at the top, (n − 1) d next, and so on. Fixing the variable order (assignments commute) brings it back to d^n.

Lesson: The maths: counting assignments and solutions

Solutions of the Australia map

$$3 \times 2 \times 3 = 18$$

Pick SA (3 ways). WA, NT, Q, NSW, V form a chain around SA and must alternate the other two colours (2 ways). T is free (3 ways).

Lesson: The maths: counting assignments and solutions

TABLES

Table: part · value

Part	Value
Variables	WA, NT, SA, Q, NSW, V, T
Domains	D_i = {red, green, blue}
Constraints	Neighbouring regions differ: WA ≠ NT, WA ≠ SA, NT ≠ SA, NT ≠ Q, SA ≠ Q, SA ≠ NSW, SA ≠ V, Q ≠ NSW, NSW ≠ V
A solution	WA = red, NT = green, SA = blue, Q = red, NSW = green, V = red, T = green

Lesson: What a constraint satisfaction problem is

Table: part · formulation

Part	Formulation
Variables	X_{ij} for $0 \le i, j < N$, one for each square (row i, column j)
Domains	{0, 1}: 1 means a queen stands on the square
Same row	$\forall\, i, j, k$: $(X_{ij}, X_{ik}) \in \{(0, 0), (0, 1), (1, 0)\}$, at most one queen
Same column	$\forall\, i, j, k$: $(X_{ij}, X_{kj}) \in \{(0, 0), (0, 1), (1, 0)\}$
Diagonals	$(X_{ij}, X_{i+k,\, j+k})$ and $(X_{ij}, X_{i+k,\, j-k}) \in \{(0, 0), (0, 1), (1, 0)\}$
Count	$\sum_{i,\, j} X_{ij} = N$, exactly N queens

Lesson: More CSPs: N queens and job scheduling

Table: part · formulation

Part	Formulation
Variables	$Axle_F$, $Axle_B$, $Wheel_{RF}$, $Wheel_{LF}$, $Wheel_{RB}$, $Wheel_{LB}$, Nuts for each wheel, Cap for each wheel, Inspect: 15 tasks, each value is a start time
Precedence	$T_1 + d_1 \leq T_2$: task 1 must finish before task 2 starts. $Axle_F + 10 \leq Wheel_{RF}$, $Wheel_{RF} + 1 \leq Nuts_{RF}$, $Nuts_{RF} + 2 \leq Cap_{RF}$, ...
Disjunctive	The two axles share one tool: $Axle_F + 10 \leq Axle_B$ or $Axle_B + 10 \leq Axle_F$
Inspection last	$X + d_X \leq$ Inspect for every other task X, and Inspect takes 3 minutes
Deadline 30 minutes	$D_i = \{1, 2, 3, ..., 27\}$, so even Inspect ends by minute 30

Lesson: More CSPs: N queens and job scheduling

Table: kind of constraint · example

Kind of constraint	Example
Unary (one variable)	SA ≠ green, B ≠ 3
Binary (two variables)	WA ≠ NT, A < B
Global (any number)	Alldiff(all cells in a Sudoku row)
Preference (soft)	Prof. R prefers morning lectures. Breaking it costs, but is allowed (constrained optimisation)

Lesson: More CSPs: N queens and job scheduling

Table: · forward checking · arc consistency (ac-3)

	Forward checking	Arc consistency (AC-3)
What it checks	Only arcs from the variable just assigned to its unassigned neighbours	Every arc in the network, again and again until nothing changes
Strength	Weaker	Stronger: it also catches conflicts between two unassigned variables
Cost per step	Cheap: O(d) per neighbour	Heavier: up to $O(c\ d^3)$
Australia, after WA = red and Q = green	NT = {blue}, SA = {blue}, and no failure is seen yet	Checks NT against SA, finds both must be blue, empties a domain: failure detected at once

Lesson: Forward checking against arc consistency

Table: level · meaning

Level	Meaning
Node (1 consistency)	Each value meets its unary constraints
Arc (2 consistency)	Each value of a variable has a partner in every neighbour
Path (3 consistency)	Each consistent pair (x, y) can be extended to any third variable z
k consistency	Any consistent assignment to k − 1 variables extends to any k-th variable
Strong n consistency	k consistent for every $k \leq n$: then a solution can be found with no backtracking, but reaching it costs exponential time in general

Lesson: Forward checking against arc consistency

TIPS AND TRAPS

COMMON MISTAKE Arc consistency cannot see that 2 colours are not enough for a triangle WA, NT, SA: each arc on its own is fine ({red, blue} against {red, blue}). Path consistency catches it.

Lesson: Forward checking against arc consistency

5.2 Backtracking search for CSPs

Backtracking, MRV, degree, LCV, forward checking, MAC, backjumping and learning.

KEY IDEAS

- **Backtracking** DFS, one variable per level (assignments commute), undo on a broken constraint. d^n leaves.
- **MRV and degree** MRV: fewest legal values (fail first). Degree: most constraints on unassigned variables, breaks ties. SA first.
- **LCV** Value ruling out fewest neighbour values first. WA = red, NT = green: Q = red, not blue.
- **FC and MAC** FC prunes neighbours of the assigned variable. MAC runs AC-3 and catches more.
- **Backjumping and learning** Jump to the latest variable in the conflict set. Record no goods so the same failure never repeats.

FORMULAS

Worst case of backtracking Exam favourite

$$O(d^n)$$

Every partial assignment may be extended until the last variable. General CSPs are NP complete (3 colouring is), so no method avoids this in the worst case.

Lesson: The maths: branching and pruning

Forward checking cost per assignment

$$O(e\ d) \text{ at most, } e = \text{number of neighbours}$$

Each neighbour loses at most the conflicting values.

Lesson: The maths: branching and pruning

Symmetry

$$k \text{ colours} \Rightarrow k! \text{ equivalent solutions}$$

Fixing the first colour divides the search by k, fixing an ordering of all colours by k!.

Lesson: The maths: branching and pruning

TABLES

Table: heuristic · rule · why it helps · slide example

Heuristic	Rule	Why it helps	Slide example
Minimum remaining values (MRV), also "most constrained variable" or "fail first"	Choose the variable with the fewest legal values left	If some variable has no values left, MRV picks it and the failure is found at once, before wasting time on others	After WA and NT are coloured, SA has only one colour left, so colour SA next
Degree heuristic	Choose the variable involved in the most constraints on other unassigned variables	Lowers the branching factor of future choices. Used to break ties of MRV, and at the very start when all domains are equal	Choose SA first: it has degree 5
Least constraining value (LCV)	Try first the value that rules out the fewest choices for neighbouring variables	Leaves the most room for the rest, so a solution is found sooner	After WA = red and NT = green, Q = blue would remove SA's last value, so LCV prefers Q = red

Lesson: Which variable next, which value first

Table: method · idea

Method	Idea
Backjumping	Jump back to the most recent variable in the conflict set of the failed variable (the assigned variables that removed its values)
Conflict directed backjumping	When jumping back from X to Y, add X's conflict set (minus Y) to Y's, so the next jump from Y can also skip unrelated variables
Constraint learning (no goods)	Record the smallest set of assignments that caused the failure, for example {Q = red, NSW = green, V = blue}, and never try that combination again
Symmetry breaking	In map colouring, any solution with colours permuted is also a solution. Fix WA = red at the start to cut the work by 3! = 6

Lesson: Smarter backtracking: backjumping and learning

Table: method · order · inference · backtracks

Method	Order	Inference	Backtracks
Plain backtracking	Fixed WA, NT, NSW, Q, SA, V, T	None	2
With forward checking	Fixed	Forward checking	2, but each dead end is found one level earlier
MRV with degree and LCV	SA first	Forward checking	0
2 colours with MAC	SA first	MAC	Proves no solution after trying both colours for SA

Lesson: Comparing CSP search strategies

Table: · search over states (chapter 2) · csp backtracking

	Search over states (chapter 2)	CSP backtracking
State	Black box	Partial assignment with visible structure
Pruning	Only by the goal test or h	Constraints prune at every step, inference removes values ahead
Heuristics	Problem specific h	General ones that work for any CSP: MRV, degree, LCV

Lesson: Comparing CSP search strategies

TIPS AND TRAPS

IN THE EXAM Variable ordering is fail first: pick the hardest variable so a dead end shows up early. Value ordering is succeed first: pick the value most likely to work. When you want all solutions, or none exists, value ordering does not matter, because every value gets tried anyway.

Lesson: Which variable next, which value first

COMMON MISTAKE Forward checking misses conflicts between two unassigned variables. After WA = red and Q = green, NT and SA both have only blue left and are neighbours. Forward checking does not see it. MAC does, because it also checks the arc NT, SA.

Lesson: Forward checking and maintaining arc consistency

REMEMBER Forward checking already finds every conflict that simple backjumping would jump over, so backjumping adds little to forward checking. Conflict directed backjumping and learning do add power, and they are the core of modern SAT solvers.

Lesson: Smarter backtracking: backjumping and learning

5.3 Local search for CSPs and problem structure

Min-conflicts local search, tabu and weighting, tree CSPs, cutset conditioning and tree decomposition.

KEY IDEAS

- **Min-conflicts** Complete random assignment, move a random conflicted variable to its least conflicting value. About 50 steps for a million queens (AIMA).
- **Escaping traps** Sideways moves, tabu search, constraint weighting, restarts. Local search cannot prove no solution.
- **Tree CSP** $O(n\ d^2)$: directional arc consistency backward, then assign forward, no backtracking.
- **Cutset conditioning** $O(d^c(n - c)d^2)$. Australia: cutset {SA}.
- **Tree decomposition** $O(n\ d^{w+1})$, w = tree width.

FORMULAS

Independent sub problems

Exam favourite

connected components solved separately: $O(d^c \cdot n/c)$ instead of $O(d^n)$

Tasmania is not connected to the mainland, so colour it on its own. If n = 80 splits into 4 parts of 20 with d = 2: $4 \times 2^{20} \approx 4$ million against $2^{80} \approx 10^{24}$.

Lesson: Using the shape of the problem: independent parts and trees

Tree structured CSP

Exam favourite

solvable in $O(n\ d^2)$

If the constraint graph is a tree (no loops), there is no need to backtrack at all.

Lesson: Using the shape of the problem: independent parts and trees

Cutset conditioning

Exam favourite

$$O(d^c \cdot (n - c)\ d^2)$$

c is the size of the cutset. Australia: removing SA leaves a tree (a chain plus T), so c = 1.

Lesson: Cutset conditioning and tree decomposition

Tree decomposition

$$O(n\ d^{w+1})$$

w is the tree width: the size of the largest cluster minus 1. CSPs with small tree width are easy.

Lesson: Cutset conditioning and tree decomposition

Backtracking worst case

$$O(d^n)$$

For any CSP.

Lesson: The maths of problem structure

Tree CSP

$$O(n\ d^2)$$

Linear in n.

Lesson: The maths of problem structure

Cutset

$$O(d^c\ (n - c)\ d^2)$$

Exponential only in the cutset size c.

Lesson: The maths of problem structure

TABLES

Table: fix · idea

Fix	Idea
Sideways moves	Allow moves to a value with the same number of conflicts, so the search can cross a plateau
Tabu search	Keep a short list of recently visited states (or recently changed variables) and do not go back to them, so the search does not bounce between the same few states
Constraint weighting	Give every constraint a weight, starting at 1. After each step, add 1 to the weight of every broken constraint. Minimise the total weight. Hard constraints that keep breaking become expensive, which changes the landscape and pushes the search out of local minima
Random restarts and random walk	Start again, or sometimes make a random move instead of the best one
Online repair	When the problem changes (a flight is cancelled), start min-conflicts from the old solution: only a few variables need to change

Lesson: Plateaux, tabu search and constraint weighting

Table: · backtracking with propagation · min-conflicts (local search) · structure based (tree, cutset)

	Backtracking with propagation	Min-conflicts (local search)	Structure based (tree, cutset)
Starts from	An empty assignment	A complete random assignment	The shape of the constraint graph
Complete (finds a solution if one exists)	Yes	No	Yes
Can prove no solution	Yes	No	Yes
Memory	O(n) for the path	O(n), one assignment	O(n d) for the domains
Best on	Tight problems with few solutions	Large problems with many solutions spread out (N queens, scheduling)	Graphs that are trees or nearly trees
N queens with a million queens	Very slow	About 50 steps (AIMA)	Not a tree

Lesson: Backtracking against min-conflicts against structure

TIPS AND TRAPS

COMMON MISTAKE Local search cannot prove that a CSP has no solution. It just fails to find one. Backtracking with propagation can prove it, for example that 2 colours are not enough for Australia.

Lesson: Plateaux, tabu search and constraint weighting

STEP 6 Core AI · Adversarial search

6.1 Games and minimax

Game formalism, minimax values, the slide examples, the cost of minimax and evaluation functions.

KEY IDEAS

- **Game** S_0, PLAYER, ACTIONS, RESULT, TERMINAL-TEST, UTILITY. Zero sum: one player's gain is the other's loss.
- **Minimax** MAX takes max, MIN takes min, terminals give utility. Root value = what MAX can guarantee.
- **Slide examples** 27 leaves: B1 = 5, B2 = 2, B3 = 3, A = 5. Tic-tac-toe: O takes the centre.
- **Cost** Complete (finite), optimal against perfect play, $O(b^m)$ time, O(b m) space. Chess ≈ 10^{154}.
- **Cutoffs** Depth limit with EVAL = $\sum w_i f_i$. Quiescence search, horizon effect.

FORMULAS

Minimax value

Exam favourite

$$\text{MINIMAX}(s) = \begin{cases} \text{UTILITY}(s) & \text{if } s \text{ is terminal} \\ \max_a \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if MAX moves} \\ \min_a \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if MIN moves} \end{cases}$$

A recursive definition, computed depth first.

Lesson: The minimax algorithm

Depth limited minimax

Exam favourite

replace TERMINAL-TEST by CUTOFF-TEST(s, d) and UTILITY by EVAL(s)

Search to a fixed depth d, then use a heuristic evaluation of the position.

Lesson: Properties, and what to do when the tree is too big

Weighted linear evaluation

$$\text{EVAL}(s) = w_1 f_1(s) + w_2 f_2(s) + \ldots + w_n f_n(s)$$

Chess material: pawn 1, knight 3, bishop 3, rook 5, queen 9, counted for MAX minus for MIN.

Lesson: Properties, and what to do when the tree is too big

Zero sum

Exam favourite

$$\text{UTILITY}(s, \text{MAX}) + \text{UTILITY}(s, \text{MIN}) = \text{constant}$$

Chess: 1 + 0, 0 + 1 or ½ + ½. So MIN minimising MAX's utility is the same as maximising its own.

Lesson: The maths: zero sum games and evaluation

Multiplayer games

each node holds a vector $(u_1, \ldots, u_k)$, and the player to move picks the child with the best u_{own}

With three players, alliances can appear and break, which plain minimax cannot express.

Lesson: The maths: zero sum games and evaluation

Size of a game tree

$$b^m$$

Tic-tac-toe has at most 9! = 362,880 move sequences (fewer, because games end early), and only 5,478 legal positions.

Lesson: The maths: zero sum games and evaluation

TABLES

Table: part (slide 2) · meaning · tic-tac-toe

Part (slide 2)	Meaning	Tic-tac-toe
S_0	The initial state	The empty board
PLAYER(s)	Whose turn it is in state s	X and O take turns
ACTIONS(s)	The legal moves in s	The empty squares
RESULT(s, a)	The transition model: the state after move a	The board with the new mark
TERMINAL-TEST(s)	True when the game is over	Three in a row, or a full board
UTILITY(s, p)	The final payoff for player p in a terminal state s	+1 win, −1 loss, 0 draw

Lesson: Games as search problems

Table: · deterministic · chance

	Deterministic	Chance
Perfect information	Chess, Go, tic-tac-toe, checkers	Backgammon, Monopoly
Imperfect information	Battleship, Kriegspiel	Poker, bridge, Scrabble

Lesson: Games as search problems

Table: property · minimax

Property	Minimax
Complete	Yes, if the tree is finite
Optimal	Yes, against an optimal opponent. Against a weak one it is safe but may miss chances
Time	$O(b^m)$: it visits the whole tree to depth m
Space	O(b m) when it explores depth first and generates all moves at once, O(m) when it generates one move at a time
Data structures	A recursion stack (or explicit stack) of states, the list of moves at each level, and the best value and move so far

Lesson: Properties, and what to do when the tree is too big

Table: problem with cutting off · fix

Problem with cutting off	Fix
Non quiet positions: the evaluation changes wildly in the next move (a capture is about to happen)	Quiescence search: keep searching captures until the position is quiet
Horizon effect: a delaying move pushes an unavoidable loss beyond the search depth, so it looks avoided	Singular extensions: search a clearly best move one level deeper
Too many moves to search	Forward pruning: skip moves that look bad (beam search on moves). Risky but used in practice

Lesson: Properties, and what to do when the tree is too big

Table: · greedy (best immediate result) · minimax · expectimax (chapter 6.3)

	Greedy (best immediate result)	Minimax	Expectimax (chapter 6.3)
Assumes about the opponent	Nothing, ignores replies	Plays perfectly against you	Plays at random with known chances
Safe against a perfect opponent	No	Yes	No
Exploits a weak opponent	Sometimes	No, it is cautious	Yes, if the chances are right
Cost	O(b)	$O(b^m)$	$O(b^m)$
Slide tic-tac-toe	Can pick a losing corner	Picks the centre, wins	Picks the centre too

Lesson: Minimax against other ways to choose a move

TIPS AND TRAPS

REMEMBER Boards 8, 10, 13 and 14 show what happens if X does not take the centre: O wins. Minimax ignores those lines because a perfect X would not play them.

Lesson: The tic-tac-toe example of the slides

6.2 Alpha beta pruning

Alpha and beta, the prune rule, the slide example, move ordering and the cost of alpha beta.

KEY IDEAS

- **α and β** α = best MAX can guarantee so far (starts −∞). β = best MIN can guarantee so far (starts +∞).
- **Rule** α, β go parent to child. Values go child to parent. Prune when α ≥ β. Root value = minimax.
- **Slide example** Leaves 2 3 5 9 0 1 7 5. Prune the 9 under E and all of G. A = 3 after 5 of 8 leaves.
- **Cost** Best $O(b^{m/2})$, random about $O(b^{3m/4})$, worst $O(b^m)$. Space O(b m). Same time, twice the depth.
- **Ordering** Best first: $b^{\lceil d/2 \rceil} + b^{\lfloor d/2 \rfloor} - 1$ leaves. Iterative deepening, killer moves, transposition tables.

FORMULAS

Fewest leaves alpha beta can read (Knuth and Moore)

Exam favourite

$$b^{\lceil d/2 \rceil} + b^{\lfloor d/2 \rfloor} - 1$$

For a uniform tree with branching b and depth d, with perfect ordering. b = 2, d = 3: 4 + 2 − 1 = 5. b = 3, d = 3: 9 + 3 − 1 = 11.

Lesson: Move ordering decides how much is pruned

Best order cost

Exam favourite

$$O(b^{m/2}) = O((\sqrt{b})^m)$$

The effective branching factor drops from b to √b. Chess: √35 ≈ 5.9.

Lesson: The maths: how much faster, and how much deeper

Depth for the same work

$$b^d = (\sqrt{b})^{2d}$$

In the time minimax searches depth d, alpha beta with good ordering searches depth 2d.

Lesson: The maths: how much faster, and how much deeper

Random order cost

$$\text{about } O(b^{3m/4})$$

Still a big saving: the depth grows by about a third.

Lesson: The maths: how much faster, and how much deeper

Negamax form

$$\max(a, b) = -\min(-a, -b)$$

So one function can serve both players: value = max over children of −value(child), with the window (−β, −α).

Lesson: The maths: how much faster, and how much deeper

TABLES

Table: value (slide 7) · meaning · starts at · changed at

Value (slide 7)	Meaning	Starts at	Changed at
α	The highest value MAX can guarantee so far on the path to this node. A lower bound	−∞	MAX nodes: α = max(α, value)
β	The lowest value MIN can guarantee so far on the path to this node. An upper bound	+∞	MIN nodes: β = min(β, value)

Lesson: Why we can skip parts of the tree

Table: direction (slide 7) · what moves

Direction (slide 7)	What moves
Parent to child	The current α and β values
Child to parent	The node value, which then updates α (at a MAX parent) or β (at a MIN parent)

Lesson: Why we can skip parts of the tree

Table: property · alpha beta

Property	Alpha beta
Same answer as minimax	Yes, always, at the root. Pruning never changes the decision
Complete and optimal	Same as minimax: complete for finite trees, optimal against a perfect opponent
Time, best order	$O(b^{m/2})$
Time, random order	About $O(b^{3m/4})$ for moderate b
Time, worst order	$O(b^{m})$, no pruning at all
Space	O(b m), like depth first minimax. Only the current path and its α, β values are stored
Data structures	The recursion stack. Each frame holds a state, its list of moves, v, α and β. Real programs add a transposition table (a hash map of positions already searched)

Lesson: The algorithm

Table: tree · leaves · read by alpha beta · pruned

Tree	Leaves	Read by alpha beta	Pruned
Slide example, slide order (b = 2, depth 3)	8	5	3
Same values, worst order	8	8	0
27 leaf slide tree, slide order (b = 3, depth 3)	27	11	16
Same 27 values, worst order	27	27	0

Lesson: Move ordering decides how much is pruned

Table: how real programs get a good order · idea

How real programs get a good order	Idea
Iterative deepening	Search depth 1, 2, 3 and so on. The best move found at depth d − 1 is tried first at depth d
Killer moves	A move that caused a prune at the same depth elsewhere is tried early, since it is likely good again
Captures first	In chess, try captures and threats before quiet moves
Transposition table	Store each searched position with its value and best move, so a position reached by a different move order is not searched twice

Lesson: Move ordering decides how much is pruned

Table: question (slide 10) · answer

Question (slide 10)	Answer
Time and space complexity	Time $O(b^{m/2})$ with best ordering, about $O(b^{3m/4})$ with random ordering, $O(b^{m})$ in the worst case. Space O(b m)
Data structures	A recursion stack with α and β per frame, a move list per level, and in practice a hash table of positions (transposition table) and a small table of killer moves per depth
Real life uses	Chess, checkers, Othello and Connect Four engines. Adversarial planning such as security patrols, where the attacker answers the defender. Puzzle game AI in mobile apps
Utility functions	Chess: material, king safety, mobility, pawn structure, weighted and added. Connect Four: number of open lines of 2 and 3. Security: value of each target minus the cost of guarding it

Lesson: Food for thought from the slides

Table: · minimax · alpha beta · depth limited alpha beta

	Minimax	Alpha beta	Depth limited alpha beta	Monte Carlo tree search (6.3)
Same answer as full minimax	Yes	Yes	Only as good as EVAL	Approaches it with many rollouts
Leaves read	All b^m	$b^{m/2}$ to b^m, depends on order	$b^{d/2}$ to b^d for depth d	Chosen by statistics, not by depth
Needs an evaluation function	No (full tree)	No (full tree)	Yes	No, uses random playouts
Good for	Tiny games	Small to medium games	Chess, checkers	Go, games with b in the hundreds
Handles chance	No	No	No	Yes, naturally

Lesson: Alpha beta against other game search methods

Table: improvement · what it adds

Improvement	What it adds
Aspiration windows	Start with a narrow window [v − δ, v + δ] around the last value instead of [−∞, +∞]. More prunes, research if the result falls outside
Principal variation search (NegaScout)	Search the first move with the full window, the rest with a zero width window to prove they are worse. Research only when the proof fails
Null move pruning	Let the opponent move twice. If MAX is still above β, the real position is surely good enough, so prune
Transposition tables	Reuse values of positions already seen through another move order

Lesson: Alpha beta against other game search methods

TIPS AND TRAPS

COMMON MISTAKE The slide text says "the initial value of alpha is +∞" in the line about beta. That is a typing slip: it is β that starts at +∞, and α starts at −∞.

Lesson: Why we can skip parts of the tree

IN THE EXAM A pruned node gets a bound, not an exact value: E is "at least 5" and C is "at most 1". These bounds are enough to choose the move, which is all we need at the root.

Lesson: The worked example of the slides

REMEMBER In class we noted that when the order of nodes is poor, or when you add your own thresholds to prune more, you may cut away a good move. Plain alpha beta never does that: its pruning is exact. Only extra rules such as forward pruning or chance thresholds can lose the best move.

Lesson: Food for thought from the slides

6.3 Games with chance: expectimax and Monte Carlo tree search

Expectimax, expectiminimax, their problems, and Monte Carlo tree search with UCB1.

KEY IDEAS

- **Chance node** V(s) = ∑ p(s' | s) V(s'). exp-value starts at v = 0 and adds p × value.
- **Slide 11** 3 12 9, 2 4 6, 15 6 0 with 1/3 each: 8, 4, 7. MAX = 8.
- **Slide 12** MIN 2, 1, 3 with 0.5, 0.4, 0.1: 1.7. Layers MAX, CHANCE, MIN, CHANCE.
- **Problems** Need probabilities, cost $O(b^m n^m)$, value scale matters (only a x + b with a > 0 is safe), pruning needs bounds.
- **MCTS** Select by UCB1 = w / n + C √(ln N / n), expand, simulate, back up. Play the most visited move.

FORMULAS

Expectimax value (slide 11)

Exam favourite

$$V(s) = \max_{s' \in \text{successors}(s)} V(s') \text{ for agent controlled states. } V(s) = \sum_{s' \in \text{successors}(s)} p(s' \mid s) V(s') \text{ for chance states. } V(s) = \text{known for terminal states}$$

The same recursion as minimax, with the MIN rule replaced by an expectation.

Lesson: When dice decide part of the game

Expectiminimax

Exam favourite

$$EMM(s) = \text{UTILITY}(s) \text{ if terminal, } \max_a EMM(\text{RESULT}(s, a)) \text{ if MAX, } \min_a EMM(\text{RESULT}(s, a)) \text{ if MIN, } \sum_r P(r) EMM(\text{RESULT}(s, r)) \text{ if CHANCE}$$

r is a chance outcome, such as a dice roll.

Lesson: Expectiminimax: chance and an opponent

UCB1 (upper confidence bound)

Exam favourite

$$UCB1(i) = w_i / n_i + C \sqrt{(\ln N / n_i)}$$

w_i wins in n_i visits of move i, N visits of the parent, C balances the two parts. The first part is exploitation (how good the move looks), the second is exploration (how unsure we still are). $C = \sqrt{2} \approx 1.4$ is the usual start.

Lesson: Monte Carlo tree search

Expected value

Exam favourite

$$E[X] = \sum_x P(x) \times x$$

The average result over many plays. The probabilities of the outcomes of one chance node add up to 1.

Lesson: The maths: expected value, UCB1 and cost

Linearity

$$E[a\,X + b] = a\,E[X] + b \text{ for } a > 0$$

This is why a positive linear change of all values keeps the expectimax choice, while other changes may not.

Lesson: The maths: expected value, UCB1 and cost

Cost of expectiminimax

$$O(b^m\, n^m)$$

b moves and n chance outcomes per level, m levels of each.

Lesson: The maths: expected value, UCB1 and cost

Where UCB1 comes from

$$P(\text{true mean} > \text{sample mean} + \varepsilon) \le e^{-2\,n\,\varepsilon^2} \text{ (Hoeffding)}$$

Choose ε so the chance of being wrong falls like $1/N^4$: $\varepsilon = \sqrt{(2 \ln N / n)}$. That is UCB1 with $C = \sqrt{2}$.

Lesson: The maths: expected value, UCB1 and cost

TABLES

Table: problem · why it hurts · solution strategies

Problem	Why it hurts	Solution strategies
We must know the probabilities	In class: a coin tossed 1,000 times is never exactly 50:50, a coin may be biased, and probabilities from experience are skewed towards what we have seen	Estimate from data, update the estimates as you play, or test how much the decision changes when the probabilities change a little
The tree is much bigger	Time $O(b^m n^m)$, where n is the number of chance outcomes. Backgammon has 21 distinct rolls of two dice and about 20 moves per roll	Depth limit with an evaluation function, or sample a few rolls instead of all (Monte Carlo)
The size of the values matters	Minimax only cares about the order of the leaves. Expectimax averages, so stretching some values can change the choice (see below)	The evaluation must be a positive linear function of the true expected payoff, for example the chance of winning
Pruning is harder	One unseen child of a chance node can change the average by any amount	If the values are bounded, say between 0 and 10, a chance node gets an upper bound after a few children, and alpha beta style pruning works again
The model of the opponent matters	Expectimax against a clever opponent can walk into a trap. Minimax against a random opponent is too careful	Pick the model that fits: MIN nodes for an adversary, chance nodes for random behaviour

Lesson: What problems do you see here?

Table: · minimax and alpha beta · expectimax · expectiminimax

	Minimax and alpha beta	Expectimax	Expectiminimax	MCTS
Other side modelled as	A perfect opponent	Random events with known chances	An opponent plus random events	Whatever the rollout policy does
Value at a non MAX node	min	Weighted average	min or weighted average	Win rate from rollouts
Needs an evaluation function	Yes, at the depth limit	Yes, on the right scale	Yes, on the right scale	No
Pruning	Alpha beta, exact	Only with bounded values	Only with bounded values	Not needed: it grows only good parts
Typical games	Chess, checkers, tic-tac-toe	Single player games with randomness, Pac-Man ghosts that move at random	Backgammon	Go, many board and card games, planning

Lesson: Which game search to use

TIPS AND TRAPS

REMEMBER

If the same tree had MIN nodes instead of chance nodes, minimax would give min values 3, 2 and 0, and MAX would still pick the first move, but with value 3. The third move looks worst to minimax (0) and second best to expectimax (7). The two methods can choose different moves when the risk differs.

Lesson: The expectimax example of the slides

REMEMBER

MCTS also handles chance naturally: a dice roll in a rollout is just one more random step. That is one reason it replaced expectimax in many games with dice and cards.

Lesson: Monte Carlo tree search

7.1 Knowledge, logic and propositional logic

Knowledge based agents, entailment, propositional logic and model checking.

KEY IDEAS

- **KB agent** TELL percepts, ASK for an action, TELL the action. Inference happens inside ASK.
- **Entailment** KB ⊨ α iff M(KB) ⊆ M(α). Sound: derives only entailed. Complete: derives all entailed.
- **Connectives** ¬ ∧ ∨ ⇒ ⇔, in that order of precedence. P ⇒ Q false only for P = T, Q = F.
- **Wumpus KB** R1 ¬P11, R2 B11 ⇔ (P12 ∨ P21), R3 B21 ⇔ (P11 ∨ P22 ∨ P31), R4 ¬B11, R5 B21. 3 of 128 models. Entails ¬P12.
- **Validity** KB ⊨ α iff KB ⇒ α is valid iff KB ∧ ¬α is unsatisfiable. SAT is NP-complete.

FORMULAS

Entailment Exam favourite

$$KB \models \alpha \text{ if and only if } M(KB) \subseteq M(\alpha)$$

The more a KB says, the fewer models it has. If all of them make α true, α follows.

Lesson: Logic: syntax, semantics, models and entailment

Deduction theorem Exam favourite

$$KB \models \alpha \text{ if and only if } (KB \Rightarrow \alpha) \text{ is valid}$$

Entailment can be checked as validity.

Lesson: Validity, satisfiability and equivalence

Proof by contradiction (refutation) Exam favourite

$$KB \models \alpha \text{ if and only if } (KB \wedge \neg\alpha) \text{ is unsatisfiable}$$

Assume α is false and show this is impossible. Resolution (chapter 7.2) works this way.

Lesson: Validity, satisfiability and equivalence

Number of models Exam favourite

$$2^n \text{ for n symbols}$$

3 symbols: 8 models. 7 symbols: 128. 30 symbols: over 10^9.

Lesson: The maths: counting models

Number of different sentences up to equivalence

$$2^{2^n}$$

Each sentence is fixed by the set of models where it is true, and there are 2^{2^n} such sets. For n = 2 there are 16 different truth tables.

Lesson: The maths: counting models

Entailment as sets

$$KB \models \alpha \iff M(KB) \subseteq M(\alpha) \iff M(KB) \cap M(\neg\alpha) = \varnothing$$

The last form is proof by contradiction.

Lesson: The maths: counting models

TABLES

Table: peas part · wumpus world

PEAS part	Wumpus world
Performance	+1000 for climbing out with the gold, −1000 for falling into a pit or being eaten, −1 for each action, −10 for using the arrow
Environment	4 × 4 grid of rooms, agent starts in [1,1]. The wumpus and the gold are placed at random in other rooms, and each other room has a pit with probability 0.2
Actuators	Forward, TurnLeft, TurnRight, Grab, Shoot, Climb
Sensors	Stench (wumpus next door), Breeze (pit next door), Glitter (gold here), Bump (walked into a wall), Scream (the wumpus died)

Lesson: The wumpus world

Table: idea · meaning · example

Idea	Meaning	Example
Syntax	Which strings are well formed sentences	x + y = 4 is a sentence, x4y+ = is not
Semantics	When a sentence is true. A sentence is true or false in each possible world	x + y = 4 is true in the world where x = 2, y = 2
Model	A possible world, fixed exactly. In propositional logic: a true or false value for every symbol	P = T, Q = F, R = T
M(α)	The set of all models in which α is true	M(P ∨ Q) has 3 of the 4 models of P, Q
Entailment KB ⊨ α	α is true in every model where the KB is true	{x = 0} ⊨ x y = 0

Lesson: Logic: syntax, semantics, models and entailment

Table: property of an inference procedure i · meaning

Property of an inference procedure i	Meaning
KB $\vdash_i$ α	Procedure i derives α from the KB
Sound (truth preserving)	It derives only entailed sentences: if KB $\vdash_i$ α then KB ⊨ α
Complete	It can derive every entailed sentence: if KB ⊨ α then KB $\vdash_i$ α
Grounding	The link between the sentences and the real world. It comes from the sensors and from learning, not from logic

Lesson: Logic: syntax, semantics, models and entailment

Table: connective · name · read as · true when

Connective	Name	Read as	True when
¬P	negation	not P	P is false
P ∧ Q	conjunction	P and Q	both are true
P ∨ Q	disjunction	P or Q	at least one is true
P ⇒ Q	implication	if P then Q	P is false, or Q is true. False only for P = T, Q = F
P ⇔ Q	biconditional	P if and only if Q	both have the same value

Lesson: Propositional logic

Table: name · sentence · meaning

Name	Sentence	Meaning
R1	$\neg P_{1,1}$	no pit in the start room
R2	$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$	a room is breezy if and only if a neighbour has a pit
R3	$B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$	the same rule for [2,1]
R4	$\neg B_{1,1}$	percept: no breeze in [1,1]
R5	$B_{2,1}$	percept: breeze in [2,1]

Lesson: A knowledge base for the wumpus world, and model checking

Table: term · meaning · example

Term	Meaning	Example
Valid (a tautology)	True in every model	$P \vee \neg P$
Satisfiable	True in at least one model	$P \wedge Q$
Unsatisfiable	True in no model	$P \wedge \neg P$
Equivalent, $\alpha \equiv \beta$	True in exactly the same models. Same as $\alpha \models \beta$ and $\beta \models \alpha$	$P \Rightarrow Q \equiv \neg P \vee Q$

Lesson: Validity, satisfiability and equivalence

Table: equivalence · name

Equivalence	Name
$\alpha \wedge \beta \equiv \beta \wedge \alpha$, $\alpha \vee \beta \equiv \beta \vee \alpha$	commutativity
$(\alpha \wedge \beta) \wedge \gamma \equiv \alpha \wedge (\beta \wedge \gamma)$, and the same for $\vee$	associativity
$\neg(\neg\alpha) \equiv \alpha$	double negation
$\alpha \Rightarrow \beta \equiv \neg\beta \Rightarrow \neg\alpha$	contraposition
$\alpha \Rightarrow \beta \equiv \neg\alpha \vee \beta$	implication elimination
$\alpha \Leftrightarrow \beta \equiv (\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$	biconditional elimination
$\neg(\alpha \wedge \beta) \equiv \neg\alpha \vee \neg\beta$, $\neg(\alpha \vee \beta) \equiv \neg\alpha \wedge \neg\beta$	De Morgan
$\alpha \wedge (\beta \vee \gamma) \equiv (\alpha \wedge \beta) \vee (\alpha \wedge \gamma)$, $\alpha \vee (\beta \wedge \gamma) \equiv (\alpha \vee \beta) \wedge (\alpha \vee \gamma)$	distributivity

Lesson: Validity, satisfiability and equivalence

Table: · atomic states (search) · propositional logic · first order logic (7.3)

	Atomic states (search)	Propositional logic	First order logic (7.3)	Probability (step 9)
A state is	one name, no inside parts	true or false values for fixed facts	objects, relations and functions	a distribution over possible worlds
Can say "pits cause breezes in every room" in one sentence	No	No, one sentence per room	Yes	Yes, with a rule per variable
Handles uncertainty	No	Only by listing cases with $\vee$	Only by listing cases	Yes, with numbers
Inference	Search	Model checking, resolution, chaining	Unification, chaining, resolution	Bayes rule, Bayesian networks
Decidable	Yes	Yes (but NP-complete)	Only semidecidable	Yes for finite models

Lesson: Logic against other ways to store knowledge

TIPS AND TRAPS

REMEMBER Room names are [column, row]. [2,1] is one step right of the start, [1,2] is one step up.

Lesson: The wumpus world

COMMON MISTAKE $P \Rightarrow Q$ is true whenever P is false. "If 5 is even, then Sam is smart" is true in propositional logic, whatever Sam is like. Implication does not mean that P causes Q.

Lesson: Propositional logic

7.2 Reasoning patterns: rules, resolution and chaining

Inference rules, resolution, CNF, forward and backward chaining, DPLL and WalkSAT.

KEY IDEAS

- **Rules** Modus Ponens: $\alpha \Rightarrow \beta, \alpha \vdash \beta$. And-Elimination. Equivalences as rules. Monotonic.

- **Resolution** $\ell \vee \alpha, \neg\ell \vee \beta \vdash \alpha \vee \beta$. One pair at a time. Refutation complete.
- **CNF steps** Remove $\Leftrightarrow$, remove $\Rightarrow$, move $\neg$ in (De Morgan, double negation), distribute $\vee$ over $\wedge$.
- **Chaining** Definite clause: exactly one positive literal. Forward from facts, backward from goal. Linear time.
- **SAT** DPLL: early termination, pure symbol, unit clause. WalkSAT: local search, incomplete. CDCL in practice.

FORMULAS

The resolution rule

Exam favourite

from $\ell_1 \vee \ldots \vee \ell_k$ and $m_1 \vee \ldots \vee m_n$, where ℓ_i and m_j are opposite literals, conclude the clause of all the other literals

Example: from $P \vee Q$ and $\neg Q \vee R$, conclude $P \vee R$. Copies of the same literal are kept once (factoring).

Lesson: Resolution and conjunctive normal form

Why it is sound

if ℓ_i is true then m_j is false, so the rest of the second clause must be true. If ℓ_i is false, the rest of the first clause must be true

Either way, the joined rest is true.

Lesson: Resolution and conjunctive normal form

Possible clauses over n symbols

Exam favourite

$$3^n$$

Each symbol is positive, negative or absent in a clause. So resolution always stops: there are at most 3^n distinct clauses to add.

Lesson: The maths: how many clauses, and how fast

Clauses from distributing

$(a \wedge b) \vee (c \wedge d)$ gives $(a \vee c) \wedge (a \vee d) \wedge (b \vee c) \wedge (b \vee d)$

An OR of k ANDs with 2 parts each gives 2^k clauses: CNF can blow up. The Tseitin trick adds new symbols to keep it linear.

Lesson: The maths: how many clauses, and how fast

Forward chaining cost

$O(n)$ for a KB of total size n

Each premise of each rule is counted down at most once.

Lesson: The maths: how many clauses, and how fast

Random 3-SAT hardness

$$m/n \approx 4.26$$

With m clauses over n symbols, problems are hardest near this ratio. Below it almost all are satisfiable, above it almost none.

Lesson: The maths: how many clauses, and how fast

TABLES

Table: rule · from · conclude

Rule	From	Conclude
Modus Ponens	$\alpha \Rightarrow \beta$ and α	β
And-Elimination	$\alpha \wedge \beta$	α (and β)
Biconditional elimination	$\alpha \Leftrightarrow \beta$	$(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$
Contraposition	$\alpha \Rightarrow \beta$	$\neg\beta \Rightarrow \neg\alpha$
De Morgan	$\neg(\alpha \vee \beta)$	$\neg\alpha \wedge \neg\beta$
Unit resolution	$\alpha \vee \beta$ and $\neg\beta$	α

Lesson: Inference rules and proofs

Table: clause type · at most / exactly · example · as an implication

Clause type	At most / exactly	Example	As an implication
Definite clause	exactly one positive literal	¬L ∨ ¬M ∨ P	L ∧ M ⇒ P
Fact	one positive literal and nothing else	A	A
Horn clause	at most one positive literal	¬A ∨ ¬B	A ∧ B ⇒ False (a goal clause)

Lesson: Horn clauses, forward and backward chaining

Table: · forward chaining · backward chaining

	Forward chaining	Backward chaining
Driven by	Data (the facts)	The goal
Good for	Watching a stream of new facts, such as sensors	Answering one question, such as "what should I do now?"
Wasted work	May derive many facts unrelated to the query	Only touches relevant rules
Cost	O(size of KB), each rule counted down at most once per premise	Often much less than the KB size
Sound and complete for definite clauses	Yes	Yes, with the loop check

Lesson: Horn clauses, forward and backward chaining

Table: method · complete · typical use

Method	Complete	Typical use
Truth table (TT-ENTAILS)	Yes	Teaching, tiny problems
Resolution	Yes (refutation complete)	Theorem provers, the basis of first order provers
Forward or backward chaining	Yes, for definite clauses only	Rule based systems, Prolog
DPLL and its modern form CDCL	Yes	Industrial SAT solving with millions of clauses
WalkSAT	No: it can find a model, but it cannot prove there is none	Large satisfiable problems

Lesson: Fast satisfiability: DPLL and WalkSAT

Table: · model checking · inference rules by hand · resolution

	Model checking	Inference rules by hand	Resolution	Chaining	DPLL and WalkSAT
Works on	Any sentences	Any sentences	CNF	Definite clauses only	CNF
Sound and complete	Yes	Depends on the rules chosen	Refutation complete	Yes, for definite clauses	DPLL yes, WalkSAT not for proving
Time	2^n always	Search over proofs	Can be exponential	Linear	Exponential worst case, fast in practice
Easy for people to follow	Yes, but long	Yes	Fairly	Yes	No

Lesson: Which inference method to use

TIPS AND TRAPS

REMEMBER

Logic is **monotonic**: adding sentences to a KB never removes something it already entails. If KB ⊨ α then KB ∧ β ⊨ α. So a rule can be used whenever its premises are in the KB, whatever else is there.

Lesson: Inference rules and proofs

IN THE EXAM

The ground resolution theorem: if a set of clauses is unsatisfiable, resolution will reach the empty clause. So resolution is a complete refutation procedure for propositional logic. It cannot list every entailed sentence directly, but it can always confirm or refute a given one.

Lesson: Resolution and conjunctive normal form

7.3 First order logic: syntax and semantics

Objects, relations, functions, quantifiers, models and how to write knowledge in FOL.

KEY IDEAS

- **Syntax** Constants, predicates, functions, variables. Terms name objects, atoms are true or false.
- **Semantics** Model = domain + interpretation. Infinitely many models, so no truth tables.
- **Quantifier traps** ∀ with ⇒, ∃ with ∧. ∀x ∃y ≠ ∃y ∀x. ¬∀x P ≡ ∃x ¬P.
- **Equality** x = y means same object. Needed for "at least two". Database semantics: unique names, closed world, domain closure.
- **Engineering** Questions, knowledge, vocabulary, general axioms, instance facts, queries, debug.

FORMULAS

De Morgan for quantifiers Exam favourite

$$\forall x\ \neg P \equiv \neg\exists x\ P,\ \neg\forall x\ P \equiv \exists x\ \neg P,\ \forall x\ P \equiv \neg\exists x\ \neg P,\ \exists x\ P \equiv \neg\forall x\ \neg P$$

"Nobody likes parsnips" = "everybody dislikes parsnips".

Lesson: Quantifiers and their traps

∀ as a big AND, ∃ as a big OR

$$\forall x\ P(x) \equiv P(o_1) \wedge P(o_2) \wedge \ldots,\ \exists x\ P(x) \equiv P(o_1) \vee P(o_2) \vee \ldots$$

Over a finite domain with objects o_1, o_2, ...

Lesson: Quantifiers and their traps

Interpretations over a fixed domain of d objects Exam favourite

$$d^c \times (2^d)^u \times (2^{d^2})^b$$

c constants (each names one of d objects), u unary predicates (each is any subset of the d objects), b binary predicates (any subset of the d^2 pairs). Functions add more.

Lesson: The maths: counting interpretations

A k-ary relation

$$2^{d^k} \text{ possible extensions}$$

There are d^k tuples, each in or out.

Lesson: The maths: counting interpretations

Grounding ∀

$$\forall x\ P(x) \text{ becomes } d \text{ ground sentences}$$

Over a finite domain, FOL can be turned into propositional logic. Functions make the set of ground terms infinite.

Lesson: The maths: counting interpretations

TABLES

Table: language (aima figure 8.1) · what exists in the world (ontological commitment) · what an agent believes about facts (epistemological commitment)

Language (AIMA figure 8.1)	What exists in the world (ontological commitment)	What an agent believes about facts (epistemological commitment)
Propositional logic	facts	true, false, unknown
First order logic	facts, objects, relations	true, false, unknown
Temporal logic	facts, objects, relations, times	true, false, unknown
Probability theory	facts	a degree of belief between 0 and 1
Fuzzy logic	facts with a degree of truth between 0 and 1	a known interval value

Lesson: Why we need more than propositional logic

Table: element · what it names · examples

Element	What it names	Examples
Constant symbol	one object	Richard, John, M1, 2
Predicate symbol	a relation (true or false for its arguments)	Brother(·, ·), King(·), Adjacent(·, ·)
Function symbol	a function: objects in, one object out	LeftLeg(·), Mother(·), +
Variable	any object, to be fixed by a quantifier	x, y, s
Term	a constant, a variable, or a function applied to terms	John, x, LeftLeg(John), Mother(Mother(x))
Atomic sentence	a predicate applied to terms, or term = term	Brother(Richard, John), Father(x) = John
Complex sentence	atoms joined by ¬ ∧ ∨ ⇒ ⇔	King(Richard) ∨ King(John)
Quantified sentence	∀x S or ∃x S	∀x King(x) ⇒ Person(x)

Lesson: Syntax: terms, atoms and sentences

Table: sentence · read as · true when

Sentence	Read as	True when
∀x King(x) ⇒ Person(x)	all kings are persons	for every object, if it is a king then it is a person
∃x Crown(x) ∧ OnHead(x, John)	some crown is on John's head	at least one object is a crown and on John's head
∀x ∃y Loves(x, y)	everybody loves somebody	for each x there is a y (maybe different for each x)
∃y ∀x Loves(x, y)	there is someone whom everybody loves	one y works for all x

Lesson: Quantifiers and their traps

Table: rule of thumb · why

Rule of thumb	Why
∀ goes with ⇒	∀x King(x) ∧ Person(x) says everything is a king and a person, which is far too strong
∃ goes with ∧	∃x Crown(x) ⇒ OnHead(x, John) is true as soon as one object is not a crown, so it says almost nothing
Order matters for mixed quantifiers	∀x ∃y and ∃y ∀x mean different things
The same kind can be swapped	∀x ∀y ≡ ∀y ∀x, ∃x ∃y ≡ ∃y ∃x

Lesson: Quantifiers and their traps

Table: kinship axiom · meaning

Kinship axiom	Meaning
∀m, c Mother(c) = m ⇔ Female(m) ∧ Parent(m, c)	a mother is a female parent
∀w, h Husband(h, w) ⇔ Male(h) ∧ Spouse(h, w)	a husband is a male spouse
∀p, c Parent(p, c) ⇔ Child(c, p)	parent and child are inverse relations
∀g, c Grandparent(g, c) ⇔ ∃p Parent(g, p) ∧ Parent(p, c)	a grandparent is a parent of a parent
∀x, y Sibling(x, y) ⇔ x ≠ y ∧ ∃p Parent(p, x) ∧ Parent(p, y)	siblings share a parent and are different people

Lesson: Using first order logic: kinship and the wumpus world

Table: wumpus world in fol · meaning

Wumpus world in FOL	Meaning
∀x, y, a, b Adjacent([x, y], [a, b]) ⇔ (x = a ∧ (y = b − 1 ∨ y = b + 1)) ∨ (y = b ∧ (x = a − 1 ∨ x = a + 1))	which rooms are neighbours, for any cave size
∀s Breezy(s) ⇔ ∃r Adjacent(r, s) ∧ Pit(r)	one sentence for every room
∀t, s At(Agent, s, t) ∧ Breeze(t) ⇒ Breezy(s)	what a percept at time t tells about room s

Lesson: Using first order logic: kinship and the wumpus world

Table: · propositional · first order · higher order

	Propositional	First order	Higher order	Description logics
Talks about	fixed facts	objects, relations, functions	also relations of relations	classes and properties
Can quantify over	nothing	objects	relations and functions too	objects, in limited patterns
Entailment decidable	Yes	Semidecidable: a yes answer is found if true, but a search may never end if false	No complete proof method exists	Yes, for most variants
Compactness	Low: one sentence per case	High	Highest	High for taxonomies
Used in	SAT, hardware checking	Theorem provers, Prolog, databases	Proof assistants such as Coq and Lean	Web ontologies (OWL), medical terms (SNOMED)

Lesson: First order logic against its neighbours

TIPS AND TRAPS

REMEMBER In this course and in the animations, names that start with a lower case letter are variables, and names that start with a capital letter are constants, predicates or functions. Prolog uses the opposite convention.

Lesson: Syntax: terms, atoms and sentences

COMMON MISTAKE First order logic has infinitely many models: any domain size from 1 upwards, and every interpretation of every symbol. So entailment cannot be checked by listing models, as we did with truth tables.

Lesson: Semantics: models and interpretations

7.4 Inference in first order logic: unification, chaining and resolution

Instantiation, unification, GMP, forward and backward chaining, and resolution in FOL.

KEY IDEAS

- **UI and EI** UI: any ground term, many times. EI: new Skolem constant, once. Propositionalisation: k^v copies. Semidecidable.
- **Unify** {x/Jane}, {y/John, x/Bill}, {y/John, x/Mother(John)}, fail for Knows(x, Elizabeth). Occurs check. Standardise apart. MGU.
- **GMP and FC** Generalised Modus Ponens with θ. Crime: round 1 Weapon, Sells, Hostile, round 2 Criminal(West). Datalog stops.
- **BC** Goal driven, depth first, left to right (Prolog). Left recursion loops. Negation as failure.
- **FOL resolution** CNF: remove ⇒, ¬ in, standardise, Skolemise (∃y inside ∀x gives F(x)), drop ∀, distribute. Refutation complete.

FORMULAS

Generalised Modus Ponens (GMP)

Exam favourite

$$\text{from } p_1', \ldots, p_n' \text{ and } (p_1 \wedge \ldots \wedge p_n \Rightarrow q), \text{ infer } \text{SUBST}(\theta, q), \text{ where } \text{SUBST}(\theta, p_i') = \text{SUBST}(\theta, p_i) \text{ for every } i$$

King(John), Greedy(y) and King(x) ∧ Greedy(x) ⇒ Evil(x) with θ = {x/John, y/John} give Evil(John).

Lesson: Unification and lifting

UNIFY

$$\text{UNIFY}(p, q) = \theta \text{ where } \text{SUBST}(\theta, p) = \text{SUBST}(\theta, q)$$

θ is written as a set of bindings variable/term.

Lesson: Unification and lifting

Binary resolution

Exam favourite

$$\text{from } \ell_1 \vee \ldots \vee \ell_k \text{ and } m_1 \vee \ldots \vee m_n \text{ with } \text{UNIFY}(\ell_i, \neg m_j) = \theta, \text{ infer } \text{SUBST}(\theta, \text{all other literals})$$

Example: [Animal(F(x)) ∨ Loves(G(x), x)] and [¬Loves(u, v) ∨ ¬Kills(u, v)] with θ = {u/G(x), v/x} give Animal(F(x)) ∨ ¬Kills(G(x), x).

Lesson: Resolution in first order logic

Ground instances of one rule

Exam favourite

$$k^v$$

k constants, v variables. The crime law has v = 3 and the KB has k = 4 constants (West, M1, Nono, America): 64 copies, of which forward chaining needs 1.

Lesson: The maths: instances, ground terms and proof size

Ground terms with one unary function F and one constant A, up to depth d

$$d + 1$$

A, F(A), F(F(A)), and so on.

Lesson: The maths: instances, ground terms and proof size

Ground terms with c constants and one binary function, depth at most 1

$$c + c^2$$

Each argument is a constant. Depth 2 grows much faster.

Lesson: The maths: instances, ground terms and proof size

TABLES

Table: rule · what it does · example

Rule	What it does	Example
Universal Instantiation (UI)	From ∀v α, infer α with v replaced by any ground term g	From ∀x King(x) ∧ Greedy(x) ⇒ Evil(x), infer King(John) ∧ Greedy(John) ⇒ Evil(John)
Existential Instantiation (EI)	From ∃v α, infer α with v replaced by a brand new constant k (a Skolem constant)	From ∃x Crown(x) ∧ OnHead(x, John), infer Crown(C1) ∧ OnHead(C1, John)

Lesson: From first order inference to propositional inference

Table: p · q · θ = unify(p, q)

p	q	θ = UNIFY(p, q)
Knows(John, x)	Knows(John, Jane)	{x/Jane}
Knows(John, x)	Knows(y, Bill)	{x/Bill, y/John}
Knows(John, x)	Knows(y, Mother(y))	{y/John, x/Mother(John)}
Knows(John, x)	Knows(x, Elizabeth)	fail: x cannot be John and Elizabeth
Knows(John, x)	Knows(x17, Elizabeth)	{x17/John, x/Elizabeth}, after standardising apart

Lesson: Unification and lifting

Table: sentence in definite clauses · meaning

Sentence in definite clauses	Meaning
American(x) ∧ Weapon(y) ∧ Sells(x, y, z) ∧ Hostile(z) ⇒ Criminal(x)	the law
Owns(Nono, M1), Missile(M1)	Nono has a missile, M1 (EI on "Nono has some missiles")
Missile(x) ∧ Owns(Nono, x) ⇒ Sells(West, x, Nono)	all of Nono's missiles were sold by West
Missile(x) ⇒ Weapon(x)	missiles are weapons
Enemy(x, America) ⇒ Hostile(x)	an enemy of America counts as hostile
American(West), Enemy(Nono, America)	facts

Lesson: Forward chaining: the crime example

Table: fact about first order forward chaining · detail

Fact about first order forward chaining	Detail
Sound and complete for first order definite clauses	Every entailed atom is eventually derived
Datalog	Definite clauses with no function symbols. Forward chaining then stops after polynomially many steps
Pattern matching is hard	Finding all θ that match a rule's premises to the facts is like solving a CSP, and is NP-hard in general. Order the premises, most constrained first
Incremental forward chaining	In round t, only try rules with a premise that matches a fact new in round t – 1
Rete	Store partial matches in a network so new facts update them cheaply (used in rule engines)

Lesson: Forward chaining: the crime example

Table: prolog feature · what it means

Prolog feature	What it means
Syntax	criminal(X) :- american(X), weapon(Y), sells(X, Y, Z), hostile(Z). Upper case letters are variables
Search	Depth first, rules in the order written, premises left to right
Negation as failure	not P (written with a special operator in Prolog) succeeds if P cannot be proved. This is the closed world assumption, not real negation
No occurs check	Faster unification, but unsound in rare cases
Incomplete	A left recursive rule such as path(X, Z) :- path(X, Y), link(Y, Z) can loop for ever. Writing link first fixes it
Tabling	Remembering solved subgoals (memoisation) avoids repeated work and many loops

Lesson: Backward chaining and Prolog

Table: fact about first order resolution · detail

Fact about first order resolution	Detail
Refutation complete (Robinson, 1965)	If KB ⊨ α, resolution with factoring derives { } from KB ∧ ¬α
Factoring	Merge two literals of one clause that unify, for example P(x) ∨ P(A) becomes P(A)
Equality	Needs extra rules: demodulation (rewrite with x = y) and paramodulation
Strategies	Unit preference (resolve with short clauses), set of support (always use a clause from ¬α or derived from it), input resolution, subsumption (delete clauses implied by shorter ones)
Used in	Theorem provers such as Vampire and E, and software verification

Lesson: Resolution in first order logic

Table: · propositionalisation · forward chaining · backward chaining (prolog)

	Propositionalisation	Forward chaining	Backward chaining (Prolog)	Resolution
Works on	Any FOL KB	Definite clauses	Definite clauses	Any FOL KB in CNF
Complete	Yes, with growing term depth	Yes	No with plain depth first search (loops), yes with tabling	Yes, refutation complete
Main cost	k^v copies of every rule	Matching premises to facts	Repeated subgoals	Many useless resolvents
Best for	Small finite domains	Databases, rule engines, Datalog	Answering queries, logic programming	Mathematics, software verification

Lesson: Which first order inference method to use

TIPS AND TRAPS

COMMON MISTAKE The occurs check (does the variable appear inside the term?) makes unification cost quadratic in the size of the terms. Prolog leaves it out for speed, so Prolog can unify x with F(x) and build an endless term.

Lesson: Unification and lifting

REMEMBER Backward chaining is the reason Prolog programs can be read as logic and run as programs. But the order of rules and premises changes what happens, which pure logic does not care about.

Lesson: Backward chaining and Prolog

STEP 8 Core AI · Planning

8.1 Situation calculus and deductive planning

Situations, fluents, action axioms, the frame problem, successor state axioms and deductive planning.

KEY IDEAS

- **Situations** S0, Result(a, s). Fluents take s as last argument. Atemporal predicates do not.
- **Axioms** Possibility: Poss(a, s) ⇔ preconditions. Effect: Poss(a, s) ⇒ effect in Result(a, s).
- **Frame problem** Effect axioms do not say what stays. A × F frame axioms, or F successor state axioms (Reiter).
- **SSA form** F(Result(a, s)) ⇔ a made F true ∨ (F(s) ∧ a did not make F false). Needs unique action axioms.
- **Deductive planning** Green 1969: prove ∃s Goal(s), extract s. Slow, so STRIPS (1971).

FORMULAS

Successor state axiom (one per fluent F)

Exam favourite

$$\text{Poss}(a, s) \Rightarrow [F(\text{Result}(a, s)) \Leftrightarrow (a \text{ made } F \text{ true}) \vee (F(s) \wedge a \text{ did not make } F \text{ false})]$$

A fluent is true after an action if the action made it true, or if it was already true and the action did not make it false.

Lesson: The frame problem and successor state axioms

Frame axioms needed

Exam favourite

$$A \times F$$

One for each action and fluent pair that the action does not change. 20 actions and 50 fluents: up to 1,000.

Lesson: The maths: counting axioms and situations

Successor state axioms needed

Exam favourite

$$F$$

One per fluent, however many actions there are. 50 fluents: 50 axioms.

Lesson: The maths: counting axioms and situations

Situations up to depth d

$$1 + b + b^2 + \ldots + b^d$$

b actions possible in each situation. Situations are terms, so two different action sequences are always two different situations, even when the world looks the same.

Lesson: The maths: counting axioms and situations

TABLES

Table: planning language · states · actions · main use

Planning language	States	Actions	Main use
Situation calculus (McCarthy, 1963)	situations: terms such as Result(Go(B), S0)	axioms in first order logic	Deductive planning, theory of action
STRIPS (Fikes and Nilsson, 1971)	sets of ground facts	precondition, add list, delete list	Fast state space planners
PDDL (1998)	like STRIPS, with types and more	action schemas with parameters	The standard language of planning competitions

Lesson: What planning is

Table: part · meaning · example

Part	Meaning	Example
Initial situation S0	the start	At(Robot, A, S0), At(Gold, B, S0)
Result(a, s)	the situation after action a in s	S1 = Result(Go(B), S0)
Fluent	a relation or function whose value changes, with the situation as last argument	Holding(Gold, s)
Atemporal (eternal) predicate	true or false in every situation	Gold(G1), Adjacent(A, B)
Possibility axiom	when an action can be done	Poss(Grab, s) ⇔ ∃x At(Gold, x, s) ∧ At(Robot, x, s) ∧ ¬Holding(Gold, s)
Effect axiom	what an action changes	Poss(Grab, s) ⇒ Holding(Gold, Result(Grab, s))

Lesson: Situation calculus

Table: version of the problem · what goes wrong

Version of the problem	What goes wrong
Representational frame problem	Writing a frame axiom such as At(Gold, x, s) ∧ ¬Holding(Gold, s) ⇒ At(Gold, x, Result(Go(y), s)) for every pair of action and fluent: A × F axioms
Inferential frame problem	Carrying every unchanged fact forward one action at a time costs time for every fact at every step
Qualification problem	Listing every condition for an action to succeed is endless: the robot can grab the gold unless it is glued down, too heavy, ...
Ramification problem	Indirect effects: when the robot moves while holding the gold, the gold moves too

Lesson: The frame problem and successor state axioms

Table: strength of deductive planning · weakness

Strength of deductive planning	Weakness
Very expressive: quantifiers, functions, partial knowledge	General theorem provers are slow and have no planning heuristics
A proof that the plan works	No guarantee of a short plan
Clear meaning (semantics)	The frame problem must be handled with care, and the search space of proofs is huge

Lesson: Deductive planning: a plan is a proof

Table: · situation calculus · strips and pddl · event calculus

	Situation calculus	STRIPS and PDDL	Event calculus
Logic	Full first order	Ground facts, no quantifiers in states	First order, time points
Frame problem	Solved by successor state axioms	Solved by the STRIPS assumption: facts not in the delete list stay	Solved by circumscription over events
Actions at the same time	Hard	No (sequential plans)	Yes, events happen at time points
Planner speed	Slow (theorem proving)	Fast (heuristic search)	Slow
Used for	Theory, GOLOG robot programs, reasoning about what is known	Practical planners, competitions	Narratives, commonsense reasoning

Lesson: Situation calculus against other ways to describe actions

TIPS AND TRAPS

REMEMBER The search above is not a resolution prover. It looks through the same situation terms breadth first, which is what a fair prover ends up doing for this small problem, and it finds the same answer term.

Lesson: Deductive planning: a plan is a proof

8.2 STRIPS planning and sub goals

STRIPS actions, forward and backward search, planning heuristics, sub goals and the Sussman anomaly.

KEY IDEAS

- **STRIPS** State = set of facts. Action = PRE, ADD, DEL. RESULT = (s − DEL) ∪ ADD. Unmentioned facts stay.
- **Regression** g' = (g − ADD) ∪ PRE, for actions that add part of g and delete none of it.
- **Heuristics** Ignore preconditions, ignore delete lists (h^+ admissible, h^{FF} greedy), sub goal independence.
- **Sussman anomaly** C on A, goal A on B and B on C. Either sub goal order undoes the other. Shortest plan 3: C to table, B on C, A on B.
- **Complexity** STRIPS planning is PSPACE-complete. Blocks world states: 13 for 3 blocks, 73 for 4.

FORMULAS

Applying an action

Exam favourite

$$\text{RESULT}(s, a) = (s - \text{DEL}(a)) \cup \text{ADD}(a), \text{ if } \text{PRE}(a) \subseteq s$$

Delete first, then add.

Lesson: STRIPS: actions with preconditions, add lists and delete lists

Regression

Exam favourite

$$g' = (g - \text{ADD}(a)) \cup \text{PRE}(a)$$

a is **relevant** if it adds some fact of g, and **consistent** if it deletes none.

Lesson: Searching forwards and backwards

Ground actions in the blocks world with n blocks

Exam favourite

$$n(n-1) \text{ MoveToTable} + n(n-1) \text{ Move from the table} + n(n-1)(n-2) \text{ Move between blocks}$$

n = 3: 6 + 6 + 6 = 18. n = 4: 12 + 12 + 24 = 48.

Lesson: The maths: how big planning problems are

States of the blocks world

$$1, 3, 13, 73, 501, \ldots \text{ for } n = 1, 2, 3, 4, 5$$

The ways to arrange n named blocks into stacks. It grows faster than n!.

Lesson: The maths: how big planning problems are

Complexity

PlanSAT (is there any plan?) is PSPACE-complete for STRIPS

Bylander, 1994. Bounded PlanSAT (is there a plan of length ≤ k?) is also PSPACE-complete in general, and NP-complete for many restricted forms.

Lesson: The maths: how big planning problems are

TABLES

Table: domain · action schema (pddl style)

Domain	Action schema (PDDL style)
Blocks world	Move(b, x, y): PRE On(b, x) ∧ Clear(b) ∧ Clear(y). ADD On(b, y) ∧ Clear(x). DEL On(b, x) ∧ Clear(y)
Blocks world	MoveToTable(b, x): PRE On(b, x) ∧ Clear(b). ADD On(b, Table) ∧ Clear(x). DEL On(b, x)
Air cargo	Fly(p, from, to): PRE At(p, from) ∧ Plane(p) ∧ Airport(from) ∧ Airport(to). ADD At(p, to). DEL At(p, from)
Air cargo	Load(c, p, a): PRE At(c, a) ∧ At(p, a) ∧ Cargo(c) ∧ Plane(p) ∧ Airport(a). ADD In(c, p). DEL At(c, a)
Spare tyre	PutOn(t, Axle): PRE Tire(t) ∧ At(t, Ground) ∧ ¬At(Flat, Axle). ADD At(t, Axle). DEL At(t, Ground)

Lesson: STRIPS: actions with preconditions, add lists and delete lists

Table: · forward (progression) search · backward (regression) search

	Forward (progression) search	Backward (regression) search
Starts from	the initial state	the goal
A step	apply an action whose preconditions hold	regress the goal through a relevant action
Branching	every applicable action, often many irrelevant ones	only actions that achieve part of the goal
Search nodes	complete states	sets of goal facts (partial descriptions)
Heuristics	easy to compute from a full state	harder, since nodes are sets of states

Lesson: Searching forwards and backwards

Table: relaxation · heuristic · admissible

Relaxation	Heuristic	Admissible
Ignore preconditions	number of unsatisfied goal facts (when each action adds one goal fact), or a set cover	Yes in its exact form, the set cover version is hard to compute exactly
Ignore delete lists	h^+, the cost of the best relaxed plan	Yes, but finding h^+ is NP-hard
Ignore delete lists, greedy relaxed plan	h^{FF} (FF planner)	No, but very informative
Sub goal independence	sum (or max) of the costs of the goals solved one at a time	Max is admissible, sum is not

Lesson: Heuristics for planning

Table: · forward search + heuristic · backward search · linear sub goal planning

	Forward search + heuristic	Backward search	Linear sub goal planning	Partial order planning (8.3)	GraphPlan and SATPlan
Searches over	states	goal sets	sub goals, one at a time	partial plans	a planning graph, or a SAT formula
Handles interacting sub goals	Yes	Yes	No (gives long plans or fails)	Yes	Yes
Plan form	sequence	sequence	sequence	partial order: flexible	sequence of parallel steps
Strength	Fastest today (FF, Fast Downward)	Few relevant actions	Simple	Least commitment, easy to explain	Good for plans with parallel actions
Weakness	Many irrelevant actions without a heuristic	Goal sets can be impossible	Incomplete for interacting goals	Slower than heuristic state search	Large graphs or formulas

Lesson: Ways to plan with STRIPS actions

TIPS AND TRAPS

REMEMBER PDDL (Planning Domain Definition Language, 1998) is the modern form of STRIPS. It adds types, negative preconditions, equality and more, and is used in the International Planning Competition.

Lesson: STRIPS: actions with preconditions, add lists and delete lists

COMMON MISTAKE Regressing On(A, B) ∧ On(B, C) through Move(B, Table, C) gives a goal with both On(A, B) and Clear(B), which no state can satisfy. Backward planners need extra rules (state constraints) to throw such goals away early.

Lesson: Searching forwards and backwards

8.3 Partial order planning

Partial order plans, causal links, threats, promotion and demotion, and the POP algorithm.

KEY IDEAS

- **Partial plan** Steps, orderings A ≺ B, causal links A →p→ B, open preconditions. Start gives the initial state, Finish needs the goal.
- **Threat** C deletes p and could run between A and B of the link A →p→ B.
- **Fixes** Demotion: C ≺ A. Promotion: B ≺ C. Keep the orderings free of cycles, else go back.
- **Solution** No open preconditions, no threats, no cycles. Every linearization then reaches the goal.
- **Counting** Independent chains $n_1, \ldots, n_k$ give $(\Sigma\, n_i)! \div \Pi\, n_i!$ linearizations. Socks and shoes: 6.
- **Sussman by POP** MoveToTable(C, A) ≺ Move(B, Table, C) ≺ Move(A, Table, B), found by two promotions.

FORMULAS

Threat

Exam favourite

$$\text{C deletes p, and neither C} \prec \text{A nor B} \prec \text{C follows from the orderings}$$

C could sit between A and B.

Lesson: Threats, promotion and demotion

Demotion

$$\text{add C} \prec \text{A}$$

C runs before p is made.

Lesson: Threats, promotion and demotion

Promotion

$$\text{add B} \prec \text{C}$$

C runs after p is used.

Lesson: Threats, promotion and demotion

Linearizations of independent chains

Exam favourite

$$(n_1 + n_2 + \ldots + n_k)! \div (n_1!\, n_2!\, \ldots\, n_k!)$$

k chains of steps, chain i has n_i steps in a fixed order, and the chains do not constrain each other. This is the multinomial coefficient.

Lesson: The maths: counting orders and checking threats

Two chains of length 2

$$4! \div (2! \times 2!) = 24 \div 4 = 6$$

The socks and shoes plan allows 6 total orders.

Lesson: The maths: counting orders and checking threats

Steps with no orderings at all

$$n!$$

Every order is allowed.

Lesson: The maths: counting orders and checking threats

Causal links in a finished plan

$$\text{number of links = total number of preconditions of all steps (including Finish)}$$

Each precondition gets exactly one link.

Lesson: The maths: counting orders and checking threats

Cost of one threat check

$$\text{links} \times \text{steps pairs, each needing a reachability test in the ordering graph}$$

Reachability is a graph search, about O(steps + orderings) each time.

Lesson: The maths: counting orders and checking threats

TABLES

Table: part · meaning · socks and shoes example

Part	Meaning	Socks and shoes example
Steps	actions in the plan, plus Start and Finish	RightSock, RightShoe, LeftSock, LeftShoe
Ordering A ≺ B	A must run before B	RightSock ≺ RightShoe
Causal link A →p→ B	A makes p true for B, and nothing may undo p between them	RightSock →RightSockOn→ RightShoe
Open precondition	a precondition with no causal link yet	at the start: RightShoeOn and LeftShoeOn of Finish

Lesson: Partial plans: steps, orderings and causal links

Table: · partial order planning · forward state search with heuristics · linear sub goal planning

	Partial order planning	Forward state search with heuristics	Linear sub goal planning	GraphPlan	Hierarchical task networks (HTN)
Searches over	partial plans	world states	one sub goal at a time	a layered planning graph	ways to break big tasks into smaller ones
Plan form	partial order, flexible	total order	total order	layers of parallel actions	partial or total order
Interacting sub goals	handled by threat fixes	handled by search	often fails or gives long plans	handled by mutex links	handled by the method author
Speed today	slower on large problems	fastest general method (FF, Fast Downward)	fast but incomplete	good on small parallel problems	very fast with good methods
Best use	flexible plans to explain or to run in parallel	large benchmark problems	teaching, independent goals	parallel plans, heuristics	real systems with expert knowledge

Lesson: POP compared with other planners

TIPS AND TRAPS

REMEMBER

A partial plan is **consistent** when its orderings have no cycle ($A \prec B$ and $B \prec A$ cannot both hold) and no causal link is threatened. A consistent plan with no open preconditions is a **solution**: every linearization of it reaches the goal.

Lesson: Partial plans: steps, orderings and causal links

REMEMBER

Some textbooks add a third fix for lifted plans with variables: **separation**, a constraint such as $x \neq C$ that stops the threatening step from deleting this exact fact.

Lesson: Threats, promotion and demotion

REMEMBER

POP is **sound** (every plan it returns works) and **complete** (it finds a plan if one exists) when the step and threat choices are searched fully, for example breadth first or with iterative deepening on the number of steps. The animation uses iterative deepening, so it reports the step limit it needed.

Lesson: The POP algorithm

STEP 9 Core AI · Probability and Bayesian networks

9.1 Probability for AI agents

Probability axioms, joint distributions, Bayes' rule, independence and naive Bayes.

KEY IDEAS

- **Axioms** $0 \le P(\omega) \le 1$, $\Sigma\, P(\omega) = 1$. $P(a \vee b) = P(a) + P(b) - P(a \wedge b)$. $P(a \mid b) = P(a \wedge b) \div P(b)$.
- **Enumeration** $P(X \mid e) = \alpha\, \Sigma_y P(X, e, y)$. Marginalise, keep matching worlds, normalise. Full joint: 2^n entries.
- **Bayes' rule** $P(c \mid e) = P(e \mid c)\, P(c) \div P(e)$, $P(e) = \Sigma_c P(e \mid c)\, P(c)$. Odds form: posterior odds = prior odds × LR.
- **Base rate** Prior 0.01, sensitivity 0.9, false positive 0.09 gives $P(\text{ill} \mid +) \approx 0.092$.
- **Conditional independence** $P(a, b \mid c) = P(a \mid c)\, P(b \mid c)$. Naive Bayes: $P(C \mid e) = \alpha\, P(C) \prod P(e_i \mid C)$, $2n + 1$ numbers.
- **Dentist numbers** $P(\text{cavity}) = 0.2$, $P(\text{toothache}) = 0.2$, $P(\text{cavity} \mid \text{toothache}) = 0.6$, $P(\text{cavity} \mid \text{toothache}, \text{catch}) \approx 0.871$.

FORMULAS

Kolmogorov axioms

Exam favourite

$$0 \le P(\omega) \le 1 \text{ for every world } \omega, \text{ and the sum of } P(\omega) \text{ over all worlds is } 1$$

Everything else follows from these.

Lesson: Why agents need probability

Probability of a sentence

$$P(\varphi) = \Sigma\, P(\omega) \text{ over worlds } \omega \text{ where } \varphi \text{ is true}$$

A sentence is a set of worlds.

Lesson: Why agents need probability

Inclusion and exclusion

$$P(a \vee b) = P(a) + P(b) - P(a \wedge b)$$

Do not count the shared worlds twice.

Lesson: Why agents need probability

Conditional probability

Exam favourite

$$P(a \mid b) = P(a \wedge b) \div P(b), \text{ when } P(b) > 0$$

Belief in a after learning only b.

Lesson: Why agents need probability

Marginalisation

$$P(Y) = \Sigma \text{ over } z \text{ of } P(Y, z)$$

Sum out the other variables Z.

Lesson: The full joint distribution and inference by enumeration

Inference by enumeration

Exam favourite

$$P(X \mid e) = \alpha\, P(X, e) = \alpha\, \Sigma \text{ over } y \text{ of } P(X, e, y)$$

X query, e evidence, y hidden variables.

Lesson: The full joint distribution and inference by enumeration

Size of the full joint

$$2^n \text{ entries for n true or false variables } (2^n - 1 \text{ free numbers})$$

This is why we need Bayesian networks.

Lesson: The full joint distribution and inference by enumeration

Bayes' rule

Exam favourite

$$P(c \mid e) = P(e \mid c)\, P(c) \div P(e)$$

posterior = likelihood × prior ÷ evidence.

Lesson: Bayes' rule

Total probability

$$P(e) = P(e \mid c)\, P(c) + P(e \mid \neg c)\, P(\neg c)$$

The bottom line of Bayes' rule.

Lesson: Bayes' rule

Odds form

$$\text{posterior odds} = \text{prior odds} \times \text{likelihood ratio, where the likelihood ratio is } P(e \mid c) \div P(e \mid \neg c)$$

Easy to update one test at a time.

Lesson: Bayes' rule

Independence

$$P(a \wedge b) = P(a)\, P(b)$$

Same as $P(a \mid b) = P(a)$.

Lesson: Independence, conditional independence and naive Bayes

Conditional independence

Exam favourite

$$P(a \wedge b \mid c) = P(a \mid c)\, P(b \mid c)$$

Written $a \perp\!\!\!\perp b \mid c$.

Lesson: Independence, conditional independence and naive Bayes

Naive Bayes

$$P(\text{Cause}, e_1, \ldots, e_n) = P(\text{Cause}) \times \Pi\, P(e_i \mid \text{Cause})$$

Then $P(\text{Cause} \mid e) = \alpha\, P(\text{Cause})\, \Pi\, P(e_i \mid \text{Cause})$.

Lesson: Independence, conditional independence and naive Bayes

Laplace (add one) smoothing

$$P(w \mid c) = (\text{count}(w, c) + 1) \div (\text{total words in } c + \text{vocabulary size})$$

A word never seen with class c no longer gives probability 0.

Lesson: Independence, conditional independence and naive Bayes

Product rule

$$P(a \wedge b) = P(a \mid b)\,P(b) = P(b \mid a)\,P(a)$$

Bayes' rule comes from these two forms.

Lesson: The maths: rules every probability step uses

Chain rule Exam favourite

$$P(x_1, \ldots, x_n) = \Pi \text{ over } i \text{ of } P(x_i \mid x_1, \ldots, x_{i-1})$$

Always true. Bayesian networks shorten each condition.

Lesson: The maths: rules every probability step uses

Bayes with background evidence

$$P(c \mid e, b) = P(e \mid c, b)\,P(c \mid b) \div P(e \mid b)$$

Keep b on every term.

Lesson: The maths: rules every probability step uses

Expected value

$$E[X] = \Sigma\, x\, P(X = x)$$

Used again for decisions and for reinforcement learning.

Lesson: The maths: rules every probability step uses

Variance

$$\mathrm{Var}[X] = E[X^2] - (E[X])^2$$

How spread out X is.

Lesson: The maths: rules every probability step uses

TABLES

Table: · probability theory · logic with default rules · certainty factors (mycin)

	Probability theory	Logic with default rules	Certainty factors (MYCIN)	Fuzzy logic	Dempster Shafer theory
What a number means	degree of belief that a fact is true	no numbers, rules hold "by default"	change in belief from one rule	degree to which a vague fact holds ("tall")	belief and plausibility, an interval
Combines evidence by	Bayes' rule	withdrawing conclusions	fixed formulas per rule	min and max	Dempster's rule
Sound for chains of reasoning	Yes	Partly	No (can double count evidence)	It answers a different question (vagueness)	Yes, but costly
Used today	Everywhere: medicine, robotics, speech, spam	Knowledge bases	Historic	Control systems	Sensor fusion niches

Lesson: Probability compared with other ways to handle uncertainty

TIPS AND TRAPS

REMEMBER Why these rules and no others: de Finetti (1931) showed that an agent whose beliefs break the axioms can be offered a set of bets it accepts and always loses (a Dutch book). So a rational agent that bets on its beliefs must follow the axioms.

Lesson: Why agents need probability

COMMON MISTAKE P(a | b) is not "if b then a with probability p". It is belief in a when b is all the agent knows. If more evidence c arrives, the agent must use P(a | b ∧ c).

Lesson: Why agents need probability

COMMON MISTAKE The base rate fallacy: most people guess about 90 percent for the test above. Healthy people are so many that their few false positives outnumber the true positives.

Lesson: Bayes' rule

REMEMBER

Naive Bayes is called naive because the effects are rarely truly independent given the class. It still classifies well in practice (spam filters, document topics), because the class with the highest score is often right even when the probabilities themselves are off.

Lesson: Independence, conditional independence and naive Bayes

9.2 Bayesian networks and Markov networks

Directed and undirected graphical models: CPTs, d-separation, Markov blankets, factors and Z.

KEY IDEAS

- **Bayesian network** DAG + CPT per node. $P(x_1..x_n) = \Pi\ P(x_i \mid \text{parents})$. Size $\Sigma\ 2\text{^}(\text{parents})$. Burglary: 10 vs 31.
- **d-separation** Chain and fork blocked if the middle is observed. Collider blocked unless the middle or a descendant is observed.
- **Markov blanket** Parents, children, children's other parents. Given it, a node is independent of all others.
- **Noisy OR** $P(\neg x \mid \text{causes}) = \Pi\ q_j$ over present causes. n numbers instead of 2^n.
- **Markov network** $P(x) = (1 \div Z)\ \Pi\ \varphi_c(x_c)$, $Z = \Sigma_x\ \Pi\ \varphi_c$. Separation in the graph gives independence. Moralise a BN: marry parents, drop arrows.
- **Key numbers** $P(j, m, a, \neg b, \neg e) \approx 0.000628$. Misconception network $Z = 7{,}201{,}840$.

FORMULAS

Network semantics — Exam favourite

$$P(x_1, ..., x_n) = \Pi \text{ over } i \text{ of } P(x_i \mid \text{parents}(X_i))$$

One table entry per variable.

Lesson: Bayesian networks: a graph plus small tables

Size of a CPT

a true or false node with k true or false parents needs 2^k numbers

One number per row, since P(false) = 1 − P(true).

Lesson: Bayesian networks: a graph plus small tables

Size of the network

Σ over nodes of $2\text{^}(\text{number of parents})$

n nodes with at most k parents: at most $n\ 2^k$, against $2^n - 1$.

Lesson: Bayesian networks: a graph plus small tables

Markov blanket — Exam favourite

parents, children, and the children's other parents

Given its blanket, a node is independent of every other node.

Lesson: Reading independence from the graph: d-separation

Noisy OR — Exam favourite

$$P(\neg x \mid \text{parents}) = \Pi \text{ over true parents } j \text{ of } q_j$$

q_j is the chance that cause j alone fails.

Lesson: Compact tables: deterministic nodes, noisy OR and continuous variables

Linear Gaussian

$$P(x \mid h) = N(x \mid \text{mean } a\ h + b, \text{variance } \sigma^2)$$

A continuous child of a continuous parent h.

Lesson: Compact tables: deterministic nodes, noisy OR and continuous variables

Gibbs distribution — Exam favourite

$$P(x) = (1 \div Z)\ \Pi \text{ over cliques } c \text{ of } \varphi_c(x_c)$$

Factors need not add up to 1.

Lesson: Undirected models: Markov networks

Partition function

$$Z = \Sigma \text{ over all worlds } x \text{ of } \Pi\ \varphi_c(x_c)$$

A sum with 2^n terms: computing it is the hard part.

Lesson: Undirected models: Markov networks

Hammersley Clifford theorem

a positive distribution has the independencies of the graph exactly when it factorises over the cliques

Links the graph to the formula.

Lesson: Undirected models: Markov networks

Log linear (energy) form

$$P(x) = (1 \div Z) \exp(\Sigma\ w_i\ f_i(x))$$

Used by conditional random fields and Boltzmann machines.

Lesson: Undirected models: Markov networks

Parameters with at most k parents Exam favourite

$n\ 2^k$, against $2^n - 1$ for the full joint

n = 30, k = 5: 960 against about 1.07×10^9.

Lesson: The maths: counting parameters and checking independence

Local Markov property

$$X \perp\!\!\!\perp \text{NonDescendants}(X) \mid \text{Parents}(X)$$

Equivalent to the product formula for a Bayesian network.

Lesson: The maths: counting parameters and checking independence

Markov blanket property

$P(X \mid \text{everything else}) = P(X \mid MB(X)) \propto P(X \mid \text{parents}) \times \Pi$ over children Y of $P(y \mid \text{parents}(Y))$

Used by Gibbs sampling in chapter 9.3.

Lesson: The maths: counting parameters and checking independence

Moral ancestral graph test

$X \perp\!\!\!\perp Y \mid Z$ exactly when X and Y are not joined in the moralised graph of the ancestors of X, Y and Z, after removing Z

Lauritzen and others, 1990. This is the algorithm in the code above.

Lesson: The maths: counting parameters and checking independence

TABLES

Table: node · parents · table (p of true)

Node	Parents	Table (P of true)
Burglary	none	0.001
Earthquake	none	0.002
Alarm	Burglary, Earthquake	b e: 0.95, b ¬e: 0.94, ¬b e: 0.29, ¬b ¬e: 0.001
JohnCalls	Alarm	a: 0.90, ¬a: 0.05
MaryCalls	Alarm	a: 0.70, ¬a: 0.01

Lesson: Bayesian networks: a graph plus small tables

Table: pattern · example · blocked when · plain meaning

Pattern	Example	Blocked when	Plain meaning
Chain A → M → B	Burglary → Alarm → JohnCalls	M is observed	once you know the alarm rang, the burglary tells nothing more about John
Fork A ← M → B	JohnCalls ← Alarm → MaryCalls	M is observed	both calls depend only on the alarm
Collider A → M ← B	Burglary → Alarm ← Earthquake	M and all its descendants are not observed	two causes are independent until we see their common effect (explaining away)

Lesson: Reading independence from the graph: d-separation

Table: · bayesian network (directed) · markov network (undirected)

	Bayesian network (directed)	Markov network (undirected)
Parts	CPTs P(X | parents)	factors φ on cliques
Normalised	each CPT row adds up to 1, so the product is a probability	only after dividing by Z
Independence test	d-separation	plain graph separation
Can express	explaining away (colliders)	cycles of mutual influence, such as A - B - C - D - A
Learning the numbers	counting, one table at a time	needs Z, so iterative methods

Lesson: Undirected models: Markov networks

Table: · full joint table · naive bayes · bayesian network

	Full joint table	Naive Bayes	Bayesian network	Markov network	Factor graph
Structure	none	one cause, independent effects	directed graph, no cycles	undirected graph	variables joined to factors
Size	$2^n - 1$	2n + 1	Σ 2^(parents)	Σ clique table sizes	Σ factor table sizes
Strength	answers anything	tiny and fast	causal, readable, easy to learn from counts	symmetric relations, images, text labels	one form for both directed and undirected models, used by message passing
Weakness	exponential size	ignores links between effects	cannot show some symmetric dependencies	Z is hard, numbers are hard to read	less intuitive for experts

Lesson: Graphical models compared

TIPS AND TRAPS

REMEMBER Build a network in causal order: causes first, then effects. The diagnostic order (MaryCalls, JohnCalls, Alarm, Burglary, Earthquake) is still correct but needs more arrows and harder numbers, such as P(Earthquake | Burglary, Alarm).

Lesson: Bayesian networks: a graph plus small tables

REMEMBER Explaining away: if the alarm rang, a burglary is likely. If we then learn an earthquake happened, the burglary becomes less likely, because the earthquake already explains the alarm. The two causes were independent before we saw the alarm.

Lesson: Reading independence from the graph: d-separation

REMEMBER To turn a Bayesian network into a Markov network, **moralise** it: join the parents of each child (they "marry"), then drop the arrow heads. The result may lose some independencies, such as Burglary ⫫ Earthquake.

Lesson: Undirected models: Markov networks

9.3 Inference in Bayesian networks

Enumeration, variable elimination, complexity, rejection sampling, likelihood weighting and Gibbs sampling.

KEY IDEAS

- **Enumeration** P(X | e) = α Σ_y Π P(x_i | parents). O(2^n) time, O(n) space. P(b | j, m) ≈ 0.284.
- **Variable elimination** Factors from CPTs with evidence. Multiply factors with X, sum out X. Cost exponential in the largest factor (treewidth). Drop non ancestors.

- **Complexity** Polytree: linear. General: #P-hard (Cooper 1990).
- **Samplers** Rejection: N(x, e) ÷ N(e). LW: w = Π P(e_i | parents). Gibbs: sample from P(x | mb(X)). Error ≈ √(p(1 − p) ÷ N).
- **Key numbers** LW sample weight 0.1 × 0.99 = 0.099. P(Rain | s, w) = 0.3204. P(Rain | s) = 0.3.

FORMULAS

Enumeration

Exam favourite

$$P(B \mid j, m) = \alpha \Sigma_e \Sigma_a P(B) P(e) P(a \mid B, e) P(j \mid a) P(m \mid a)$$

Hidden variables E and A are summed out.

Lesson: Exact inference by enumeration

Moving factors outside

$$\alpha P(B) \Sigma_e P(e) \Sigma_a P(a \mid B, e) P(j \mid a) P(m \mid a)$$

P(B) does not depend on e or a, so it moves out of both sums.

Lesson: Exact inference by enumeration

Cost

$$\text{time } O(2^n), \text{ memory } O(n)$$

Depth first over the hidden variables.

Lesson: Exact inference by enumeration

Pointwise product

$$f(X, Y) \times g(Y, Z) = h(X, Y, Z), \text{ with } h(x, y, z) = f(x, y)\, g(y, z)$$

The result covers the union of the variables.

Lesson: Variable elimination

Summing out

Exam favourite

$$h(X, Z) = \Sigma_y f(X, y, Z)$$

Add the entries for each value of the removed variable.

Lesson: Variable elimination

Cost

$$\text{exponential in the size of the largest factor made, which depends on the order}$$

The best possible largest factor size, minus one, is the treewidth of the graph.

Lesson: Variable elimination

Rejection sampling

$$P(X \mid e) \approx N(X, e) \div N(e)$$

Consistent, but wastes the fraction 1 − P(e) of samples.

Lesson: Approximate inference by sampling

Likelihood weight

Exam favourite

$$w = \Pi \text{ over evidence variables } E_i \text{ of } P(e_i \mid \text{parents}(E_i))$$

Estimate: sum of weights with X true ÷ sum of all weights.

Lesson: Approximate inference by sampling

Gibbs step

$$P(x \mid mb(X)) = \alpha\, P(x \mid \text{parents}(X))\ \Pi \text{ over children } Y \text{ of } P(y \mid \text{parents}(Y))$$

Only the Markov blanket is needed.

Lesson: Approximate inference by sampling

Error of an estimate

$$\text{standard error} \approx \sqrt{p\,(1 - p) \div N}$$

Four times the samples halves the error.

Lesson: Approximate inference by sampling

Prior sampling is consistent

$$S(x_1, \ldots, x_n) = \Pi\, P(x_i \mid \text{parents}(X_i)) = P(x_1, \ldots, x_n)$$

Each world is drawn with exactly its probability.

Lesson: The maths: why the samplers give the right answer

Likelihood weighting is consistent

Exam favourite

$$S_WS(z, e)\, w(z, e) = \Pi\, P(z_i \mid \text{parents}) \times \Pi\, P(e_i \mid \text{parents}) = P(z, e)$$

The sampling distribution times the weight is the true joint.

Lesson: The maths: why the samplers give the right answer

Gibbs sampling

the posterior P(x | e) is the stationary distribution of the Markov chain

Detailed balance: π(x) q(x → x') = π(x') q(x' → x).

Lesson: The maths: why the samplers give the right answer

Variable elimination size

eliminating X makes a factor over all variables that share a factor with X

Bad orders build large factors.

Lesson: The maths: why the samplers give the right answer

TABLES

Table: · enumeration · variable elimination · junction tree

	Enumeration	Variable elimination	Junction tree	Rejection sampling	Likelihood weighting	Gibbs sampling (MCMC)	Loopy belief propagation
Exact	Yes	Yes	Yes	No (converges)	No (converges)	No (converges)	No (may not converge)
Cost	$O(2^n)$ time, O(n) memory	exponential in the largest factor	exponential in the largest cluster	grows as 1 ÷ P(e)	cheap per sample	cheap per step	cheap per pass
Good for	tiny networks, checking	one query, small treewidth	all marginals at once	likely evidence	evidence near the roots	large networks, any evidence	large networks with loops, error correcting codes
Weak when	more than about 25 variables	large treewidth	large treewidth	rare evidence	evidence at the leaves	near deterministic tables (slow mixing)	strong loops

Lesson: Inference methods compared

TIPS AND TRAPS

COMMON MISTAKE Enumeration repeats work: P(j | a) P(m | a) is computed again for each value of e. Variable elimination stores such parts once.

Lesson: Exact inference by enumeration

REMEMBER Irrelevant variables: every variable that is not an ancestor of the query or the evidence can be removed before starting. Summing it out only gives factors full of 1s, as the second tip of the animation shows.

Lesson: Variable elimination

COMMON MISTAKE Likelihood weighting suffers when evidence comes late in the order (at the leaves): the samples ignore it while choosing the earlier variables, so most of them get tiny weights.

Lesson: Approximate inference by sampling

9.4 Causality: seeing, doing and imagining

Causal networks, the do operator, back door adjustment, Simpson's paradox and counterfactuals.

KEY IDEAS

- **Ladder** Seeing P(y | x), doing P(y | do(x)), imagining counterfactuals.
- **do operator** Cut arrows into X, drop its CPT (truncated product). Root X: do equals see.
- **Back door** Z has no descendant of T and blocks all T ← … paths: P(r | do(t)) = Σ_z P(r | t, z) P(z).
- **Simpson's paradox** Kidney stones: seeing A 0.78 < B 0.826, doing A 0.833 > B 0.779. Adjust for confounders, not mediators or colliders.
- **Counterfactual** Abduction (find U), action (set X), prediction (recompute Y). Front door: Σ_m P(m | x) Σ_x' P(y | m, x') P(x').

- **Sprinkler** P(Rain | s) = 0.3 but P(Rain | do(s)) = 0.5. P(WetGrass | do(s)) = 0.945.

FORMULAS

Truncated factorisation

Exam favourite

$$P(v \mid do(x)) = \Pi \text{ over } i \text{ with } V_i \text{ not } X \text{ of } P(v_i \mid \text{parents}(V_i)), \text{ with } X = x$$

The table of X is removed.

Lesson: The do operator and graph surgery

Root causes

$$\text{if } X \text{ has no parents, } P(y \mid do(x)) = P(y \mid x)$$

Nothing to cut, so seeing equals doing.

Lesson: The do operator and graph surgery

Average causal effect

$$ACE = P(y \mid do(x)) - P(y \mid do(\neg x))$$

The quantity a randomised trial estimates.

Lesson: The do operator and graph surgery

Back door adjustment

Exam favourite

$$P(r \mid do(t)) = \Sigma_z \, P(r \mid t, z) \, P(z)$$

Weight each group by its share of everyone, not its share among those treated.

Lesson: Confounders, the back door criterion and Simpson's paradox

Compare with seeing

$$P(r \mid t) = \Sigma_z \, P(r \mid t, z) \, P(z \mid t)$$

The only difference is P(z | t) in place of P(z).

Lesson: Confounders, the back door criterion and Simpson's paradox

Back door

Exam favourite

$$P(y \mid do(x)) = \Sigma_z \, P(y \mid x, z) \, P(z)$$

Z blocks every back door path and has no descendants of X.

Lesson: The maths: adjustment, front door and effects

Front door

$$P(y \mid do(x)) = \Sigma_m \, P(m \mid x) \, \Sigma_x' \, P(y \mid m, x') \, P(x')$$

M carries all of the effect and has no hidden common cause with X.

Lesson: The maths: adjustment, front door and effects

Inverse probability weighting

$$P(y \mid do(x)) = \Sigma \text{ over units with } X = x \text{ of } 1 \div (N \, P(x \mid z))$$

Same answer as back door adjustment, used with many variables in Z.

Lesson: The maths: adjustment, front door and effects

TABLES

Table: rung · question · written as · example

Rung	Question	Written as	Example
1. Association (seeing)	What if I see x?	P(y | x)	Patients who took the drug recovered more often.
2. Intervention (doing)	What if I do x?	P(y | do(x))	Would recovery rise if we gave everyone the drug?
3. Counterfactual (imagining)	What if I had done x instead?	P(y_x | x', y')	This patient took the drug and recovered. Would they have recovered without it?

Lesson: Seeing, doing and imagining: the ladder of causation

Table: role of z · graph · adjust for z?

Role of Z	Graph	Adjust for Z?
Confounder	T ← Z → R	Yes: it closes the back door
Mediator	T → Z → R	No: it would remove part of the effect you want
Collider	T → Z ← R	No: conditioning on it creates a false link (Berkson's paradox)

Lesson: Confounders, the back door criterion and Simpson's paradox

Table: · randomised trial · back door adjustment · front door adjustment

	Randomised trial	Back door adjustment	Front door adjustment	Instrumental variables	Causal discovery from data
Needs	the power to assign treatment	all confounders measured	a mediator that carries the whole effect	a variable that moves T but affects R only through T	assumptions such as faithfulness
Gives	P(r | do(t)) directly	P(r | do(t)) from observations	P(r | do(t)) even with a hidden confounder	the effect under linear or monotone models	a set of possible graphs
Weak when	costly, slow or unethical	a confounder is missed	the mediator has other causes shared with T	the instrument is weak	data are few or variables hidden
Example	drug trials	kidney stones	smoking, tar and cancer	distance to a hospital as an instrument	the PC algorithm (Spirtes and Glymour, 1991)

Lesson: Ways to learn what causes what

TIPS AND TRAPS

COMMON MISTAKE A plain Bayesian network can be read in a non causal order and still give the right P(y | x). It can only answer rung 2 questions when its arrows are causal.

Lesson: Seeing, doing and imagining: the ladder of causation

REMEMBER A randomised controlled trial (Fisher, 1935) performs do(X) in the real world: a coin, not the patient or the doctor, decides the treatment, so every arrow into the treatment is cut.

Lesson: The do operator and graph surgery

COMMON MISTAKE Simpson's paradox is not a fault in arithmetic. Which answer is right, the pooled one or the one inside groups, depends on the causal graph. With stone size as a confounder, the group answer (A is better) is right. If Z were a mediator, the pooled answer would be right.

Lesson: Confounders, the back door criterion and Simpson's paradox

STEP 10 Core AI · Reinforcement learning

10.1 Markov decision processes

MDPs, the Bellman equation, value iteration, policy iteration and Q-values.

KEY IDEAS

- **MDP** States, actions, P(s' | s, a), R, γ. Solution: policy π(s). Utility = E[Σ γ^t R_t] ≤ R_max ÷ (1 − γ).
- **Bellman** U(s) = R(s) + γ max_a Σ P(s' | s, a) U(s'). Q(s, a) = Σ P [R + γ max Q(s', a')]. U = max_a Q.
- **Value iteration** Repeat the update. Contraction by γ. Stop when δ < ε(1 − γ) ÷ γ. 4 × 3: U(1,1) = 0.705.
- **Policy iteration** Evaluate (linear equations, O(n³)), improve greedily, stop when unchanged. Few rounds.
- **Racing car** γ = 0.9: V_1 = (2, 1), V_2 = (3.35, 2.35). Optimal: cool fast, warm slow, V = (15.5, 14.5).

FORMULAS

Utility of a reward sequence Exam favourite

$$U = R_0 + \gamma R_1 + \gamma^2 R_2 + \ldots = \Sigma \gamma^t R_t$$

γ between 0 and 1. With γ < 1 the sum is at most R_max ÷ (1 − γ).

Lesson: Markov decision processes

Utility of a state under π

$$U^{\pi}(s) = E[\Sigma \gamma^t R(S_t) \mid S_0 = s, \text{actions from } \pi]$$

Expected discounted reward when starting in s.

Lesson: Markov decision processes

Optimal policy

$$\pi^*(s) = \text{argmax over } a \text{ of } \Sigma\, P(s' \mid s, a)\, U(s')$$

One step look ahead with the true utilities.

Lesson: Markov decision processes

Bellman equation Exam favourite

$$U(s) = R(s) + \gamma \max \text{ over } a \text{ of } \Sigma \text{ over } s' \text{ of } P(s' \mid s, a)\, U(s')$$

One equation per state.

Lesson: The Bellman equation and value iteration

Bellman update

$$U_{i+1}(s) \leftarrow R(s) + \gamma \max \text{ over } a \text{ of } \Sigma\, P(s' \mid s, a)\, U_i(s')$$

Value iteration repeats this.

Lesson: The Bellman equation and value iteration

Contraction

$$\|U_{i+1} - U\| \le \gamma\, \|U_i - U\|$$

Each sweep shrinks the error by at least γ (in the max norm), so value iteration always converges for $\gamma < 1$.

Lesson: The Bellman equation and value iteration

Stopping rule

$$\text{if } \|U_{i+1} - U_i\| < \varepsilon\,(1-\gamma) \div \gamma \text{ then } \|U_{i+1} - U\| < \varepsilon$$

A safe test for "close enough".

Lesson: The Bellman equation and value iteration

Policy evaluation Exam favourite

$$U^{\pi}(s) = R(s) + \gamma\, \Sigma\, P(s' \mid s, \pi(s))\, U^{\pi}(s')$$

n linear equations, $O(n^3)$ to solve exactly.

Lesson: Policy iteration and Q-values

Q-value

$$Q(s, a) = \Sigma\, P(s' \mid s, a)\, [R(s, a, s') + \gamma \max \text{ over } a' \text{ of } Q(s', a')]$$

$U(s) = \max$ over a of $Q(s, a)$.

Lesson: Policy iteration and Q-values

Termination

there are $|A|^{|S|}$ policies and each round is at least as good as the last

So policy iteration ends after a finite number of rounds.

Lesson: Policy iteration and Q-values

Bound on utilities Exam favourite

$$|U(s)| \le R_{max} \div (1-\gamma)$$

Geometric series: $R_{max}\,(1 + \gamma + \gamma^2 + \ldots)$.

Lesson: The maths: bounds and counts

Sweeps needed

$$N = \lceil \log(2\, R_{max} \div (\varepsilon\,(1-\gamma))) \div \log(1 \div \gamma) \rceil$$

Enough sweeps for error below ε, starting from 0. It grows fast as γ nears 1.

Lesson: The maths: bounds and counts

Policy loss

$$\text{if } \|U_i - U\| < \varepsilon \text{ then } \|U^{\pi_i} - U\| < 2\,\varepsilon\,\gamma \div (1-\gamma)$$

The greedy policy is good even before the values have fully converged.

Lesson: The maths: bounds and counts

Number of policies

$$|A|^{|S|}$$

4×3 world: $4^9 = 262{,}144$ policies for the 9 squares where a choice matters.

Lesson: The maths: bounds and counts

TABLES

Table: part · 4 × 3 world · racing car

Part	4 × 3 world	Racing car
States	11 squares	cool, warm, overheated (end)
Actions	Up, Down, Left, Right	slow, fast
Transitions	intended move 0.8, each side 0.1	cool + fast: cool or warm (0.5 each), warm + fast: overheated
Rewards	R = −0.04 per step, +1 and −1 at the end squares	slow +1, fast +2, overheating −10
Discount	γ = 1	γ = 0.9

Lesson: Markov decision processes

Table: · value iteration · policy iteration · modified policy iteration

	Value iteration	Policy iteration	Modified policy iteration	Linear programming	Expectimax search (6.3)
Works on	utilities	policies	policies, with a few sweeps for evaluation	utilities as LP variables	a tree from the current state
Cost per round	$O(\|S\|^2 \|A\|)$	$O(\|S\|^3)$ for exact evaluation plus $O(\|S\|^2 \|A\|)$	$O(k \|S\|^2)$ plus $O(\|S\|^2 \|A\|)$	polynomial overall	exponential in the depth
Rounds	many when γ is near 1	few	few	one LP	one search per move
Best for	simple and general	small state spaces	the usual choice in practice	theory: MDPs are solvable in polynomial time	huge state spaces, online decisions

Lesson: MDP solvers compared

TIPS AND TRAPS

REMEMBER — Why discount: a reward now is worth more than the same reward later, an agent may stop at any time, and with $\gamma < 1$ infinite sums stay finite. γ close to 1 means a patient agent.

Lesson: Markov decision processes

10.2 Passive reinforcement learning

Direct utility estimation, adaptive dynamic programming and temporal difference learning for a fixed policy.

KEY IDEAS

- **Passive RL** Fixed π, unknown P and R, learn U^{π} from trials. Direct estimation, ADP, TD.
- **TD(0)** $U(s) \leftarrow U(s) + \alpha\,(R(s) + \gamma\,U(s') - U(s))$. δ = TD error. Robbins Monro: $\Sigma\alpha = \infty$, $\Sigma\alpha^2 < \infty$.
- **ADP** $\hat{P}(s' \mid s, a) = N(s, a, s') \div N(s, a)$, then policy evaluation. Fewest trials, most work per step.
- **Bias and variance** Monte Carlo return: unbiased, high variance. TD target: biased, low variance. TD(λ) mixes.
- **Key numbers** Three trials: $U(1,1) \approx 0.0933$ (true 0.705). $\hat{P}((2,3) \mid (1,3)) = 2/3$. Random walk values $k \div 6$.

FORMULAS

TD(0) update — Exam favourite

$$U(s) \leftarrow U(s) + \alpha\,(R(s) + \gamma\,U(s') - U(s))$$

α is the learning rate.

Lesson: Temporal difference learning

TD error

$$\delta = R(s) + \gamma\,U(s') - U(s)$$

Positive: things went better than expected.

Lesson: Temporal difference learning

Learning rate schedule

$\Sigma\ \alpha_n = \infty$ and $\Sigma\ \alpha_n^2 < \infty$, for example $\alpha_n = 1 \div n$ or $60 \div (59 + n)$

Robbins and Monro (1951). Then TD converges.

Lesson: Temporal difference learning

TD(λ)

$U(x) \leftarrow U(x) + \alpha\ \delta\ e(x)$, with traces $e(x)$ that decay by $\gamma\ \lambda$ each step

$\lambda = 0$ is TD(0), $\lambda = 1$ is like averaging whole trials.

Lesson: Temporal difference learning

Learned model Exam favourite

$\hat{P}(s' \mid s, a) = N(s, a, s') \div N(s, a)$

Maximum likelihood from counts.

Lesson: Adaptive dynamic programming

Monte Carlo target

$G_t = R_t + \gamma\ R_{t+1} + \gamma^2\ R_{t+2} + \ldots$

Unbiased, but high variance: it depends on every later random step.

Lesson: The maths: bias, variance and fixed points

TD target Exam favourite

$R_t + \gamma\ U(S_{t+1})$

Biased while U is wrong, but low variance: it depends on one step.

Lesson: The maths: bias, variance and fixed points

Eligibility trace

$e(s) \leftarrow \gamma\ \lambda\ e(s)$, then $e(S_t) \leftarrow e(S_t) + 1$

Credit fades by $\gamma\lambda$ per step.

Lesson: The maths: bias, variance and fixed points

Batch TD answer

the values of the maximum likelihood Markov model of the data

Batch Monte Carlo gives the averages of the observed returns instead.

Lesson: The maths: bias, variance and fixed points

TABLES

Table: method · what it learns · update · speed

Method	What it learns	Update	Speed
Direct utility estimation	U(s) as an average	average of all rewards to go seen from s	slow: ignores links between states
Adaptive dynamic programming (ADP)	$\hat{P}(s' \mid s, \pi(s))$ and R(s)	solve $U(s) = R(s) + \gamma\ \Sigma\ \hat{P}\ U(s')$ after each step	fastest per trial, costly per step
Temporal difference (TD)	U(s) directly	$U(s) \leftarrow U(s) + \alpha\ (R(s) + \gamma\ U(s') - U(s))$	cheap per step, a bit slower than ADP

Lesson: Passive learning: judge a fixed policy without a model

Table: · direct estimation (monte carlo) · adp · td(0)

	Direct estimation (Monte Carlo)	ADP	TD(0)	TD(λ)
Needs a model	No	learns one	No	No
Work per step	tiny	solve a system of equations	one update	update all traced states
Data needed	most	least	medium	medium, often less than TD(0)
Uses the Markov links	No	Yes, fully	Yes, one step	Yes, mixed
Famous use	Monte Carlo evaluation	model based RL	TD-Gammon (Tesauro, 1995)	TD-Gammon also used TD(λ)

Lesson: Passive learners compared

TIPS AND TRAPS

REMEMBER

TD learning from experience is the same idea as the Bellman update, but it uses the one next state that actually happened instead of an average over all next states. Over many steps, states are reached in proportion to P(s' | s, a), so the averages come out right.

Lesson: Temporal difference learning

10.3 Active reinforcement learning

Exploration, Q-learning, SARSA, function approximation and deep Q-networks.

KEY IDEAS

- **Explore or exploit** ε-greedy, exploration function f(u, n) = R^+ if n < N_e else u, UCB1 $\bar{x} + \sqrt{(2 \ln n \div n_i)}$. GLIE.
- **Q-learning** $Q \leftarrow Q + \alpha [r + \gamma \max Q(s', a') - Q]$. Off policy, model free. Converges with every pair tried and Robbins Monro α.
- **SARSA** $Q \leftarrow Q + \alpha [r + \gamma Q(s', a') - Q]$ with the a' actually taken. On policy. Safer path on the cliff.
- **Approximation** $\theta \leftarrow \theta + \alpha [\text{target} - \hat{Q}] \nabla\hat{Q}$. DQN: replay buffer + target network. Deadly triad.
- **Key numbers** ε-greedy best action: 1 − ε + ε/|A|. Corridor Q*: 1, 0.9, 0.81. Cliff: Q-learning path 13 steps.

FORMULAS

ε-greedy

with probability ε a random action, otherwise argmax over a of Q(s, a)

Simple and widely used.

Lesson: Active learning: explore or exploit

Exploration function (AIMA) Exam favourite

f(u, n) = R^+ if n < N_e, else u

Optimistic about actions tried fewer than N_e times.

Lesson: Active learning: explore or exploit

UCB1

pick argmax over i of $\bar{x}_i + \sqrt{(2 \ln n \div n_i)}$

Mean so far plus a bonus for uncertainty. Its regret grows only like log n.

Lesson: Active learning: explore or exploit

GLIE

try every action infinitely often, and become greedy in the limit

A condition under which the learned policy becomes optimal.

Lesson: Active learning: explore or exploit

Q-learning update

Exam favourite

$$Q(s, a) \leftarrow Q(s, a) + \alpha\,[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

Off policy: the target uses the best next action.

Lesson: Q-learning

SARSA update

$$Q(s, a) \leftarrow Q(s, a) + \alpha\,[r + \gamma\, Q(s', a') - Q(s, a)]$$

On policy: a' is the action the agent really takes next.

Lesson: Q-learning

Greedy policy

$$\pi(s) = \operatorname{argmax}_a Q(s, a)$$

No model needed.

Lesson: Q-learning

Linear approximation

$$\hat{U}_\theta(s) = \theta_0 + \theta_1 f_1(s) + \ldots + \theta_n f_n(s)$$

f are features, such as the x and y of a square.

Lesson: Function approximation and deep Q-networks

Gradient TD update

Exam favourite

$$\theta_i \leftarrow \theta_i + \alpha\,[\text{target} - \hat{Q}(s, a)]\,\partial\hat{Q} \div \partial\theta_i$$

For a linear $\hat{Q}$ the derivative is just the feature value.

Lesson: Function approximation and deep Q-networks

Bellman optimality for Q

Exam favourite

$$Q^*(s, a) = \Sigma\, P(s' \mid s, a)\,[R(s, a, s') + \gamma \max_{a'} Q^*(s', a')]$$

Q-learning is a sampled version of this.

Lesson: The maths: the Bellman equation for Q and convergence

ε-greedy action chances

best action: $1 - \varepsilon + \varepsilon \div |A|$, each other action: $\varepsilon \div |A|$

The random choice can also pick the best action.

Lesson: The maths: the Bellman equation for Q and convergence

Convergence of Q-learning

every (s, a) tried infinitely often, $\Sigma\, \alpha_t = \infty$, $\Sigma\, \alpha_t^2 < \infty$

Watkins and Dayan, 1992, for tables.

Lesson: The maths: the Bellman equation for Q and convergence

UCB1 regret

expected regret after n pulls is $O(\log n)$

Lai and Robbins (1985) showed log n is the best possible order.

Lesson: The maths: the Bellman equation for Q and convergence

TABLES

Table: · q-learning · sarsa

	Q-learning	SARSA
Learns	Q* of the greedy policy (off policy)	Q of the ε-greedy policy it follows (on policy)
Cliff walking path	shortest, along the edge (13 steps)	safer, one or two rows up
Reward while learning	lower: it falls more while exploring	higher
Can learn from	any data, even another agent's	its own current policy only

Lesson: SARSA, on policy and off policy

Table: · greedy adp · adp + exploration function · q-learning

	Greedy ADP	ADP + exploration function	Q-learning	SARSA	DQN
Model	learned	learned	none	none	none
Exploration	none, gets stuck	optimistic values	ε-greedy or similar	ε-greedy or similar	ε-greedy
Learns	a policy that may be poor	near optimal policy	Q* (off policy)	Q of its own policy (on policy)	Q* with a neural network
Data use	best	best	one update per step	one update per step	replays old steps many times
Weak when	always	big state spaces (must solve the model)	huge state spaces without approximation	same as Q-learning	needs millions of frames, can be unstable

Lesson: Active learners compared

TIPS AND TRAPS

REMEMBER The same trade off appears in the expectimax chapter (6.3) in UCT, which uses UCB1 inside Monte Carlo tree search to pick which move to try next.

Lesson: Active learning: explore or exploit

COMMON MISTAKE With function approximation, Q-learning can diverge: off policy data, bootstrapping and approximation together (the deadly triad). DQN's replay buffer and target network were added to keep training stable.

Lesson: Function approximation and deep Q-networks

10.4 Policy search and policy gradients

Policy gradients, REINFORCE, baselines, actor critic and learning from experts.

KEY IDEAS

- **Policy search** Parameterise π_θ, maximise ρ(θ) = E[Σ γ^t R_t]. Use stochastic (softmax) policies so ρ is smooth.
- **Policy gradient** ∇ρ = E[∇ log π_θ(a | s) Q^π(s, a)]. Softmax: ∂ log π(a) ÷ ∂θ_k = 1[k = a] − π(k).
- **REINFORCE** θ ← θ + α (G − b) ∇ log π(a | s). Baseline unbiased since E[∇ log π] = 0.
- **Actor critic** δ = r + γV(s') − V(s). Actor θ += α δ ∇ log π, critic w += β δ ∇V. PPO clips the ratio.
- **From experts** Behaviour cloning copies actions. Inverse RL learns the reward (Ng and Russell 2000).

FORMULAS

Policy value Exam favourite

$$\rho(\theta) = E[\Sigma\ \gamma^t\ R_t \mid \text{actions from } \pi_\theta]$$

What policy search maximises.

Lesson: Policy search: tune the policy directly

Softmax policy

$$\pi_\theta(a \mid s) = e^{(\theta \cdot f(s, a))} \div \Sigma \text{ over b of } e^{(\theta \cdot f(s, b))}$$

Every action keeps some chance, so the policy explores.

Lesson: Policy search: tune the policy directly

Finite difference gradient

$$\partial\rho \div \partial\theta_i \approx (\rho(\theta + h\ e_i) - \rho(\theta)) \div h$$

Works for any policy, but needs many trials per parameter and is noisy.

Lesson: Policy search: tune the policy directly

Policy gradient theorem Exam favourite

$$\nabla\rho(\theta) = E[\nabla_\theta \log \pi_\theta(a \mid s)\ Q^\pi(s, a)]$$

Only the gradient of the policy appears.

Lesson: The policy gradient and REINFORCE

REINFORCE update

$$\theta \leftarrow \theta + \alpha\,(G_t - b)\,\nabla_\theta \log \pi_\theta(a_t \mid s_t)$$

b is a baseline that does not depend on the action.

Lesson: The policy gradient and REINFORCE

Softmax gradient

$$\partial \log \pi(a) \div \partial\theta_k = 1[k = a] - \pi(k)$$

For a softmax over preferences θ.

Lesson: The policy gradient and REINFORCE

Why a baseline is safe

$$E[\nabla \log \pi(a)] = \Sigma\, \pi(a)\, \nabla \log \pi(a) = \nabla\, \Sigma\, \pi(a) = \nabla\, 1 = 0$$

So subtracting b changes no expected value.

Lesson: The policy gradient and REINFORCE

Actor critic

Exam favourite

$$\delta = r + \gamma\, V(s') - V(s),\ \theta \leftarrow \theta + \alpha\, \delta\, \nabla \log \pi_\theta(a \mid s)$$

δ estimates the advantage of the action taken.

Lesson: Actor critic and other refinements

Advantage

$$A(s, a) = Q(s, a) - V(s)$$

How much better a is than the policy's average action.

Lesson: Actor critic and other refinements

PPO clipped objective

$$E[\min(r_t(\theta)\, \hat{A},\ \text{clip}(r_t(\theta),\ 1 - \varepsilon,\ 1 + \varepsilon)\, \hat{A})], \text{ with } r_t = \pi_\theta \div \pi_\text{old}$$

Keeps each policy update small and safe.

Lesson: Actor critic and other refinements

Log derivative trick

Exam favourite

$$\nabla\, \pi(a) = \pi(a)\, \nabla \log \pi(a)$$

Turns a gradient of a sum into an expectation.

Lesson: The maths: deriving the policy gradient

One step (bandit) case

$$\nabla \rho = \nabla\, \Sigma\, \pi(a)\, R(a) = \Sigma\, \pi(a)\, \nabla \log \pi(a)\, R(a) = E[\nabla \log \pi(a)\, R(a)]$$

Sample a, multiply the gradient of log π(a) by its reward.

Lesson: The maths: deriving the policy gradient

Exact bandit gradient for a softmax

$$\partial \rho \div \partial\theta_k = \pi(k)\, (\mu_k - \rho)$$

An action with above average reward gets pushed up.

Lesson: The maths: deriving the policy gradient

Variance with a baseline

$$\text{Var}[(G - b)\, \nabla \log \pi] \text{ is smallest near } b = E[G]$$

The usual choice is an estimate of V(s).

Lesson: The maths: deriving the policy gradient

TABLES

Table: · q-learning (value based) · reinforce · actor critic

	Q-learning (value based)	REINFORCE	Actor critic	Finite differences or evolution	Imitation and inverse RL
Learns	Q(s, a), acts greedily	π_θ directly	π_θ and V_w	π_θ by trying perturbed θ	π from an expert, or a reward
Actions	discrete (needs a max)	discrete or continuous	discrete or continuous	any	any
Variance	low	high	medium	high, unless correlated sampling	low (supervised)
Data efficiency	good (off policy, replay)	poor (on policy)	better	poor	good with an expert
Known for	DQN	simple, unbiased	A3C, PPO, SAC	PEGASUS, evolution strategies (Salimans and others, 2017)	self driving data, helicopters

Lesson: Value based and policy based methods compared

TIPS AND TRAPS

COMMON MISTAKE With a greedy policy (always the best action under some score) ρ(θ) jumps when the best action changes, so its gradient is zero almost everywhere. That is why policy gradient methods use stochastic policies.

Lesson: Policy search: tune the policy directly

STEP 11 Core AI · Challenge and revision

11.1 Challenge problems across Core AI

Hard exam style problems across every Core AI step, with proofs and a method guide.

KEY IDEAS

- **Five moves** Name the model, write the formula, check its conditions, work step by step, say what it means.
- **Search traps** Admissible is not enough for graph search without reopening. IDS costs about b ÷ (b − 1) times BFS.
- **CSPs and games** Tree CSP O(n d^2). Alpha beta best case $b^{\lceil d/2 \rceil} + b^{\lfloor d/2 \rfloor} - 1$. Expectimax needs real values.
- **Logic and planning** Resolution refutes KB ∧ ¬α. Occurs check. Regress: (goal − add) ∪ pre. Promotion or demotion.
- **Uncertainty and RL** Base rates. Observed descendant opens a collider. $\|BU - BU'\| \le \gamma \|U - U'\|$. Q += α (r + γ max Q' − Q).

TABLES

Table: move · ask yourself · example

Move	Ask yourself	Example
1. Name the model	What are the states, actions, variables or rules?	A route finder is a search problem with a step cost.
2. Write the formula	Which equation decides the answer?	f(n) = g(n) + h(n) for A*.
3. Check the conditions	Does the theorem's condition hold here?	Is h consistent, or only admissible?
4. Work it step by step	Can a reader follow each line?	Write the frontier after every expansion.
5. Say what it means	Is the answer sensible?	A probability must lie between 0 and 1.

Lesson: How to attack a hard Core AI problem

Table: problem looks like · use · why · watch out for

Problem looks like	Use	Why	Watch out for
Find a path, all step costs equal	BFS or IDS	Optimal for equal costs, IDS uses little memory	BFS memory grows as b^d
Find the cheapest path, a good estimate known	A* (IDA* or SMA* if memory is short)	Optimal with a consistent h, focused search	Inconsistent h in graph search
Actions may fail or have several outcomes	AND-OR search, AO*	Returns a conditional plan	Cycles need loops in the plan
Only the final state matters	Hill climbing, annealing, genetic algorithms	Little memory, works on huge spaces	Local maxima and plateaus
Assign values under constraints	Backtracking with MRV, forward checking, AC-3, or min-conflicts	Prunes whole sets of assignments	Variable order changes the time a lot
An opponent moves against you	Minimax with alpha beta, MCTS for huge trees	Assumes the opponent plays well	Evaluation function quality
Chance moves in a game	Expectimax, expectiminimax	Uses averages over chance	Needs real values, not only their order
Facts and rules, need a proof	Forward or backward chaining, resolution	Sound and complete for their logic	First order logic is only semi decidable
Reach a goal by a sequence of actions	STRIPS with forward or backward search, partial order planning	Factored states, reuse of heuristics	Sub-goal interaction
Uncertain facts	Bayesian networks, exact or sampling inference	Compact, answers any query	Exact inference is NP hard in general
Cause and effect, an intervention	Causal graphs and do-calculus	Separates seeing from doing	Hidden confounders
Decide over time, model known	Value or policy iteration	Exact optimal policy	Large state spaces
Decide over time, model unknown	TD learning, Q-learning, SARSA, policy gradients	Learn from experience	Exploration and slow learning

Lesson: Choosing the right method

TIPS AND TRAPS

REMEMBER Each challenge below names the chapter that teaches the idea, so you can go back if a step is not clear.

Lesson: How to attack a hard Core AI problem

IN THE EXAM The fix: use a consistent heuristic, or let graph search reopen a closed node when a cheaper path to it turns up. Tree search with an admissible heuristic is always optimal, because it never closes anything.

Lesson: Challenge set 1: search and heuristics

IN THE EXAM Forward chaining is complete for definite clauses only. Resolution is refutation complete for any clauses. A favourite exam trap gives a KB with a disjunction such as A ∨ B and asks for forward chaining.

Lesson: Challenge set 4: logic and inference

IN THE EXAM The answer P(B | j, m) ≈ 0.284 from chapter 9.3 is worth knowing by heart. Exams often give the burglary tables and ask for it, or for P(J | b) by summing out A.

Lesson: Challenge set 6: probability and Bayesian networks

11.2 Exam revision: from school to research universities

Core AI exam questions from school level to research universities, with worked answers, a formula sheet and a mock quiz.

FORMULAS

Rows of a truth table — Exam favourite

$$2^n$$

n symbols, each true or false.

Lesson: Tier 1: school and first year

Nodes at depth d — Exam favourite

$$b^d$$

Each node has b children.

Lesson: Tier 1: school and first year

Chance of an event

$$P(A) = \text{favourable cases} \div \text{all equally likely cases}$$

Only when every case is equally likely.

Lesson: Tier 1: school and first year

TABLES

Table: tier · typical format · what earns the marks · where to practise

Tier	Typical format	What earns the marks	Where to practise
School and first year	Short answers, small trees, simple chances	A clear definition with an example, careful counting	Lesson 2
University semester and IIT course exams	Run an algorithm by hand on a small graph, net or MDP	The frontier or table after every step, the answer stated with its meaning	Lesson 3
GATE and NPTEL style papers	Multiple choice, multiple select and numerical answer type	Speed, quick checks that rule out options, exact rounding	Lesson 4
MIT, Berkeley, Stanford, CMU and IISc style	Proofs, derivations and critique of a method	Conditions named, a complete argument, when it breaks	Lesson 5

Lesson: How Core AI is examined

Table: quick check · use it to

Quick check	Use it to
BFS and IDS are optimal only when every step costs the same	Reject "BFS is optimal" for unequal costs
DFS is not complete on infinite or cyclic spaces without a check	Reject "DFS always finds a solution"
The max of admissible heuristics is admissible, the sum usually is not	Settle multiple select questions on heuristics
Alpha beta never changes the root value	Reject options where pruning gives a new answer
A node is independent of its non descendants given its parents	Read independence straight off a net
$P(A \vee B) = P(A) + P(B) - P(A \wedge B)$	Avoid adding overlapping cases twice
Rejection sampling keeps about $N \times P(e)$ samples	Spot why it fails for rare evidence
Q-learning is off policy, SARSA is on policy	Answer the cliff walking question in one line

Lesson: Tier 3: GATE and NPTEL style papers

Table: topic · formula or fact

Topic	Formula or fact
BFS nodes, IDS nodes	$b + b^2 + \ldots + b^d$, $\Sigma\ (d + 1 - i)\ b^i$
Search memory	BFS $O(b^d)$, DFS $O(bm)$, IDS $O(bd)$
A*	$f(n) = g(n) + h(n)$, optimal with admissible h (tree) or consistent h (graph)
Consistency	$h(n) \le c(n, n') + h(n')$ and $h(\text{goal}) = 0$
Effective branching factor	$N + 1 = 1 + b^* + (b^*)^2 + \ldots + (b^*)^d$
Simulated annealing	accept a worse move with probability $e^{\wedge}(\Delta E \div T)$
Tree CSP	$O(n\ d^2)$
Alpha beta best case	$b^{\wedge}\lceil d \div 2\rceil + b^{\wedge}\lfloor d \div 2\rfloor - 1$ leaves
Resolution	from $A \vee \ell$ and $B \vee \neg\ell$ infer $A \vee B$, prove $KB \models \alpha$ by refuting $KB \wedge \neg\alpha$
STRIPS regression	new goal = (goal minus add list) plus preconditions
Bayes rule	$P(h \mid e) = P(e \mid h)\ P(h) \div P(e)$, $P(e) = \Sigma\ P(e \mid h)\ P(h)$
Bayesian network	$P(x_1, \ldots, x_n) = \Pi\ P(x_i \mid \text{parents}(X_i))$
Adjustment for a cause	$P(y \mid do(x)) = \Sigma$ over z of $P(y \mid x, z)\ P(z)$
Bellman optimality	$V(s) = \max$ over a of $\Sigma\ P(s' \mid s, a)\ (R + \gamma\ V(s'))$
TD and Q-learning	$V \leftarrow V + \alpha\ (r + \gamma\ V(s') - V)$, $Q \leftarrow Q + \alpha\ (r + \gamma \max Q(s', \cdot) - Q)$
REINFORCE	$\theta \leftarrow \theta + \alpha\ (G - b)\ \nabla \log \pi(a \mid s)$

Lesson: Formula sheet and last minute checks

TIPS AND TRAPS

IN THE EXAM Round only at the very end. A numerical answer type key often has a narrow range such as 0.69 to 0.70.

Lesson: Tier 3: GATE and NPTEL style papers

IN THE EXAM In proofs, name each condition when you use it: consistency, positive step costs, definite clauses, a finite domain, $\gamma < 1$. Examiners often ask what breaks when one fails.

Lesson: Tier 4: proofs, derivations and critique

STEP 1 Advanced AI · Rational choice

1.1 Decision problems and preferences

Decision problems, the three preference relations, completeness, transitivity, cycles and what carries over from weak to strict preference.

KEY IDEAS

- **Decision problem** $(A, X, c, \succeq)$: actions, outcomes, consequence map $c : A \to X$, preference. Utility goes on outcomes, $v(a) = u(c(a))$.
- **Three relations** Weak $x \succeq y$. Strict $x \succ y$ means $x \succeq y$ and not $y \succeq x$. Indifference $x \sim y$ means both directions.
- **Rationality** Complete (every pair compared) plus transitive (rankings chain). Cycles have no best element and allow a money pump.
- **What carries over** Transitivity of $\succeq$ passes to $\succ$ and $\sim$. Completeness does not pass to either.
- **Indifference is not incompleteness** Indifferent: equally good. Incomplete: cannot compare. Equality: the same object.

FORMULAS

A decision problem

Exam favourite

$$(A, X, c, \succeq)$$

A is the set of actions you can take. X is the set of outcomes that can happen. c is the map that says which outcome each action leads to. ≥ is your preference over outcomes.

Lesson: What a decision problem is

The consequence map

$$c : A \to X$$

With certainty, each action leads to exactly one outcome. Later chapters let an action lead to a lottery over outcomes.

Lesson: What a decision problem is

Action value

$$v(a) = u(c(a))$$

We like outcomes, not actions. An action is worth whatever its outcome is worth.

Lesson: What a decision problem is

Weak preference

Exam favourite

$$x \succeq y$$

x is at least as good as y. It allows a tie.

Lesson: Weak preference, strict preference and indifference

Strict preference

Exam favourite

$$x \succ y \iff x \succeq y \text{ and not } y \succeq x$$

x is better and the reverse does not hold.

Lesson: Weak preference, strict preference and indifference

Indifference

Exam favourite

$$x \sim y \iff x \succeq y \text{ and } y \succeq x$$

Both directions hold, so the two are equally good.

Lesson: Weak preference, strict preference and indifference

Completeness

Exam favourite

$$\text{for all } x, y \in X: \quad x \succeq y \text{ or } y \succeq x$$

You can always compare. You may say better, worse or equally good, but you may not leave a pair unanswered.

Lesson: Completeness and transitivity

Transitivity

Exam favourite

$$x \succeq y \text{ and } y \succeq z \implies x \succeq z$$

Rankings chain together without contradiction.

Lesson: Completeness and transitivity

Rational preference

$$\text{rational} = \text{complete} + \text{transitive}$$

A rational agent has such a preference and picks an action whose outcome it likes most.

Lesson: Completeness and transitivity

TABLES

Table: part · everyday example · ai example

Part	Everyday example	AI example
Actions A	Go to INOX or go to PVR	Move left, right, up or down
Outcomes X	The film you end up watching	The square the robot reaches
Consequence c	INOX shows Aliens	Moving right from (1, 1) reaches (2, 1)
Preference ≥	Aliens is better than Casablanca	The charging dock is better than the stairs

Lesson: What a decision problem is

Table: relation · pairs it contains

Relation	Pairs it contains
≥ weak	(A, A), (B, B), (C, C), (A, B), (A, C), (B, C), (C, B)
≻ strict	(A, B), (A, C)
~ indifference	(A, A), (B, B), (C, C), (B, C), (C, B)

Lesson: Weak preference, strict preference and indifference

Table: does it carry over from ≥? · to ≻ · to ~

Does it carry over from ≥?	To ≻	To ~
Transitivity	Yes, proof above	Yes, proof above
Completeness	No. If x ~ y then neither x ≻ y nor y ≻ x	No. If x ≻ y then x ~ y is false and so is y ~ x
Reflexivity	No. x ≻ x never holds	Yes. x ≥ x in both directions

Lesson: Completeness and transitivity

TIPS AND TRAPS

COMMON MISTAKE Equality and indifference are different ideas. Equality means two things are the same object. Indifference means two different outcomes are judged equally good. Tea and coffee are not equal, but you may be indifferent between them.

Lesson: Weak preference, strict preference and indifference

COMMON MISTAKE Completeness does not mean you like every option. It means you can compare any two, possibly by saying you are indifferent.

Lesson: From intuition to research

1.2 Utility functions

Utility representation, ordinal invariance, the existence theorem with its construction proof, and counting rankings.

KEY IDEAS

- **Representation** x ≥ y ⟺ u(x) ≥ u(y). Strict gives >, indifference gives =. Not "=" for weak preference.
- **Ordinal** Only order matters. Any strictly increasing f gives f(u), another valid utility. Gaps and ratios mean nothing.
- **Existence** Finite X + complete + transitive ⟹ a utility exists. Build it by peeling off top classes: A ≻ B ~ C ≻ D gives 3, 2, 2, 1.
- **Best exists** In finite X an endless chain of improvements must cycle, which transitivity forbids. Ties at the top are allowed.
- **Counting** n strict outcomes give n! rankings (5! = 120). With ties: 1, 3, 13, 75 weak orders.

FORMULAS

Utility function

$$u : X \to \mathbb{R}$$

A score for each outcome. It may stand for satisfaction, value, benefit or happiness. It is not money unless the model says so.

Lesson: A number for every outcome

Representation Exam favourite

$$x \succeq y \iff u(x) \geq u(y)$$

u represents ≥ when the order of the numbers is exactly the order of the preference.

Lesson: A number for every outcome

The two consequences

$$x \succ y \iff u(x) > u(y), \quad x \sim y \iff u(x) = u(y)$$

Strict preference matches a strictly larger number, and a tie matches equal numbers.

Lesson: A number for every outcome

Ordinal invariance Exam favourite

if f is strictly increasing, $v(x) = f(u(x))$ represents the same $\succeq$

Examples: $2u + 7$, u^3, e^u, and $\log u$ when u is positive. A decreasing f such as $-u$ reverses the ranking.

Lesson: Ordinal and cardinal utility

Cardinal utility

$u(A) - u(B)$ compared with $u(C) - u(D)$

Cardinal utility treats the size of a difference as meaningful. It needs more assumptions than ranking alone, and it returns in chapter 2.2 with lotteries.

Lesson: Ordinal and cardinal utility

Existence theorem Exam favourite

X finite and $\succeq$ complete and transitive $\Longrightarrow$ some $u : X \to \mathbb{R}$ represents $\succeq$

Three conditions: finitely many outcomes, completeness, transitivity.

Lesson: When does a utility function exist?

TABLES

Table: statement about u(a) = 10, u(b) = 5, u(c) = 1 · ordinal meaning · cardinal meaning

Statement about u(A) = 10, u(B) = 5, u(C) = 1	Ordinal meaning	Cardinal meaning
A is preferred to B	Yes, meaningful	Yes, meaningful
A is twice as good as B	Not meaningful	Not meaningful either. Ratios change with 2u + 7
The step from B to A is bigger than from C to B	Not meaningful	Meaningful only when u is cardinal

Lesson: Ordinal and cardinal utility

Table: condition dropped · what goes wrong

Condition dropped	What goes wrong
Transitivity	A ≻ B ≻ C ≻ A would need u(A) > u(B) > u(C) > u(A), so u(A) > u(A). Impossible.
Completeness	Real numbers are always comparable, so any u would rank pairs the agent cannot rank.
Finiteness	The construction may never end. Lexicographic preference on the plane has no utility at all.

Lesson: When does a utility function exist?

TIPS AND TRAPS

COMMON MISTAKE The slide transcript once read "u(x) = u(y) ⟺ x ≥ y". That is a transcription slip. The correct condition for weak preference uses ≥. Equality belongs to indifference only.

Lesson: A number for every outcome

COMMON MISTAKE With ordinal utility, differences mean nothing. u(A) − u(B) = 2 does not say A is liked twice as much as some other pair with difference 1.

Lesson: From intuition to research

1.3 Choosing the best action

Rational choice as arg max, decision trees, the movie, fruit tree and city park problems, and constrained optimization on an interval.

KEY IDEAS

- **Rational choice** v(a) = u(c(a)). a* ∈ arg max v, i.e. v(a*) ≥ v(a) for all a. Ties are all rational.
- **Movie tree** Utilities 6 to 1, minus 1 per mile. INOX 1 mile, PVR 3. Best: INOX Aliens, net 5.

- **Fruit trees 1.6** Plain: AA at X − 60. Second same tree saves half: AA, OO become X + 5, PP X + 22.5, best AP at X − 55.
- **Parks 1.7** v = √(400c) − c/80 gives √c = 800, c* = 640,000 under the 1,000,000 cap. With √(1600c) the peak 2,560,000 is infeasible, so c* = 1,000,000.
- **Other ideas** Satisficing picks the first v ≥ T. Minimizing loss = maximizing −loss. Check feasibility and boundaries.

FORMULAS

Value of an action

Exam favourite

$$v(a) = u(\,c(a)\,)$$

First find the consequence, then score it.

Lesson: From preferences to the best action

Rational choice

Exam favourite

$$a^* \in \arg\max_{a \in A} v(a) \iff v(a^*) \geq v(a) \text{ for every } a$$

arg max is a set. If two actions tie for the top, both are rational.

Lesson: From preferences to the best action

Loss as negative utility

$$\min\,(y - \hat{y})^2 \iff \max\,-(y - \hat{y})^2$$

Minimizing a loss and maximizing its negative pick the same answer.

Lesson: From preferences to the best action

Spending after a plan

Exam favourite

$$u(\text{plan}) = X + \Sigma\ \text{maintenance} - \Sigma\ \text{reduction}$$

X is what you spend with no trees. A smaller number is better.

Lesson: Problem 1.6: the fruit trees

Variety version

second tree of the same kind saves only half

Part (d) of the problem. Mixed pairs keep the full saving.

Lesson: Problem 1.6: the fruit trees

Benefit

Exam favourite

$$v(c) = \sqrt{400c} - \frac{c}{80} = 20\sqrt{c} - \frac{c}{80}$$

The action set is the interval A = [0, 1,000,000]. It is continuous, so we cannot list the actions, we must use calculus.

Lesson: Problem 1.7: how much to spend on parks

First order condition

Exam favourite

$$v'(c) = \frac{10}{\sqrt{c}} - \frac{1}{80} = 0$$

Marginal benefit equals marginal cost.

Lesson: Problem 1.7: how much to spend on parks

Second order condition

$$v''(c) = -5\,c^{-3/2} < 0$$

v is strictly concave, so a stationary point inside the interval is the unique maximum.

Lesson: Problem 1.7: how much to spend on parks

TABLES

Table: · maximizing · satisficing

	Maximizing	Satisficing
Rule	Pick an action in arg max v	Pick any action with v(a) ≥ T, an aspiration level
Search	Must compare everything	Can stop at the first good enough action
Used	Throughout this course	Herbert Simon's model of real human choice under limited time

Lesson: From preferences to the best action

Table: tree · maintenance · food bill reduction · net change per tree

Tree	Maintenance	Food bill reduction	Net change per tree
Apple	$100	$130	−30
Orange	$70	$90	−20
Pear	$120	$145	−25

Lesson: Problem 1.6: the fruit trees

TIPS AND TRAPS

COMMON MISTAKE The transcript sometimes wrote v(a*) > v(a) for every other a. That strict version only fits when the best action is unique. With ties the right condition is ≥.

Lesson: From preferences to the best action

REMEMBER A change in the payoff structure changed the rational choice. Nothing about the person's rationality changed, only the numbers on the leaves.

Lesson: Problem 1.6: the fruit trees

COMMON MISTAKE Setting the derivative to zero is not enough. Check the second derivative and the end points of the allowed range.

Lesson: From intuition to research

STEP 2 Advanced AI · Choice under risk

2.1 Lotteries and distributions

Lotteries, the CDF and its properties, intervals, densities and continuous outcomes.

KEY IDEAS

- **Lottery** Action a gives $P(x_i \mid a) = p_i$ with p_i in [0, 1] and $\Sigma\, p_i = 1$. Nature picks the outcome.
- **CDF** $F(x) = P(X \le x \mid a)$. Non decreasing, 0 at $-\infty$, 1 at $+\infty$, right continuous. F(50) = 0.7 is "at most 50".
- **Intervals** $P(r < X \le s) = F(s) - F(r)$. Notes example 0.8 − 0.3 = 0.5.
- **Density** $F = \int f$, $f = F'$. $P(X = x) = 0$ for continuous X. Densities can exceed 1.
- **Compound paths** Multiply along each path, add the paths that reach the same outcome: 0.6 × 0.8 + 0.4 × 0.3 = 0.6.

FORMULAS

Lottery of an action Exam favourite

$$P(\, x_i \mid a \,) = p_i, \quad 0 \le p_i \le 1, \quad \sum p_i = 1$$

A simple lottery lists every final outcome with its probability. The probabilities add to 1 because exactly one outcome happens.

Lesson: When an action has more than one outcome

Deterministic as a special case

$$P(\, c(a) \mid a \,) = 1$$

A certain action is a lottery that puts all its probability on one outcome.

Lesson: When an action has more than one outcome

CDF of an action Exam favourite

$$F_a(x) = P(\, X \le x \mid a \,)$$

The probability that the outcome is at most x when you take action a.

Lesson: The cumulative distribution function

Interval probability Exam favourite

$$P(\, r < X \le s \mid a \,) = F_a(s) - F_a(r)$$

Everything up to s minus everything up to r.

Lesson: The cumulative distribution function

From density to CDF Exam favourite

$$F_a(x) = \int_{-\infty}^{x} f_a(t)\,\mathrm{d}t$$

Add up the density from the far left up to x.

Lesson: Densities and continuous outcomes

From CDF to density

$$f_a(x) = \frac{\mathrm{d}}{\mathrm{d}x} F_a(x)$$

Wherever the derivative exists.

Lesson: Densities and continuous outcomes

Single points

$$P(X = x) = 0 \quad \text{for a continuous } X$$

Only intervals carry probability.

Lesson: Densities and continuous outcomes

TABLES

Table: · deterministic choice · risky choice

	Deterministic choice	Risky choice
Action leads to	One known outcome	A distribution of outcomes
Compare	u(c(a))	E[u(X) | a]
Tree contains	Decision nodes only	Decision nodes and chance nodes
Ranking enough?	Yes, ordinal utility works	No, cardinal utility is needed (chapter 2.2)

Lesson: When an action has more than one outcome

Table: property · formula · plain meaning

Property	Formula	Plain meaning
Non decreasing	$x < y \Longrightarrow F(x) \le F(y)$	More room below means at least as much probability
Lower limit	$F(x) \to 0$ as $x \to -\infty$	Nothing lies below everything
Upper limit	$F(x) \to 1$ as $x \to +\infty$	Everything lies below infinity
Right continuous	$F(x) = \lim F(x + h)$ as $h \downarrow 0$	At a jump the filled dot sits on the upper step

Lesson: The cumulative distribution function

Table: lottery · cdf f(x) · shape

Lottery	CDF F(x)	Shape
Discrete, outcomes x_i with p_i	Sum of p_i with $x_i \le x$	A staircase with jumps of size p_i
Uniform on [L, U]	$(x - L) \div (U - L)$ inside	A straight ramp from 0 to 1
Exponential with rate λ	$1 - e^{-\lambda x}$ for $x \ge 0$	Rises fast then flattens
Normal with mean μ and spread σ	$\Phi((x - \mu) \div \sigma)$	An S curve centred on μ

Lesson: Densities and continuous outcomes

TIPS AND TRAPS

COMMON MISTAKE F(50) = 0.7 means a 70% chance the outcome is at most 50. It does not mean a 70% chance the outcome is exactly 50. This misreading is the most common exam error on CDFs.

Lesson: The cumulative distribution function

COMMON MISTAKE A density can be larger than 1. The uniform density on [0, 0.5] equals 2. Only the area under it must be 1.

Lesson: From intuition to research

2.2 Expected utility and compound lotteries

Expected utility, reducing compound lotteries, the R&D example, the reversal under non affine changes and why utility under risk is cardinal.

KEY IDEAS

- **Expected utility** EU(a) = Σ P(x_i | a) u(x_i), or ∫ u dF. Choose arg max EU, not arg max E[X].
- **R&D compound lottery** P(10 | G) = 0.625 × 0.9 + 0.375 × 0.5 = 0.75, EU(G) = 7.5 > EU(S) = 5. Multiply along paths, add paths.
- **Reversal** u = 9, −1, 10, 0: EU(G) 6.5 > 5. u(B) = −8: 4.75 < 5. Same ranking, opposite choice.
- **Affine invariance** Only v = a u + b with a > 0 keeps every EU comparison. Utility under risk is cardinal.
- **Other rules and PV** Optimistic (P best), pessimistic (P worst). PV = FV ÷ $(1 + r)^t$. vNM axioms give EU.

FORMULAS

Expected utility, discrete — Exam favourite

$$EU(a) = \sum_i P(x_i \mid a)\, u(x_i)$$

Probability times utility, summed over outcomes.

Lesson: Expected utility

Expected utility, continuous

$$EU(a) = \int u(x)\, \mathrm{d}F_a(x) = \int u(x)\, f_a(x)\, \mathrm{d}x$$

The second form needs a density.

Lesson: Expected utility

Rational choice under risk — Exam favourite

$$a^* \in \arg\max_a \mathrm{E}[\, u(X) \mid a \,]$$

Maximize expected utility, not expected money.

Lesson: Expected utility

Reduction of compound lotteries — Exam favourite

$$P(\text{final outcome}) = \sum_{\text{paths}} \prod_{\text{branches on the path}} P(\text{branch})$$

Multiply along each path, then add the paths that end at the same outcome.

Lesson: Compound lotteries: the R&D example

Positive affine transformation — Exam favourite

$$v(x) = a\, u(x) + b, \quad a > 0$$

The only transformations that keep every expected utility comparison.

Lesson: Why ranking is not enough under risk

Why it works

$$\mathrm{E}[\, a\, u(X) + b \,] = a\, \mathrm{E}[\, u(X) \,] + b$$

Expectation is linear. Scaling by a positive a and shifting by b keeps the order of the expected values.

Lesson: Why ranking is not enough under risk

Present value

$$\mathrm{PV} = \frac{\mathrm{FV}}{(1 + r)^t}$$

For a risky payoff: E[$u(X_t)$] ÷ $(1 + r)^t$.

Lesson: Why ranking is not enough under risk

TABLES

Table: rule · looks at · r&d answer

Rule	Looks at	R&D answer
Expected utility	Σ p u	G: 7.5 against 5
Optimistic	Chance of the best outcome	G: 0.75 against 0.5
Pessimistic	Chance of the worst outcome, lower is better	G: 0.25 against 0.5
Others	Median, quantiles, worst case, variance, chance of beating a target	Depends on the rule

Lesson: Compound lotteries: the R&D example

TIPS AND TRAPS

IN THE EXAM Moving u(B) from −1 to −8 is not affine: it stretches one gap and leaves the others alone. Under risk the sizes of the gaps matter, so expected utility needs cardinal utility, unique up to a u + b with a > 0.

Lesson: Why ranking is not enough under risk

COMMON MISTAKE Expected value and expected utility differ whenever u is not a straight line. Write which one a question asks for.

Lesson: From intuition to research

2.3 Risk attitudes and the St Petersburg paradox

Risk attitudes from concavity, certainty equivalent and risk premium, and the St Petersburg paradox with its log utility resolution.

KEY IDEAS

- **Three attitudes** Concave u (ln, √): averse, E[u] ≤ u(E[X]). Linear: neutral. Convex (x²): loving. Jensen with the chord picture.
- **Certainty equivalent** u(CE) = E[u(X)]. Risk premium = E[X] − CE. √x with 100 at 0.2: EU = 2, CE = 4, sure 30 wins.
- **Fair gamble** +1 or −1 at 50/50 has mean 0. Play when 0.5u(1) + 0.5u(−1) > u(0). Averse agents refuse.
- **St Petersburg** Pays 2^{n-1} w.p. $1/2^n$. E[X] = Σ ½ = ∞. Log utility: E = ln 2, CE = 2. x² still diverges.
- **Practice 5** u = x², 0 or 10: E[X] = 5, u(E[X]) = 25, E[u] = 50, so risk loving.

FORMULAS

Risk neutral Exam favourite

$$\mathrm{E}[\,u(X)\,] = u(\,\mathrm{E}[X]\,)$$

u is linear, u(x) = ax + b. Only expected money matters.

Lesson: Three attitudes to risk

Risk averse Exam favourite

$$\mathrm{E}[\,u(X)\,] \le u(\,\mathrm{E}[X]\,)$$

u is concave (u" < 0), for example ln x or √x. The sure amount is preferred.

Lesson: Three attitudes to risk

Risk loving Exam favourite

$$\mathrm{E}[\,u(X)\,] \ge u(\,\mathrm{E}[X]\,)$$

u is convex (u" > 0), for example x². The gamble is preferred.

Lesson: Three attitudes to risk

Certainty equivalent Exam favourite

$$u(\mathrm{CE}) = \mathrm{E}[\,u(X)\,] \implies \mathrm{CE} = u^{-1}(\,\mathrm{E}[u(X)]\,)$$

The sure amount you value exactly as much as the lottery.

Lesson: The certainty equivalent

Risk premium

$$RP = E[X] - CE$$

What you would give up in expectation to remove the risk. Positive for a risk averse agent.

Lesson: The certainty equivalent

Ordering

$$\text{averse: } CE \le E[X], \quad \text{neutral: } CE = E[X], \quad \text{loving: } CE \ge E[X]$$

Lesson: The certainty equivalent

Expected money Exam favourite

$$E[X] = \sum_{n=1}^{\infty} \frac{1}{2^n} \cdot 2^{n-1} = \sum \frac{1}{2} = \infty$$

Every toss adds exactly one half to the sum, forever.

Lesson: The St Petersburg paradox

Expected log utility Exam favourite

$$E[\ln X] = \sum \frac{(n-1)\ln 2}{2^n} = \ln 2$$

Since $\sum (n-1) \div 2^n = 1$. Finite.

Lesson: The St Petersburg paradox

Its certainty equivalent

$$CE = e^{\ln 2} = 2$$

A log utility agent would pay only about 2.

Lesson: The St Petersburg paradox

TABLES

Table: utility · u" · attitude · ce of 0 or 100 at 50/50

Utility	u"	Attitude	CE of 0 or 100 at 50/50
x	0	Neutral	50
√x	$-\frac{1}{4}\, x^{-3/2} < 0$	Averse	25
ln(1 + x)	$-1 \div (1+x)^2 < 0$	Averse	9.05
x^2	$2 > 0$	Loving	70.71

Lesson: The certainty equivalent

TIPS AND TRAPS

IN THE EXAM Concavity is what tames the paradox. With the convex u = x² the terms are 2^{n-2} and the sum still diverges. Even √x converges, to about 1.707, which gives a certainty equivalent near 2.91.

Lesson: The St Petersburg paradox

COMMON MISTAKE A risk averse person may still gamble. They need the gamble to pay enough more than the sure amount, the risk premium.

Lesson: From intuition to research

STEP 3 Advanced AI · Decisions over time

3.1 Backward induction

Backward induction on the R&D and marketing tree, full strategies, the film studio problem and Bellman's principle.

KEY IDEAS

- **Method** From the last decision back: chance node = Σ p × value, decision node = max. Never compare raw leaves at a chance node.
- **R&D tree** Success: m 11 > d 8. Failure: m 3 < d 4. Go = 0.625 × 11 + 0.375 × 4 = 8.375 > skip 5.
- **Full strategies** MM 8.0, MD 8.375, DM 6.125, DD 6.5. Same answer as folding, but 2^k plans for k binary nodes.

- **Practice A** Good buzz push 14.4, weak buzz hold 3.7, big budget 7.98 > small 3.5.
- **Why it works** Bellman's principle of optimality: each tail of an optimal plan is optimal. Folding is linear in the tree size.

FORMULAS

Backward induction Exam favourite

start at the last decision, replace each chance node by its expected value, keep the best action at each decision node, and move one level back

Repeat until you reach the root.

Lesson: Decisions that unfold over time

At a chance node

$$V(n) = \sum_k p_k \, V(\text{child}_k)$$

Nature moves, so you average.

Lesson: Decisions that unfold over time

At a decision node

$$V(n) = \max_a V(\text{child}_a)$$

You move, so you take the best.

Lesson: Decisions that unfold over time

TABLES

Table: path · then · payoffs

Path	Then	Payoffs
Skip research (s)	Nature picks the market	10 or 0, each with 0.5
Go for research (g), success with 0.625	Choose marketing m or not d, then the market is good with 0.9	m: 13 or −7, d: 9 or −1
Go for research (g), failure with 0.375	Choose m or d, then the market is good with 0.5	m: 13 or −7, d: 9 or −1

Lesson: Decisions that unfold over time

Table: after success, after failure · expected payoff

After success, after failure	Expected payoff
M, M (always market)	0.625 × 11 + 0.375 × 3 = 8.000
M, D (market only after success)	0.625 × 11 + 0.375 × 4 = 8.375, the best
D, M (market only after failure)	0.625 × 8 + 0.375 × 3 = 6.125
D, D (never market)	0.625 × 8 + 0.375 × 4 = 6.500

Lesson: Solving the R&D and marketing tree

Table: buzz · push: success chance, payoffs · no push: payoffs

Buzz	Push: success chance, payoffs	No push: payoffs
Good (0.4)	0.8: 18 or 0	0.8: 14 or 2
Weak (0.6)	0.3: 18 or −4	0.3: 10 or 1

Lesson: Practice problem A: the film studio

TIPS AND TRAPS

COMMON MISTAKE Never compare raw leaf payoffs where nature moves. 13 is not "better than" −7 in any useful sense at a chance node, because the firm does not choose between them. Convert every chance node to its expected value first, and compare only at nodes where the firm chooses.

Lesson: Solving the R&D and marketing tree

COMMON MISTAKE At chance nodes average with the probabilities. Taking the maximum there assumes luck always goes your way.

Lesson: From intuition to research

3.2 Discounting and consumption over time

Discount factors and present value, optimal consumption with ln and √x, corner solutions, and the time consistency of exponential discounting.

KEY IDEAS

- **Discount factor** $\delta = 1/(1 + r)$, $0 < \delta < 1$, patience not a choice. A after n periods is worth $A\delta^n$. Practice B: PV = 520.2.
- **Timing** Discount each piece by when it happens: R&D branch $10\delta - 1$, not $(10 - 1)\delta$.
- **ln consumption** 2 periods $K/(1 + \delta)$, $\delta K/(1 + \delta)$. 3 periods K, δK, $\delta^2 K$ over $1 + \delta + \delta^2$. Equal discounted marginal utility.
- **Corners** Linear u: (K, 0, 0). Convex $u = x^2$: FOC finds the minimum, best is a corner. √x: $x_1 = K/(1 + \delta^2)$, K = 181 gives 100, 81.
- **Time consistency** Exponential: re-planning keeps x_2^*, x_3^*. Hyperbolic $1/(1 + kt)$ is time inconsistent.

FORMULAS

Discount factor — Exam favourite

$$\delta = \frac{1}{1 + r}, \quad 0 < \delta < 1$$

A rupee next period is worth δ rupees today.

Lesson: Why money later is worth less

n periods ahead — Exam favourite

$$\text{value today of } A \text{ received after } n \text{ periods} = A\,\delta^n = \frac{A}{(1 + r)^n}$$

Two periods: 100 grows to $100(1 + r)^2$, so it is worth $100\delta^2$ today.

Lesson: Why money later is worth less

Present value of a stream

$$\text{PV} = \sum_t x_t\,\delta^t$$

Discount each payment by its own delay, then add.

Lesson: Why money later is worth less

Two periods, u = ln x — Exam favourite

$$\max \ln x_1 + \delta \ln(K - x_1) \implies x_1^* = \frac{K}{1 + \delta}, \quad x_2^* = \frac{\delta K}{1 + \delta}$$

From $1/x_1 = \delta/(K - x_1)$.

Lesson: Spreading consumption over time

Three periods, u = ln x — Exam favourite

$$x_1^* = \frac{K}{1 + \delta + \delta^2}, \quad x_2^* = \frac{\delta K}{1 + \delta + \delta^2}, \quad x_3^* = \frac{\delta^2 K}{1 + \delta + \delta^2}$$

The board solution from lecture 4.

Lesson: Spreading consumption over time

Equal discounted marginal utility

$$1/x_1^* = \delta/x_2^* = \delta^2/x_3^*$$

No small move of money between two periods can raise the total.

Lesson: Spreading consumption over time

Linear utility

$$x^* = (K, 0, 0)$$

A linear objective over a budget is maximized at a corner.

Lesson: Corners, homework and time consistency

TABLES

Table: discounting · weight on period t · re-planning later

Discounting	Weight on period t	Re-planning later
Exponential	δ^t	Keeps the original plan (time consistent)
Hyperbolic	1/(1 + kt)	Wants to change it (time inconsistent): impatience is strongest about the near future

Lesson: Corners, homework and time consistency

TIPS AND TRAPS

REMEMBER Diminishing marginal utility means u" < 0, not u' falling to zero. Going from 10 to 11 rupees adds more than going from 110 to 111, but both add something. The first order condition sets the derivative of the whole objective to zero, never u' itself.

Lesson: Spreading consumption over time

COMMON MISTAKE The discount rate r and the discount factor δ = 1 ÷ (1 + r) are different numbers. A higher rate means a lower factor.

Lesson: From intuition to research

3.3 The value of information

The value of perfect and imperfect information, the MBA and venue examples, and why information is never worth less than zero.

KEY IDEAS

- **VOI formula** VOI = E[max_a u(a, s)] − max_a E[u(a, s)]. Choose per state then average, minus average per action then choose.
- **MBA** Cost 10, states 0.25/0.5/0.25. No info: 9 (MBA) vs 8. Oracle: 22, 8, 4 gives 10.5. VOI = 1.5.
- **Never negative** You can ignore the oracle, so VOI ≥ 0. It is 0 when one action is best in every state.
- **Venue practice C** Outdoor 50/−20, indoor 30/25 at 0.5 each. 27.5 without, 37.5 with, VOI = 10. 90% forecast: 6.75.
- **Imperfect signals** Update by Bayes, pick the best per signal, average. Worth 0 if it cannot change the decision (q = 0.5).

FORMULAS

Value of perfect information Exam favourite

$$\mathrm{VOI} = \mathrm{E}[\ \max_a u(a, s)\ ] - \max_a \mathrm{E}[\ u(a, s)\ ]$$

Best with the oracle (choose per state, then average) minus best without (average per action, then choose).

Lesson: What is it worth to know first?

MBA example Exam favourite

$$\mathrm{VOI} = 10.5 - 9 = 1.5$$

Pay an oracle anything below 1.5. Above 1.5 you are better off deciding now.

Lesson: What is it worth to know first?

Bayes update Exam favourite

$$P(s \mid \text{signal}) = \frac{P(\text{signal} \mid s)\, P(s)}{P(\text{signal})}$$

The forecast shifts the probabilities, not the payoffs.

Lesson: Imperfect information and Bayes

Value of imperfect information

$$\text{VOI} = \sum_{\text{signals}} P(\text{signal}) \max_a \mathrm{E}[u(a, s) \mid \text{signal}] - \max_a \mathrm{E}[u(a, s)]$$

Between 0 and the value of perfect information.

Lesson: Imperfect information and Bayes

TABLES

Table: market (probability) · with mba, gross · with mba, net of 10 · without mba

Market (probability)	With MBA, gross	With MBA, net of 10	Without MBA
Good (0.25)	32	22	12
Medium (0.5)	16	6	8
Poor (0.25)	12	2	4

Lesson: What is it worth to know first?

TIPS AND TRAPS

COMMON MISTAKE A student in class computed 0.25 × 22 + 0.5 × 8 + 0.25 × 4 as the no information value. That mixes actions across states, which you can only do once you know the state. Without information you must use one action in every state.

Lesson: What is it worth to know first?

COMMON MISTAKE If every forecast would lead to the same choice, the forecast is worth 0 to you, however accurate it is.

Lesson: From intuition to research

STEP 4 Advanced AI · Games between rational agents

4.1 Static games and the normal form

Static games of complete information, common knowledge, the normal form, the Prisoner's Dilemma and best responses.

KEY IDEAS

- **Two axes** Complete vs incomplete information. Static (no one sees others' choices) vs dynamic. Static is about observation, not clock time.
- **Common knowledge** Everyone knows, everyone knows everyone knows, forever. Complete info: actions, outcomes, map, preferences are common knowledge.
- **Normal form** $\langle N, S_i, v_i \rangle$ with v_i on full profiles. Finite if all S_i finite. Bimatrix cell = (row payoff, column payoff).
- **Prisoner's Dilemma** (M,M) = (−2,−2), (M,C) = (−5,−1), (C,M) = (−1,−5), (C,C) = (−4,−4). C is the best reply to both.
- **Best response** $BR_i(s_{-i})$ maximizes $v_i(\cdot, s_{-i})$. A profile with no profitable unilateral deviation is self enforcing.

FORMULAS

Common knowledge

Exam favourite

$$E \text{ is common knowledge} \iff (\text{everyone knows})^n\, E \quad \text{for every } n = 1, 2, 3, \ldots$$

Knowing a fact privately is mutual knowledge. Common knowledge needs the whole infinite chain.

Lesson: Many agents: static games of complete information

Normal form game

Exam favourite

$$\langle N, \{\{S_i\}\}, \{\{v_i\}\} \rangle$$

N = {1, ..., n} players. S_i = pure strategies of player i. v_i = payoff of player i.

Lesson: The normal form of a game

Strategy profile

$$s = (s_1, \ldots, s_n) \in S_1 \times \ldots \times S_n$$

One choice per player. In a static game a strategy is simply an action.

Lesson: The normal form of a game

Payoff function

Exam favourite

$$v_i : S_1 \times \ldots \times S_n \to \mathbb{R}$$

Your payoff depends on the whole profile, not only on your own choice.

Lesson: The normal form of a game

Everyone else

$$s_{-i} = (s_1, \ldots, s_{i-1}, s_{i+1}, \ldots, s_n)$$

Read "s minus i". We write $s = (s_i, s_{-i})$.

Lesson: The normal form of a game

Best response

Exam favourite

$$s_i \in \mathrm{BR}_i(s_{-i}) \iff v_i(s_i, s_{-i}) \geq v_i(s'_i, s_{-i}) \text{ for all } s'_i$$

Hold everyone else fixed and maximize your own payoff.

Lesson: Best responses and unilateral deviation

Unilateral deviation

$$\text{from } s \text{ to } (s'_i, s_{-i})$$

Only player i changes. A profile where no unilateral deviation helps anyone is self enforcing.

Lesson: Best responses and unilateral deviation

TABLES

Table: axis · one end · other end

Axis	One end	Other end
What do players know about each other?	Complete information	Incomplete information
When do players act?	Static: choose without seeing others' choices	Dynamic: some players see earlier moves

Lesson: Many agents: static games of complete information

Table: player 1 (rows), player 2 (columns) · m · c

Player 1 (rows), Player 2 (columns)	M	C
M	(−2, −2)	(−5, −1)
C	(−1, −5)	(−4, −4)

Lesson: The normal form of a game

TIPS AND TRAPS

REMEMBER

A best response is computed against fixed choices of the others. It is not a guess about what they will do. Whether the others actually play those choices is what a solution concept, in chapter 4.3, decides.

Lesson: Best responses and unilateral deviation

COMMON MISTAKE Payoff pairs are written (row player, column player). Swapping them is the most common slip in exam tables.

Lesson: From intuition to research

4.2 Modelling games: Cournot, Bertrand and the streetlamp

The modelling recipe, Cournot and Bertrand duopoly, and the streetlamp public good with free riding.

KEY IDEAS

- **Recipe** Players, strategies (with their range), payoffs split into cases. Game = $\langle N, S_i, v_i \rangle$.
- **Cournot** P = a − bq. $s_i = (a - c_i - bs_j)/2b$, $s^* = (a - c)/3b$. a = 100, c = 10: 30 each, P = 40, profit 900.
- **Asymmetric Cournot** $s_1^* = (a - 2c_1 + c_2)/3b$. Practice F: 18 and 12, P = 30, profits 324 and 144.
- **Bertrand** $S_i = [0, a]$, $q = (a - p)/b$. Cheaper takes all, tie splits (divide by 2b), dearer gets $-C_i(0)$. Paradox: p = c.
- **Streetlamp** $v_i = 3 \cdot 1[\Sigma \geq 2] - s_i$, 8 profiles. Pay only when pivotal. Stable: exactly two payers, or nobody.

FORMULAS

Inverse demand

$$P(q) = a - bq \text{ if this is } \geq 0, \text{ and } 0 \text{ otherwise}$$

q is the total quantity of both firms. a and b are common knowledge.

Lesson: Cournot: competing in quantities

Cournot payoff Exam favourite

$$v_i(s_i, s_j) = (a - b(s_i + s_j))\, s_i - C_i(s_i), \quad \text{or } -C_i(s_i) \text{ if the price would be negative}$$

Strategy set $S_i = [0, \infty)$, so the game is infinite.

Lesson: Cournot: competing in quantities

Best response, linear cost c_i Exam favourite

$$s_i = \frac{a - c_i - b\, s_j}{2b}$$

From $\partial v_i/\partial s_i = a - 2bs_i - bs_j - c_i = 0$. Your best output falls by half of the rival's.

Lesson: Cournot: competing in quantities

Symmetric equilibrium Exam favourite

$$s^* = \frac{a - c}{3b}$$

Put $s_j = s^*$ in the best response: $3bs^* = a - c$.

Lesson: Cournot: competing in quantities

Quantity from price

$$q = \frac{a - p}{b}$$

Turn the demand rule around: p = a − bq gives bq = a − p.

Lesson: Bertrand: competing in prices

Bertrand payoff Exam favourite

$$v_i = p_i \frac{a - p_i}{b} - C_i\left(\frac{a - p_i}{b}\right) \text{ if cheaper, half the market (divide by } 2b) \text{ if tied, } -C_i(0) \text{ if dearer}$$

Three cases. The $-C_i(0)$ term is a fixed cost paid even when nothing sells.

Lesson: Bertrand: competing in prices

Streetlamp payoff Exam favourite

$$v_i(s) = 3 \cdot 1[\textstyle\sum_j s_j \geq 2] - s_i, \quad s_i \in \{\{0, 1\}\}$$

The indicator 1[·] is 1 when the statement is true and 0 otherwise. Split on the sum, not on the eight profiles one by one.

Lesson: The streetlamp: a public good

TABLES

Table: p_1 (with $p_2 = 40$) · case · quantity · profit

p_1 (with $p_2 = 40$)	Case	Quantity	Profit
30	cheaper	70	30 × 70 − 700 = 1,400
39.9	cheaper	60.1	2,397.99 − 601 = 1,796.99
40	tie	30	1,200 − 300 = 900
50	dearer	0	0

Lesson: Bertrand: competing in prices

Table: others who pay · you pay · you do not pay · better for you

Others who pay	You pay	You do not pay	Better for you
0	−1 (not built)	0	Do not pay
1	2 (you complete it)	0	Pay, you are pivotal
2	2	3 (free ride)	Do not pay

Lesson: The streetlamp: a public good

TIPS AND TRAPS

IN THE EXAM The Bertrand paradox: with equal unit cost c, the only stable prices are $p_1 = p_2 = c$. Two firms are enough to drive profit to zero, while Cournot firms keep the price at 40, well above cost 10.

Lesson: Bertrand: competing in prices

COMMON MISTAKE The Cournot equilibrium is not the joint best outcome for the firms. Each firm ignores that its extra output lowers the other's price.

Lesson: From intuition to research

4.3 Solution concepts and Pareto optimality

What solution concepts are, the assumptions behind them, and Pareto optimality with worked games and the zero sum proof.

KEY IDEAS

- **Solution concept** Narrows profiles to reasonable ones. Its pick is an equilibrium. Not a physics equilibrium.
- **Four assumptions** Rational (maximize), intelligent (know the game), common knowledge of both, self enforcement (no outside enforcer).
- **Good concept** Existence, uniqueness, invariance. "Anything can happen" exists but is not unique.
- **Pareto** s dominates s': ≥ for all, > for one. PD: only (C, C) fails. Marketing War: (L,L), (L,M), (M,L). Zero sum: all optimal.
- **Altruism** $v_i + \frac{1}{2}v_j$ gives (−3,−3), (−5.5,−3.5), (−3.5,−5.5), (−6,−6). Mum becomes best. Pareto is not fairness.

FORMULAS

Pareto dominance Exam favourite

$$s \text{ Pareto dominates } s' \iff v_i(s) \geq v_i(s') \text{ for all } i, \text{ and } > \text{ for at least one } j$$

Nobody is worse off and someone is strictly better off.

Lesson: Pareto optimality

Pareto optimal Exam favourite

$$s' \text{ is not Pareto dominated by any profile}$$

Also called Pareto efficient.

Lesson: Pareto optimality

Altruistic payoff Exam favourite

$$v_i^{\text{alt}}(s) = v_i(s) + \tfrac{1}{2}\, v_j(s)$$

Your own jail term at full weight plus half of the other's. Not an extra penalty added on top.

Lesson: Zero sum games, altruism and the practice problems

TABLES

Table: candidate concept · prisoner's dilemma answer · verdict

Candidate concept	Prisoner's Dilemma answer	Verdict
"Anything can happen"	All four profiles	Always exists, but gives no advice or prediction
Play a strategy that is best whatever others do	(C, C)	Sharp and useful, but not always available
Nash equilibrium (later in the course)	(C, C)	Exists far more often

Lesson: What a solution concept is for

Table: assumption · meaning in this course

Assumption	Meaning in this course
Rational	Each player maximizes her own (expected) payoff
Intelligent	Each player knows every action, the outcome map and everyone's preferences, as well as the analyst does
Common knowledge	That everyone is rational and intelligent is itself common knowledge
Self enforcement	The equilibrium must hold without any outside enforcer

Lesson: What a solution concept is for

Table: property · question it asks

Property	Question it asks
Existence	Does it give an answer for every game?
Uniqueness	Does it pick a small set, ideally one profile? "Anything can happen" fails badly here.
Invariance	Does a small change in payoffs that keeps every ranking leave the answer the same?

Lesson: What a solution concept is for

Table: test · how to apply it

Test	How to apply it
Direct test	Search for a profile that is at least as good for everyone and strictly better for someone. If one exists, the candidate is not optimal.
Improvement test	Try to make one player strictly better off. If every such move hurts someone, the candidate is optimal.

Lesson: Pareto optimality

Table: profile · original · with altruism ½

Profile	Original	With altruism ½
(M, M)	(−2, −2)	(−3, −3)
(M, C)	(−5, −1)	(−5.5, −3.5)
(C, M)	(−1, −5)	(−3.5, −5.5)
(C, C)	(−4, −4)	(−6, −6)

Lesson: Zero sum games, altruism and the practice problems

TIPS AND TRAPS

COMMON MISTAKE Pareto optimality says nothing about fairness. (2, 8) and (8, 2) are Pareto optimal next to the balanced (6, 6). It only rules out waste.

Lesson: Pareto optimality

COMMON MISTAKE Pareto optimal and fair are different ideas. Pareto says nothing about how the total is shared.

Lesson: From intuition to research

STEP 5 Advanced AI · Dominance and iterated elimination

5.1 Dominance and the dominant strategy equilibrium

Strict and weak dominance, the four dominance words, the dominant strategy equilibrium with its uniqueness proof, and its limits.

KEY IDEAS

- **Strict domination** $s_i \rhd s'_i$: $v_i(s_i, s_{-i}) > v_i(s'_i, s_{-i})$ for every s_{-i}. Row player: first numbers across columns. Column player: second numbers down rows.
- **Vocabulary** Dominating/dominated: beats/is beaten by at least one. Dominant/submissive: beats/is beaten by all. One dominator is enough to delete.
- **Partial order** $\rhd$ is irreflexive, asymmetric, transitive, not complete (RPS, M and H in the Marketing War). Weak: $\geq$ (textbook adds $>$ once).
- **DSE** Everyone plays a strictly dominant strategy. Unique if it exists. PD (C, C), altruistic PD (M, M), RPS and Marketing War none.
- **Rationality** A rational player never plays a strictly dominated strategy under any belief. Increasing transforms and $h(s_{-i})$ do not change dominance.

FORMULAS

Strict domination Exam favourite

$$s_i \rhd_i s'_i \iff v_i(s_i, s_{-i}) > v_i(s'_i, s_{-i}) \text{ for every } s_{-i}$$

Only player i's own payoffs are compared, once for each situation the others can create.

Lesson: Strict domination

Weak domination

$$s_i \trianglerighteq_i s'_i \iff \geq \text{ for every } s_{-i}$$

Textbooks add "and $>$ for at least one s_{-i}". State which version you use in an exam.

Lesson: Strict domination

Strict dominant strategy equilibrium

Exam favourite

$$s^D \text{ with } s_i^D \text{ strictly dominant for every player } i$$

Everyone plays the strategy that is best against everything. No one needs to predict anyone.

Lesson: The dominant strategy equilibrium

Rational players and dominated strategies

Exam favourite

$$E_\mu[v_i(s_i)] - E_\mu[v_i(s'_i)] = \sum \mu(s_{-i})[v_i(s_i, s_{-i}) - v_i(s'_i, s_{-i})] > 0$$

Theorem 4: whatever the belief μ, a strictly dominated strategy has lower expected payoff, so a rational player never uses it.

Lesson: The dominant strategy equilibrium

TABLES

Table: term · needs · meaning for strategy s_i

Term	Needs	Meaning for strategy s_i
Dominating	at least one	s_i beats some other strategy
Dominated	at least one	some other strategy beats s_i
Dominant	all others	s_i beats every other strategy
Submissive	all others	every other strategy beats s_i

Lesson: Dominating, dominated, dominant, submissive

Table: strategy · dominating · dominated · dominant

Strategy	Dominating	Dominated	Dominant	Submissive
PD: C	yes	no	yes	no
PD: M	no	yes	no	yes
Marketing: M	yes	no	no, does not beat H	no
Marketing: L	no	yes	no	no, H does not beat it
Marketing: H	no	no	no	no
RPS: any	no	no	no	no

Lesson: Dominating, dominated, dominant, submissive

Table: property · strict dse · why

Property	Strict DSE	Why
Existence	Fails	RPS, Marketing War, Cournot, Bertrand, streetlamp have none
Uniqueness	Holds	Theorem 3 (fails for the weak version)
Invariance	Fails	Altruism ½ flips (C, C) to (M, M)
Pareto optimality	Fails	PD picks (C, C), the only wasteful profile

Lesson: The dominant strategy equilibrium

TIPS AND TRAPS

COMMON MISTAKE The mistake made twice in class: checking player 1 by mixing in the second number of each cell, or by moving down a column. Rule: for the row player compare first numbers, moving across columns. For the column player compare second numbers, moving down rows. Moving along the other player's strategies is exactly "for every s_{-i}".

Lesson: Strict domination

COMMON MISTAKE Strict and weak dominance behave differently. Removing weakly dominated strategies can remove equilibria and depends on the order of removal.

Lesson: From intuition to research

5.2 Iterated elimination of dominated strategies

The IESDS algorithm, the slide 11 and Marketing War examples, theorems about IESDS and Nash equilibrium, and why weak elimination depends on order.

KEY IDEAS

- **Algorithm** Each round delete every strictly dominated strategy of every player, comparing only survivors. Stop when nothing is dominated.
- **Common knowledge** Round k uses k layers of "I know that you know". Real people manage about 1 to 3 layers.

- **Slide 11** R ▷ C, then U ▷ M and D, then L ▷ R. Outcome (U, L) = (4, 3), self enforcing but Pareto dominated by (9, 6).
- **Theorems** DSE found in one round. Every NE survives. Unique survivor = unique NE. Strict IESDS is order independent.
- **Limits** Marketing War leaves {M, H}², RPS deletes nothing. Weak IESDS depends on order: (2, 1) or (1, 1).

FORMULAS

IESDS, round k — Exam favourite

$$DS_i^k = \{\{\, s_i \in S_i^k : \text{some } s'_i \in S_i^k \text{ beats it against every } s_{-i} \in S_{-i}^k \,\}\}$$

Compare only against strategies that still survive.

Lesson: Why we may eliminate again and again

Update — Exam favourite

$$S_i^{k+1} = S_i^k \setminus DS_i^k$$

Remove the elements. Stop when no player has a dominated strategy. Every profile of survivors is a prediction.

Lesson: Why we may eliminate again and again

Nash equilibrium (preview) — Exam favourite

$$v_i(s_i^*, s^*_{-i}) \geq v_i(s_i, s^*_{-i}) \text{ for every } i \text{ and every } s_i$$

No player gains by a unilateral deviation. The course defines it fully in later lectures.

Lesson: Theorems about IESDS

TABLES

Table: layer · reasoning on the slide 11 game

Layer	Reasoning on the slide 11 game
0	Player 2 is rational, so she never plays column C (R beats it)
1	Player 1 knows player 2 is rational, so he ignores C. In the smaller game M and D become dominated
2	Player 2 knows player 1 knows she is rational, so she expects only row U. Now R is dominated
k	Each round of elimination uses one more layer of "I know that you know that..."

Lesson: Why we may eliminate again and again

Table: slide 11 game · l · c · r

Slide 11 game	L	C	R
U	4, 3	5, 1	6, 2
M	2, 1	8, 4	3, 6
D	3, 0	9, 6	2, 8

Lesson: Worked examples: the Marketing War and slide 11

Table: theorem · statement

Theorem	Statement
5	If a strict dominant strategy equilibrium exists, IESDS leaves exactly it after one round
6	Every Nash equilibrium survives IESDS
7	In a finite game, if IESDS leaves one profile, it is the unique Nash equilibrium
8	For strict dominance in finite games, the order of deletion does not change the survivors (Gilboa, Kalai and Zemel 1990)

Lesson: Theorems about IESDS

Table: order game · l · r

Order game	L	R
T	1, 1	0, 0
M	1, 1	2, 1
B	0, 0	2, 1

Lesson: Theorems about IESDS

Table: property · iesds · why

Property	IESDS	Why
Existence	Holds	A finite strict partial order always has an undominated element
Uniqueness	Fails	Marketing War leaves four profiles, RPS deletes nothing
Stronger than DSE	Yes	Finds any DSE in one round and solves slide 11, which has none
Needs	Common knowledge	One more layer of "knows that knows" per round
Pareto optimal	Fails	Slide 11 ends at (4, 3), dominated by (9, 6)

Lesson: Theorems about IESDS

TIPS AND TRAPS

REMEMBER Two small slip ups on slide 9: the dominator must range over the current set S_i^k, and the update is S_i^k minus DS_i^k, not minus a set that contains DS_i^k. Slide 6 also says "strictly" where it means "weakly" in two places. The formulas are the correct part.

Lesson: Why we may eliminate again and again

COMMON MISTAKE IESDS does not always leave one outcome. In many games, such as the restaurant game, nothing is dominated and every strategy survives.

Lesson: From intuition to research

5.3 Dominance at work: Cournot, auctions and voting

Cournot solved by iterated elimination, truthful bidding in second price auctions, the median voter, and where weak dominance misleads.

KEY IDEAS

- **Cournot IESDS** BR = max{0, (a − c − bq)/2b}. Rounds: [0, 45], [22.5, 45], [22.5, 33.75], … halving to (a − c)/3b = 30. E1: 40, price 50, profit 1,600.
- **Second price auction** Payoff v − B if b > B. Bidding v weakly dominates every bid (never worse, sometimes better). Strategy proof.
- **First price auction** Payoff v − b if you win. Overbidding weakly dominated, but bidding v not dominant: bidders shade.
- **Location and prices** Hotelling 1 to 5: delete 1, 5 then 2, 4, both at 3 (median). Discrete Bertrand: strict deletes nothing, weak ends at (2, 2) losing (1, 1), (3, 3).
- **Weak only** Below cost Bertrand prices and the guess game: weakly but not strictly dominated, so strict IESDS cannot start.

FORMULAS

Profit with the rival fixed

$$\pi_i = (a - c - bq_j)\, q_i - b\, q_i^2$$

An upside down parabola in q_i, with its peak at the best response.

Lesson: IESDS solves the Cournot duopoly

Best response

Exam favourite

$$\mathrm{BR}_i(q_j) = \max\{\{\, 0,\ \frac{a - c - bq_j}{2b} \,\}\}$$

With a = 100, b = 1, c = 10 this is max{0, (90 − q_j)/2}.

Lesson: IESDS solves the Cournot duopoly

Without calculus

$$Kq - bq^2 = -b(q - \frac{K}{2b})^2 + \frac{K^2}{4b}$$

Completing the square shows the peak at q = K/2b, where K = a − c − bq_j.

Lesson: IESDS solves the Cournot duopoly

Second price (Vickrey)

Exam favourite

$$\text{payoff} = v - B \text{ if } b > B, \text{ otherwise } 0$$

The winner pays the second highest bid. Bidding b = v weakly dominates every other bid.

Lesson: Auctions: when truth telling is dominant

First price

Exam favourite

$$\text{payoff} = v - b \text{ if } b > B, \text{ otherwise } 0$$

The winner pays her own bid. Overbidding is weakly dominated, but bidding v is not dominant.

Lesson: Auctions: when truth telling is dominant

TABLES

Table: second price, value v · rival bid b below v · b between v and the overbid · b above both

Second price, value v	Rival bid B below v	B between v and the overbid	B above both
Bid v	win, v − B	lose, 0	lose, 0
Overbid b > v	win, v − B (same)	win, v − B < 0 (worse)	lose, 0

Lesson: Auctions: when truth telling is dominant

Table: puzzle · answer

Puzzle	Answer
F1. Streetlamp with refunds when the lamp is not built	Contributing ties with not contributing when nobody else pays, wins with one other payer, and loses with two. Still no dominance. Refunds remove the risk but not the free riding.
F2. Two players guess an integer in [0, 100]. Closer to two thirds of the average wins	The lower guess always wins, so 0 weakly dominates every other guess, but not strictly: if the rival guesses above you, both 0 and your guess win.

Lesson: Voting, prices and puzzles

TIPS AND TRAPS

COMMON MISTAKE In a second price auction you do not pay your own bid. Students who forget this conclude that shading the bid is wise.

Lesson: From intuition to research

6.1 Quiz 1 solved: all four questions

Worked answers to Quiz 1: B, A, A and B under the standard reading, and (K, 0, 0).

FORMULAS

Profit of firm i

$$\pi_i = \frac{a\, q_i}{q_i + q_j + 1} - c\, q_i$$

Price times own quantity, minus cost.

Lesson: Question 1: Cournot with a curved demand

First order condition Exam favourite

$$\frac{a(q_j + 1)}{(q_i + q_j + 1)^2} = c$$

The derivative of a $q_i/(q_i + q_j + 1)$ is $a(q_j + 1)/(Q + 1)^2$.

Lesson: Question 1: Cournot with a curved demand

Symmetric equilibrium Exam favourite

$$q^* = \frac{(a - 4c) + \sqrt{a^2 + 8ac}}{8c}$$

From $a(q + 1) = c(2q + 1)^2$, a quadratic $4cq^2 + (4c - a)q + (c - a) = 0$.

Lesson: Question 1: Cournot with a curved demand

Joint monopoly Exam favourite

$$Q^* = \sqrt{\frac{a}{2c}} - 1$$

From $a/(Q + 1)^2 = 2c$. Positive exactly when $a > 2c$.

Lesson: Question 1: Cournot with a curved demand

Buy one more unit when Exam favourite

$$\text{marginal value} \geq \text{price}$$

Because you value money, a unit is worth buying only if it is worth more to you than what it costs.

Lesson: Question 2: bananas and candy

Buy when Exam favourite

$$0.4\sqrt{400 - p + 900} + 0.6\sqrt{400 - p + 10} \geq \sqrt{400} = 20$$

Expected utility of final wealth after paying p and receiving the prize, against keeping ₹400.

Lesson: Question 3: buying a lottery ticket

Break even price Exam favourite

$$p^* \approx 267.99$$

Found by bisection. Below p* she buys, above it she does not.

Lesson: Question 3: buying a lottery ticket

Objective

$$v = x_1^2 + \tfrac{1}{2}\, x_2^2 + \tfrac{1}{4}\, x_3^2$$

A convex function on the triangle $x_1 + x_2 + x_3 = K$.

Lesson: Question 4: consumption with a convex payoff

Answer Exam favourite

$$(x_1, x_2, x_3) = (K, 0, 0), \quad v = K^2$$

Spend everything in period 1.

Lesson: Question 4: consumption with a convex payoff

TABLES

Table: unit · marginal value · price · worth buying?

Unit	Marginal value	Price	Worth buying?
Banana 1	$1.20	$0.50	Yes, +0.70
Banana 2	$0.60	$0.50	Yes, +0.10
Banana 3	$0.30	$0.50	No, −0.20
Candy 1	$0.40	$0.25	Yes, +0.15
Candy 2	$0.20	$0.25	No, −0.05

Lesson: Question 2: bananas and candy

TIPS AND TRAPS

IN THE EXAM The trap in S1 is pattern matching. (a − c)/3 is the textbook Cournot answer, but it comes from demand a − bq. With a different demand curve you must redo the first order condition.

Lesson: Question 1: Cournot with a curved demand

REMEMBER The answer depends on the numbers, not on a rule like "always spend" or "never spend". With $3 you would still stop at $1.25, because the third banana and the second candy are worth less than they cost.

Lesson: Question 2: bananas and candy

COMMON MISTAKE This question has two true options under the standard reading (prizes added to wealth after paying p). If only one option may be marked, A is the safer choice because it covers every price below 18, and B is a special case. It is worth asking the instructor whether the question allows more than one answer.

Lesson: Question 3: buying a lottery ticket

6.2 Exam revision: from school to research universities

Exam style questions from school level to research universities, with worked answers, a formula sheet and a mock quiz.

FORMULAS

Expected value Exam favourite

$$E[X] = \Sigma\, p_i\, x_i$$

Weight each prize by its chance and add.

Lesson: Tier 1: school and first year

Present value Exam favourite

$$PV = \Sigma\, \delta^t\, x_t$$

A payment t periods away is worth δ^t of its face value today.

Lesson: Tier 1: school and first year

Strict orderings

$$n!$$

The number of ways to rank n outcomes with no ties.

Lesson: Tier 1: school and first year

TABLES

Table: tier · typical format · what earns the marks · where to practise

Tier	Typical format	What earns the marks	Where to practise
School and first year	Short answers, expected values, simple tables	A clear definition with an example, careful arithmetic	Lesson 2
University semester and IIT course exams	Numericals by hand on trees, lotteries and games	Formula first, every step shown, the answer stated with its meaning	Lesson 3
NPTEL and GATE style papers	Multiple choice, multiple select and numerical answer type	Speed, spotting the trap option, exact rounding	Lesson 4
MIT and Cambridge style	Proofs, derivations and critique of a model	Assumptions named, a complete argument, when it breaks	Lesson 5

Lesson: How Advanced AI is examined

Table: quick check · use it to

Quick check	Use it to
A strictly dominant strategy equilibrium, if it exists, is unique	Reject any option listing two
Affine change $a\,u + b$ with $a > 0$ keeps every expected utility ranking	Reject options where a rescaled utility flips a choice
Concave u means CE < EV and a positive risk premium	Spot a risk averse answer at a glance
VOI is never negative	Reject a negative VOI at once
IESDS order does not change the result, iterated weak dominance can	Answer order questions in one line
Linear Cournot $q^* = (a - c) \div 3b$, Bertrand price = c	Compute in seconds

Lesson: Tier 3: multiple choice and numerical answer papers

Table: topic · formula

Topic	Formula
Rational preference	complete and transitive
Utility representation	$x \succeq y$ if and only if $u(x) \geq u(y)$
CDF	$F(x) = P(X \leq x)$, non decreasing, from 0 to 1
First order stochastic dominance	$F_A(x) \leq F_B(x)$ for all x
Expected utility	$E[u] = \Sigma\, p_i\, u(x_i)$
Certainty equivalent, risk premium	$u(CE) = E[u]$, $RP = E[X] - CE$
Present value	$PV = \Sigma\, \delta^t\, x_t$, perpetuity $\delta \div (1 - \delta)$ per unit from next period
ln consumption, two periods	$x_1 = K \div (1 + \delta)$, $x_2 = \delta\, K \div (1 + \delta)$
Value of information	VOI = E[max with information] − max E[without]
Linear Cournot duopoly	$q^* = (a - c) \div 3b$, monopoly $Q = (a - c) \div 2b$
Bertrand with equal cost	$p^* = c$
Pareto optimal	no other profile makes someone better and nobody worse

Lesson: Formula sheet and last minute checks

TIPS AND TRAPS

REMEMBER

Try the presets stag and pennies in the game above. Stag has two equilibria and pennies has none in pure strategies.

Lesson: Tier 1: school and first year

IN THE EXAM Round only at the very end. A numerical answer type key often has a narrow range such as 52.6 to 52.7.

Lesson: Tier 3: multiple choice and numerical answer papers

IN THE EXAM In proofs, name each assumption when you use it: completeness, transitivity, the vNM axioms, common knowledge of rationality. Examiners often ask what breaks when one fails.

Lesson: Tier 4: proofs, derivations and critique